RISC-V Atomic Cryptography Extension (Zl?)

Author: Roberto Avanzi (Lead), Luis Fiolhais, Radim Krčmář

Version 0.0, 6/2025: This document is under development. Expect potential changes. Visit http://riscv.org/spec-state for further details.

Table of Contents

List of tables . . . 1
List of listings . . . 2
Acronyms . . . 3
Copyright and license information . . . 7
Contributors . . . 8
1. Introduction . . . 9
1.1. Features . . . 9
1.2. Non-Goals . . . 10
1.3. TODOs . . . 12
1.4. Open Discussion Points . . . 12
2. Unprivileged Architecture . . . 13
2.1. Fundamental Concepts and Definitions . . . 13
2.2. Extensions Overview . . . 14
2.2.1. Dependencies on Other Extensions . . . 15
2.3. Programmer-Visible Architectural State . . . 15
2.3.1. Context Registers . . . 15
2.3.2. ACE Input/Output Buffer . . . 16
2.3.3. Metadata Header . . . 17
2.3.3.1. *Algorithm* and *AlgorithmPolicy* Fields . . . 17
2.3.3.2. *SCProtection* Field . . . 18
2.3.3.3. *StateNumber* Field . . . 18
2.3.3.4. *Locality* Field . . . 19
2.3.3.5. *UsagePolicy* Field . . . 20
2.3.3.6. *AdditionalData* Field . . . 20
2.3.3.7. *ExpirationDate* Field . . . 21
2.4. CSRs . . . 21
2.4.1. `aceerror` . . . 21
2.4.2. `acemarchid` . . . 22
2.4.3. `acestart` . . . 22
2.4.4. `aceiobuflen` . . . 23
2.4.5. `aceiobuftop` . . . 23
2.4.6. `acesequence` . . . 23
2.4.7. `aceactivecr` . . . 24
2.4.8. `acehtrapmask` . . . 24
2.4.9. `acehtrapvector` . . . 24
2.5. Instructions . . . 24
2.5.1. ace.prov . . . 25
2.5.2. ace.clear/ace.reset . . . 26

2.5.3. ace.export . . . 26
2.5.4. ace.import/ace.import.S . . . 27
2.5.5. ace.exec . . . 29
2.5.6. ace.state . . . 30
2.5.7. ace.error . . . 31
2.5.8. ace.size . . . 31
2.5.9. ace.avail . . . 33
2.5.10. ace.restrict . . . 33
2.5.11. ace.clone . . . 35
2.5.12. ace.input . . . 36
2.5.13. ace.output . . . 37
2.6. Provisioning, Export and Import of Cryptographic Contexts . . . 38
2.6.1. System Keys . . . 39
2.6.2. Size Functions . . . 39
2.6.3. Interruption Points . . . 39
2.6.4. Provisioning . . . 40
2.6.5. Export and Import . . . 40
2.7. CR Invalidation . . . 44
2.8. Exception Handling via Horizontal Traps . . . 44
2.9. Algorithms . . . 45
2.9.1. Definition of an Algorithm in ACE . . . 45
2.9.2. Generic Rules . . . 47
2.9.2.1. Usage of `ace.input` and `ace.output` to Replace Vector Inputs and Outputs . . . 47
2.9.2.2. Interruptibility . . . 48
2.9.2.3. Notation in the Algorithm Descriptions . . . 48
2.9.2.4. Single-Share Key Export Rule for Threshold implementations . . . 48
2.9.2.5. Representation of System Keys in the Data Structures . . . 49
2.9.3. States for the Symmetric Algorithms Defined in this Specification . . . 49
2.9.3.1. Electronic Codebook Modes (Direct Encryption/Decryption) . . . 50
2.9.3.2. Note on Short Vectors . . . 51
2.9.3.3. Chaining/Feedback Modes . . . 52
2.9.3.4. CTR/LFSR/XCTR/XLFSR Keystream Generation . . . 52
2.9.3.5. XEX/XTS modes (one or two keys) . . . 53
2.9.3.6. Generic Tweakable Block Ciphers . . . 55
2.9.3.7. GCM Mode . . . 57
2.9.3.8. GCM with Set IV . . . 60
2.9.3.9. GCM-SIV Mode . . . 61
2.9.3.10. OCB Mode . . . 65
2.9.3.11. CMAC Functions . . . 70
2.9.3.12. Hash and MAC Functions, and XOFs . . . 72
2.9.3.13. HMAC . . . 76

2.9.3.14. SHA-3 Family Specific Rules . . . 76
2.9.3.15. Ascon-AEAD128 . . . 77
2.9.3.16. Ascon-AEAD128 with set Nonce . . . 82
2.9.3.17. Ascon-AEAD128 with Nonce Masking . . . 82
2.9.3.18. Ascon-Hash256 . . . 83
2.9.3.19. Ascon-XOF128 . . . 85
2.9.3.20. Ascon-CXOF128 . . . 85
2.9.4. Elliptic Curve Asymmetric Algorithms . . . 85
3. Privileged Architecture . . . 91
3.1. Extensions Overview . . . 91
3.2. Dependencies on Other Extensions . . . 91
3.3. CSRs . . . 91
3.3.1. ACE field in `misa` . . . 92
3.3.2. ACE Context Status in `mstatus` and `sstatus` . . . 92
3.3.3. ACE Context Status in `vsstatus` . . . 92
3.3.4. `scrstatus` for Dirtiness Tracking . . . 93
3.3.5. `vscrstatus` for Dirtiness Tracking . . . 93
3.3.6. `mcrstatus` for Dirtiness Tracking . . . 94
3.3.7. Extension of `*crstatus` for Lazy Loading . . . 94
3.3.8. Propagation of `*crstatus` Status Values . . . 94
3.3.9. U-mode Status for HFI . . . 95
3.3.10. `sacetrapmask` . . . 95
3.3.11. `macecsk` Group . . . 95
3.3.12. `macePhysBootSecret` Group . . . 95
3.3.13. `haceVirtBootSecret` Group . . . 95
3.3.14. `maceLocality` Group . . . 96
3.3.15. `haceLocality` Group . . . 96
3.3.16. `saceLocality` Group . . . 96
3.4. Trappable (Emulated) Operations . . . 96
4. Pseudocode Examples . . . 98
4.1. Electronic Codebook Modes (Direct Encryption/Decryption) . . . 98
4.2. CTR/LFSR/XCTR/XLFSR Keystream Generation . . . 98
4.3. XEX/XTS modes (one or two keys) . . . 99
4.4. GCM Mode . . . 99
4.5. GCM-SIV Mode . . . 101
4.6. OCB Mode . . . 103
4.7. CMAC Functions . . . 104
5. Annexes . . . 106
5.1. Programmable CSKs vs. ID-based Usage Control vs. Localities . . . 106
5.2. Architectural Model . . . 106
5.3. Threat Model . . . 107

Index 109
Bibliography 110

List of tables

list-of::table[hide_empty_section=true, enhanced_rendering=true]

List of listings

list-of::listing[hide_empty_section=true, enhanced_rendering=true]

Acronyms

Acronym	Description
ACE	Atomic Cryptography Extension
ACEIOBUF	ACE Input/Output Buffer
AE	Authenticated Encryption
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
ASID	Application Space Identifier
CBC	Cipher Block Chaining (mode of encryption)
CC	Cryptographic Context
CFB	Cipher Feedback (mode of encryption)
CMAC	Cipher-based Message Authentication Code
CR	Context Register
CPU	Central Processing Unit
CRF	Context Register File
CSK	Context Sealing Key
CSR	Control and Status Register
CTR	Counter (mode of encryption)
DIEL	Data-Independent Execution Latency
ECB	Electronic Codebook (mode of encryption)
EPC	Exception Program Counter
FIB	Focused Ion Beam
FP	Floating-Point
GCM	Galois Counter Mode
GPA	Guest Physical Address
GPR	General-Purpose Register
GVA	Guest Virtual Address
hart	Hardware Thread
HV	Hypervisor
ISE	Instruction Set Extension
IV	Initialization Vector
impdef	Implementation-defined
HLOS	High-Level Operating System

Acronym	Description
HV	Hypervisor
KEM	Key Encapsulation Mechanism
KMA	Key Management Applet
LF	Locker File
LFSR	Linear Feedback Shift Register (also: LFSR-based mode of operation)
LS	Locality Secrets
LST	Locality Secrets Table
MAC	Message Authentication Code
MM	Machine Mode
NOP	No Operation
OFB	Output Feedback (mode of encryption)
OS	Operating System
PA	Physical Address
PCBC	Propagating Cipher Block Chaining (mode of encryption)
PI	Provisioning Input
PRF	Pseudo-Random Function
PRP	Pseudo-Random Permutation
PRW	Privileged Read-Write
RUP	Release of Unencrypted Plaintext
SCA	Side-Channel Analysis
SCC	Sealed Cryptographic Context
SEM	Scanning Electron Microscope
SIV	Synthetic Initialization Vector
SKT	System Keys Table
TBC	Tweakable Block Cipher
TEE	Trusted Execution Environment
TPM	Trusted Platform Module
VA	Virtual Address
VMID	Virtual Machine Identifier
WARL	Write Anything Read Legal
WARZ	Write Anything Read-as Zero
XCTR	`xor`ed Counter (mode of encryption)
XEX	Xor-Encrypt-Xor mode of operation

Acronym	Description
XLFSR	`xor`ed Linear Feedback Shift Register (mode of encryption)
XOF	eXtendable Output Function
XTS	XEX mode with cipherText Stealing

Pseudocode Notation

- `0` denotes a single bit equal to zero.
- `1` denotes a single bit equal to one.
- `bin(n,m)` = binary representation of `n` to `m` bits (`m` least significant bits, or sign extended).
- `Galoismul(a, b)` = multiplication of `a` and `b` in an finite field implied by the context.
- `len_in_bits(M)` = length of `M` in bits.
- `len_in_bytes(M)` = length of `M` in bytes.
- `lsb(x)` = least significant bit of `x`.
- `m[i .. j]` = bytes from the `i`-th to the `j`-th of byte array `m`.
- `m[j:i]` = bit-slice from the `i`-th to the `j`-th bit of `m`.
- `Montmul(a, b)` = Montgomery multiplication of `a` and `b` in a ring implied by the context.
- `msb(x)` = most significant bit of `x`.
- `ones(i)` = sequence of `i` ones.
- `zeros(i)` = sequence of `i` zeros.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2025 by RISC-V International.

This document is in the Development state.

Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.
Currently, there is no separation between the ISA and non-ISA parts of the architecture, and the language may not reflect Risc-V's guidelines.

Contributors

This RISC-V specification has been contributed to directly or indirectly by:

- Roberto Avanzi <ravanzi@qti.qualcomm.com>
- Nicolas Brunie <nicolas.brunie@sifive.com>
- Ruud Derwig <Ruud.Derwig@synopsys.com>
- Luis Fiolhais <luis.fiolhais@ext.semidynamics.com>
- Nick Kossifidis <mick@ics.forth.gr>
- Radim Krčmář <rkrcmar@qti.qualcomm.com>
- G. Richard Newell <richard.newell@microchip.com>
- Jan Thoma <jan.thoma@codasip.com>
- Tolga Yalcin <tolga.yalcin@qti.qualcomm.com>
- Yan Yan <yan.yan@codasip.com>

Chapter 1. Introduction

This brief introduction to the *Atomic Cryptography Extension* (ACE) is not part of the specification.

The ratification plan can be found here.

1.1. Features

ACE defines an interface to high-performance cryptographic operations without giving software access to the keys. Its development has been prompted by use cases such as content protection (including local LLM decryption), link and storage encryption, secure boot, and attestation.

ACE's main features are:

- To prevent key misuse, ACE combines cryptographic secrets (e.g., key material) with metadata into an indivisible structure called a *Cryptographic Context* (CC). Using the metadata, the architecture restricts the use of the secret material to a specific algorithm, and may optionally enforce usage constraints based on the process, VM, security state, boot cycle, or other policies. An expiration date mechanism is defined as well, to invalidate a key at a specific time.
- To create and use CCs, ACE introduces the concept of *Context Registers* (CRs)–architectural containers that conceal secrets to software. A CC is created by writing metadata and secrets into a CR. Instead of explicitly using key values, ACE instructions operate on CRs. A CC can be provided by a different software component, or generated by the software itself, by provisioning a CR with the necessary information and then erasing the latter. This way, software can use keys even if the latter are not explicitly present in the process's memory-and leverage increased performance or side-channel-protected implementations.
- Operations such as full AES block encryption/decryption or absorption of data blocks into a hash function are performed *atomically*, i.e., in a single step.
- To enable CC reuse and support OS features like context switching, ACE provides a secure mechanism to store a CC to memory via authenticated encryption, producing a *Sealed Cryptographic Context* (SCC). One generated, an SCC can be later re-imported into a CR. If an SCC is passed to a process in place of the value of key, the latter is not exposed in that process's memory.
- ACE provides a unified interface to multiple cryptographic primitives, avoiding the need for separate instructions per primitive.
  - Supporting only primitives is insufficient for modes like AES-GCM-SIV, XTS, and OCB, because leakage of derived keys or intermediate values can compromise security. Therefore, ACE also defines modes of operation and secure key derivation.
  - In addition to GCM-SIV, XTS, and OCB, ACE defines modes such as CTR, GCM, and CMAC. If the mode applies, we also define versions with a preset IV or nonce, and versions that are encrypt-only or decrypt-only, to reduce abuse. These can be instantiated not only with AES-128/256 but also with other block ciphers like SM4.
  - Various hash functions are architected as well.
  - The same interface can support SCA-resistant implementations of the algorithms.

    - In fact, it can also be used to provide access to public-key and post-quantum algorithms.
    - *Not all these algorithms and primitives must be supported by an ACE implementation: A minimal subset and several optional extensions are defined.*
- ACE scales from embedded to server cores with a unified ISA and ABI.
- ACE is compatible with sleep, hibernation, and VM migration.

Rationale for atomic operations

Assuming the primitive is secure, the key cannot be recovered from the input and output of the full cryptographic operation. Typical round-based ISAs, which require keys to be supplied between rounds or MAC stages, allow for easy key recovery from the input and output, even if the key came from a hidden register. In fact, even if an instruction computed 4 or 5 rounds of the AES, an attacker with access to the instruction oracle would be able to efficiently recover the key cite:[Rijndael-revised].

1.2. Non-Goals

ACE intentionally excludes the following:

- **Debugging support:** The internal representation of a CR remains concealed even during debug.
- **Variable-length complete ciphertext and plaintext inputs:** Plaintext and ciphertexts, as well as messages of which digests are computed, are processed in fixed-size blocks.
- **Complete key management:** ACE provides only architectural building blocks that can be used for key management, where the key management logic resides in trusted software (e.g., TEE applets). ACE does not support external schemes for directly provisioning keys into a key store. Instead, it will be a local trusted SW or HW component that manages external schemes and, with the thus obtained keys and policies, then configures CCs for the ACE unit.
- **Key revocation:** The present specification does not include a key revocation mechanism. While future versions may be architected to support revocation, such functionality is excluded here to avoid architectural and implementation complexity. In detail, the rationale is:
    1. ACE is designed to prioritize performance and flexibility (for example, algorithmic abstraction). Comprehensive revocation mechanisms, such as usage counters or revocation lists, would potentially degrade performance and significantly increase the complexity of an implementation.
    2. Implementing a universal revocation mechanism is impractical due to the diversity of protocols across key providers. Revocation data may become unbounded, and managing it within the ACE unit would require some form of dynamic secure memory access. This functionality is better handled by isolated software components such as enclaves, realms, or TEE applets, which are designed to integrate with the local software stack while providing strong security guarantees.
    3. Revocation typically targets root keys. Since user software generally uses derived keys, such as session keys, it suffices that revocation be handled by the trusted entity managing the root keys. The expiration date mechanism can be used to limit the lifetime of derived keys.
    4. Applications that require direct software use of root keys (for instance for link and storage encryption) usually run in protected environments.

- **Prevention of misuse by compromised processes:** ACE cannot fully prevent misuse by compromised processes, as these processes keep access to their own CCs. ACE complements existing SW vulnerability exploitation countermeasures and process isolation, but cannot replace them. Furthermore, in certain modes, such as in counter-mode encryption, the same method enables both encryption and decryption, potentially enabling misuse by malicious processes. This type of risk cannot be mitigated by ACE.

1.3. TODOs

- Revise all error codes and determine which ones are placed in `aceerror` CSR for each operation/state.
- Maybe: For the hash functions (Section 2.9.3.12), use the *AdditionalData* Metadata field to include only the part filled so far.
- Define the CSRs for Horizontal Exception Handling (Section 2.8) to handle ACE errors (Section 2.4.1).
- Define RVV-mini.
- Complete list of sub-extensions.
- Add the relevant sub-extension to each instruction, CSR, feature, algorithm.
- Add a list of algorithm numbers!

1.4. Open Discussion Points

- `acestart` should probably track **bits** not bytes, as some cryptographic algorithms, such as format preserving ones, work on bits.
- We could use `L` in this document instead of `ace` but we haven't changed it yet since a different `misa` bit and a different letter may ultimately be allocated to the extension. `L` is nice because it goes "beyond K" and is also the initial of "Locker", which could replace Cryptographic Context.
- Decide on which units to use through the spec for data, whether bytes or bits. The extra content length should probably be in number of 64-bit blocks. On the other hand, it seems reasonable to use bytes for `ace.size`, as well as `acestart`, `aceiobuflen` and `aceiobuftop`.
- Discuss `acehtrapmask`, `acehtrapvector`, and `sacetrapmask`.
- Consider the interruptibility of the SIV computation in `ace.export`.
- Everything in this specification is still valid if the number of CRs is reduced to, say, 8 or 16. The important point here is that this number is fixed, architectural, and discoverable. The indirect addressing would allow powerful self-management of CCs/CRs. Immediate values to refer to CCs are still 5 bits long because the same field can also be used to refer to a GPR, but the only valid values for the CR number are 0 to 7. Therefore, it makes sense to define an architectural value MAX_CRS that can take the values 4, 8, 16, and 32.
- Regarding Trappable (Emulated) Operations (Section 3.4): There are ongoing discussions on whether to formalize the auxiliary instructions and make them normative.
- Regarding filtering by time-stamp: hours since January 1st, 2027?

Chapter 2. Unprivileged Architecture

2.1. Fundamental Concepts and Definitions

ACE is a complex architecture and relies on certain fundamental concepts, which we introduce here. While this leads to some repetitions, we believe that it also brings better clarity to the presentation.

Cryptographic Context

The **Cryptographic Context** (CC) is the foundation of ACE. A CC is an indivisible structure consisting of a **Metadata** section and a **Content** section; all cryptographic operations performed by ACE execute within the constraints defined by a CC. The Metadata has a fixed format and specifies the cryptographic algorithm and various optional policies associated with the CC. The Content has an algorithm-specific format and holds sensitive material such as keys and internal algorithm state. The Metadata requires integrity; the Content requires both integrity and confidentiality. These requirements are architecturally enforced by ACE and by the cryptographic algorithms it implements.

Context Register

To create CCs, ACE uses **Context Registers** (CRs), per-hart architectural containers that safeguard their contents. A CR is **provisioned** by writing Metadata and Content into it; this operation binds the latter to form a CC. There is no store-like instruction to extract the Content of the CR in a manner that compromises its confidentiality. A CC may be used for cryptographic operations only while resident in a CR. Only the ACE unit may access or modify a CC residing in a CR, in accordance with architectural rules and the algorithm bound to the CC.

Sealed Cryptographic Context

To support context switching while at the same time protecting CCs outside architectural state, a CC may be **exported** from a CR as a **Sealed Cryptographic Context** (SCC), an encrypted and authenticated representation of a CC that can later be **imported** into a CR. SCC export and import use a *Context Sealing Key* (CSK), which may be programmable by M-mode. A CC sealed under one CSK cannot be re-imported under a different CSK, enabling cryptographic domain separation across spatial and temporal domains.

Object-Orientation

CCs act as **objects** exposing a uniform interface across algorithms. A CC encapsulates algorithm-specific behavior, enabling interchangeable use of different primitives (e.g., AES-128, AES-256, SM4) in identical modes (e.g., ECB, GCM-SIV, XTS) without requiring changes to compiled code. Side-channel attack (SCA) resistant variants are supported transparently.

Usage Control Policy

ACE distinguishes between CC/CR **management** and **usage** operations. Management operations include **configuration** (e.g., CR provisioning, SCC import, cloning, key derivation) and **export** of a CR as an SCC. Usage operations include cryptographic operations and state changes. Key derivation is both a configuration and a usage operation. All operations represent a form of **access** to CRs. To prevent denial-of-service scenarios, management operations, in particular configuration operations, are far less restricted than usage operations. *Usage Control* policies

restrict only certain CC/CR usage operations.

Localities

ACE can bind CCs to a **Localities**, i.e., restrict usage to specific devices or device classes, hardware configurations, physical or virtual boot cycles, and process domains at various privilege modes. This binding is achieved by using the Localities as *tweaks* in the encryption and decryption of SCCs.

2.2. Extensions Overview

This list is an early draft.

- `Zlio` ⇒ Support for `ace.input`/`ace.output` operations.
- `Zleaes128p` ⇒ Support for AES-128 encryption and decryption (all applicable modes)—the `p` at the end denotes a primitive.
- `Zleaes192p` ⇒ Support for AES-192 encryption and decryption (all applicable modes).
- `Zleaes256p` ⇒ Support for AES-256 encryption and decryption (all applicable modes).
- `Zlsm4p` ⇒ Support for SM-4 encryption and decryption (all applicable modes).
- `Zlctrm` ⇒ Support for counter mode-the `m` at the end denotes a mode.
- `Zlxctrm` ⇒ Support for XCTR mode.
- `Zlgcmm` ⇒ Support for GCM mode.
- `Zlgcmsivm` ⇒ Support for GCM-SIV mode.
- `Zlxexm` ⇒ Support for XEX and XTS modes.
- `Zlocbm` ⇒ Support for OCB mode.
- `Zlcmacm` ⇒ Support for CMAC.
- `Zlesha2h` ⇒ Support for SHA-2 (all applicable modes)—the `h` at the end denotes a hash function.
- `Zlesha3h` ⇒ Support for SHA-3 (all applicable modes).
- `Zlsm3h` ⇒ Support for SM-3 hash function (all applicable modes).
- `Zlhmacm` ⇒ Support for HMAC.
- `Zlascon` ⇒ Support for the ASCON suite.
- `Zlkned` ⇒ NIST Suite: AES Block Cipher. Depends on `Zleaes128` and `Zleaes256`.
- `Zlkn` ⇒ NIST Algorithm Suite. Depends on `Zlkned` and `Zlesha2`.
- `Zlv` ⇒ Requires V (or a suitable subset thereof). Must support for the following CSRs: `aceerror` and `acestart` (this is a dependency for Zicsr); must support the following instructions: `ace.prov`, all four forms of `ace.exec`, `ace.clear`, `ace.export`, `ace.import`, `ace.clone`, `ace.size` (and thus `ace.avail`), and `ace.restrict`; must support `Zleaes128` and `Zleaes256` in ECB and GCM-SIV modes.
- `Zlio` ⇒ Must support for the following CSRs: `aceerror`, `acestart` (this is a dependency for Zicsr), `aceiobuflen` and `aceiobuftop`; must support the following instructions: `ace.prov`, Form D of `ace.exec`, `ace.input` and `ace.output`, `ace.clear`, `ace.export`, `ace.import`, `ace.clone`, `ace.size` (and

thus `ace.avail`), and `ace.restrict`; must support `Zleaes128` and `Zleaes256` in ECB and GCM-SIV modes.

- At least one of `Zlv` or `Zlio` must be implemented. (Note that, currently, we are not considering a version of ACE that uses the GPRs in place of the vectors registers.)

2.2.1. Dependencies on Other Extensions

ACE inputs and outputs may be conveyed through vector registers or through an architectural input/output buffer (ACEIOBUF) loadable from and storable to memory; these two mechanisms are not mutually exclusive.

When vector registers are used, the full V extension is not required. ACE depends only on a minimal subset, denoted RVV-mini. RVV-mini must be fully opcode-compatible with RVV: code written for RVV-mini must execute with identical behavior under RVV. The required features are:

- Vector registers wide enough to support the inputs and outputs of the implemented algorithms;
- The `vsetvli` instruction with restricted parameters (e.g., `vsetvli zero,4,e32,m1,ta,ma` for 128-bit operations); unsupported configurations must produce defined errors;
- Unit-strided vector loads and stores; and
- Vector bitwise logical operations: `vxor`, `vand`, and `vor`.

Floating-point, fixed-point, and general arithmetic vector operations are not required. Complex operations for certain cryptographic algorithms may be performed in GPRs, with data inserted into vector registers and result extracted from them via `vins`/`vext`.

> RVV-mini need not be formally defined before ACE ratification begins; it may be defined later, with the ACE specification amended accordingly. This allows additional requirements to be collected, for example whether to mandate `vclmul.v[vx]` and `vclmulh.v[vx]` from `Zvbc`.

2.3. Programmer-Visible Architectural State

2.3.1. Context Registers

ACE defines up to 32 Context Registers (CRs), numbered 0 to 31 and denoted `K0`–`K31`. CRs are architectural, i.e., per-hart. CRs reside in a dedicated, per-hart *CR file* (CRF) accessible only to the ACE implementation.

In ACE instructions, a CR may be addressed directly, as a 5-bit immediate, or indirectly, via a GPR value. Accessing a CR with an index outside [0..31] raises an illegal instruction exception.

Unlike a conventional register, a CR can be *unconfigured*, meaning it holds no information.

We summarize the most important operations on CRs:

- Metadata and Content are loaded into a CR via `ace.prov` (Section 2.5.1).
- A CR is cleared and unconfigured using `ace.clear` (Section 2.5.2).

- `ace.export` (Section 2.5.3) exports a CR to memory as an SCC.
- An SCC may later re-imported into a CR via `ace.import` (Section 2.5.4). Import may fail if the SCC has been modified.
- `ace.exec` (Section 2.5.5) performs cryptographic operations on data blocks according to the algorithm configured in a CR.
- `ace.state` (Section 2.5.6) transitions between states of a cryptographic algorithm.

CRF capacity consumed by a CR depends on the configured algorithm; two CRs may consume different amounts. Insufficient residual CRF capacity may prevent execution of a configuration instruction (`ace.prov`, `ace.import`, `ace.clone` (Section 2.5.11); software must then free capacity by unconfiguring other CRs.

CRF capacity is implementation-defined. Implementations must provide sufficient capacity for all supported operations. At a minimum, the CRF must hold the largest possible single CR, or the largest CR pair used in cloning or derivation operations. An optional, implementation-specific mechanism for estimating required capacity is described in Section 2.5.8.

Indirect CR addressing has two microarchitectural implications:

1. Adding CR renaming would significantly increase microarchitectural complexity. However, CRs are redefined far less frequently than GPRs or FP registers (internal implied changes aside), so the absence of CR renaming is not expected to cause performance issues.
2. An additional bit in the instruction encoding selects between a 5-bit immediate CR number and a GPR-supplied CR number.

VM migration can become an issue when the destination machine has a smaller CRF than the source machine. Migration could fail, or the VM may have to let some processes terminate abnormally. A lazy-loading mechanism may be used to load CRs on demand; see Section 3.3.7.

2.3.2. ACE Input/Output Buffer

The preferred mechanism for supplying inputs to ACE operations and collecting their outputs is the vector register file of the V extension. ACE also defines an optional architectural, per-hart input/output buffer (ACEIOBUF) that may be used in place of vector registers, as described in Section 2.9.2.1. The ACEIOBUF is not part of any CR. It may reside within CRF memory or in its own memory region internal to the ACE state. Support for vector registers and for the ACEIOBUF is not mutually exclusive.

The total length and the range of the ACEIOBUF used in cryptographic instructions are controlled by the `aceiobuflen` (Section 2.4.4) and `aceiobuftop` (Section 2.4.5) CSRs.

2.3.3. Metadata Header

ACE defines two external CC formats:

1. The *Provisioning Input* (PI), loaded into a CR to create a CC; and
2. The *Sealed Cryptographic Context* (SCC), an encrypted and authenticated representation of a CC used when exporting a CR to memory for later re-import.

Both formats begin with a 64-bit Metadata Header, whose format is defined below.

Table 1. Format of the Metadata Header

Field	Width	Name	Description
[0:0]	1	*Format*	Set to 0 for the current Metadata format version.
[11:1]	11	*Algorithm*	Cryptographic algorithm (primitive, mode, or protocol). See Section 2.3.3.1.
[13:12]	2	*AlgorithmPolicy*	Allowed algorithm variants(s). See Section 2.3.3.1.
[15:14]	2	*SCProtection*	Side-channel protection level. See Section 2.3.3.2.
[16:16]	1	*KeyType*	0: key is an explicit value in Content. 1: key is an index into the system-specific System Keys Table (SKT).
[20:17]	4	*StateNumber*	Current algorithm state-machine state, encoded as an integer.
[28:21]	8	*Locality*	Zero, or an index determining a Locality Secret. See Section 2.3.3.4.
[31:29]	3	*UsagePolicy*	Usage (Control) Policy. Bits 29, 30, and 31, when set, disallow CC usage in U-mode, (V)S-mode, and H-mode, respectively.
[45:32]	14	*AdditionalData*	If non-zero, indicates an additional data segment appended to Content. The length of this additional Data segment is *AdditionalData* × 8 bytes (up to 128 KiB).
[63:46]	18	*ExpirationDate*	If non-zero, the expiration date of this CC. See Section 2.3.3.7.

Selected fields require additional explanation, provided in the following subsections.

2.3.3.1. *Algorithm* and *AlgorithmPolicy* Fields

Algorithms are encoded in an 11-bit field. Values 0–1535 are architecture-defined; the list of encodings is maintained by RVI (see Section 2.9). The remaining values are available for custom algorithms.

For encryption/decryption primitives and modes, the *AlgorithmPolicy* field indicates whether the CC may be used for encryption (lower bit set), decryption (upper bit set), or both. For asymmetric primitives supporting signatures, the lower bit permits signature generation and the upper bit permits verification. For other primitives, the *AlgorithmPolicy* field may encode additional algorithm identifiers to expand the effective encoding space of the *Algorithm* field.

2.3.3.2. *SCProtection* Field

The architecture defines the following side-channel protection levels.

Table 2. Encoding of Side-Channel Protection levels

Value	Description
0	No protection.
1	Data-independent execution latency (DIEL); first-order threshold implementation.
2	DIEL; first-order threshold, fault-tolerant implementation.
3	Custom, algorithm-dependent.

Custom algorithms offering stronger protection than encodings 0–2 shall set *SCProtection* to 3.

2.3.3.3. *StateNumber* Field

The *StateNumber* field shall be zero in a PI.

The architected *StateNumber* values common to all algorithms are defined in Table 3.

Table 3. Global CC State Numbers

Value	Mnemonic	State Name	Description
0	ace_state_initial	*initial*	CC immediately after provisioning or reset; ready for operation.
14	ace_state_invalid	*invalid*	Algorithm terminated abnormally, or interrupted provisioning/import.
15	ace_state_completed	*completed*	Algorithm has completed (not necessarily without error).

All states other than *invalid* are termed *valid*. Unless an algorithm's specification explicitly states otherwise, a transition from any valid state to *initial* is always permitted.

If an algorithm defines 16 or fewer states (including the three fixed states), the current state is stored in the *StateNumber* field. For algorithms with more than 16 states, additional state information is stored in the Content section. In all cases, the *StateNumber* field shall be set to *initial* (0), *invalid* (14), or *completed* (15) when the CC is in the corresponding state, and to a different value otherwise. These *initial*, *invalid*, or *completed* shall be unambiguously identifiable solely from the *StateNumber* field, without inspecting the Content; in other words, the values 0, 14, and 15 in *StateNumber* cannot be used together with additional information in the Content to encode additional states.

During `ace.prov` and `ace.import`, *StateNumber* is set to *invalid* for the duration of the operation and updated to the correct value only upon successful completion (see Section 2.6). By the rules of Section 2.7, a partially provisioned or imported CR cannot be exported. If a context switch occurs during such an operation, privileged code must unconfigure the CR; upon resumption, the instruction is restarted. An invalid CR with a nonzero `acestart` is recognized as an incomplete CR,

permitting resumption of provisioning or import. This state change to *invalid* does not occur during `ace.export`.

2.3.3.4. *Locality* Field

ACE maintains a table of 128-bit Locality Secrets (LS), the Locality Secrets Table (LST). Some entries are fixed or configurable only via an implementation-specific authenticated hardware procedure; these are global and shared across all ACE units in the same SoC. Other entries are per-hart architectural state programmed by privileged modes.

Table 4 lists the architected Localities. Eleven LST entries are defined: the first six constitute the *HW Binding Group*, binding SCCs to the hardware, entries 6–7 form the *Boot Session Group*, and entries 8–10 form the *SW Filter Group*. Seven of the eight *Locality* field bits are defined; bit 7 is reserved for future use or custom Localities (□ To be defined.). For each Locality we provide: index in the LST; domain (SoC-, device-, VM-wide, or per-process); configurability; bit-field encoding and value.

Table 4. Architected Localities

Idx	Field	Description	Domain	Config?	Bits	Value
No HW Binding		Independent of Chip, OEM, Device	—	—	[2:0]	0
0	SiPSecret	Identifies the designer of the SoC. Permanent.	SoC	No	[2:0]	1
1	ChipFamilySecret	Identifies specific chip/SoC model/family. Permanent.	SoC	No	[2:0]	2
2	ChipSecret	Unique for each SoC. Permanent.	SoC	No	[2:0]	3
3	OEMSecret	Provisioned by OEM to differentiate their products from the competition. Permanent, or reconfigurable via a custom hardware authenticated mechanism.	Device	HW	[2:0]	4
4	ProductSecret	Identifies device/system type/model (product). Permanent or reconfigurable like OEMSecret.	Device	HW	[2:0]	5
5	DeviceSecret	Unique device (entire system) secret. Shared across all harts in a SoC and shareable across multiple SoCs in a single device using an implementation-defined mechanism. Reconfigurable like OEMSecret.	Device	HW	[2:0]	6
No Boot Binding		Independent of Physical or Virtual Boot Session	—	—	[4:3]	0
6	PhysBootSecret	Unique per the boot session of the host hardware and its firmware stack. Configured by M-mode during the boot process of the host (see Section 3.3.12).	Device	M	[4:3]	1

Idx	Field	Description	Domain	Config?	Bits	Value
7	VirtBootSecret	Unique per the boot session of the virtual hardware and its software stack. Regenerated by H or higher at each virtual boot of a VM (see Section 3.3.13).	VM	H/M	[4:3]	2
No Binding at Modes		Independent of M-, S-, or VS-mode filters.	—	—	[6:5]	0
8	MLocality	Configurable by M to mutually isolate processes in lower-privileged modes.	S/HS	M	[6:5]	1
9	HLocality	Configurable by H to mutually isolate processes in lower-privileged modes.	VS	H	[6:5]	2
10	SLocality	Configurable by (V)S to mutually isolate processes in U-mode.	U	(V)S	[6:5]	3

Implementations are not required to populate every entry in the HW Binding Group, except for ChipSecret and DeviceSecret. If an entry is unconfigured, it is transparently replaced by the next defined entry in its chain: SiPSecret → ChipFamilySecret → ChipSecret, and OEMSecret → ProductSecret → DeviceSecret.

HLocality is active only when the H extension is enabled and V=1; otherwise, it mirrors MLocality.

> A system-key index may refer to keys with unrelated purposes on different devices, leading to unspecified behavior. Implementations shall assign usage policies at least as restrictive as the key's own native policies, and supply an appropriate *Locality* during CR provisioning.

Up to three Localities may be active concurrently: one from the HW Binding Group, one of either PhysBootSecret or VirtBootSecret, and one from the SW Filter Group. In Section 2.6 it is explained how the associated policies are enforced.

2.3.3.5. *UsagePolicy* Field

Bit 0, 1, resp., 2 of this field is ignored if U-, S-, resp., H-mode is not supported or not enabled.

2.3.3.6. *AdditionalData* Field

The Additional Data segment of the content consists of two parts: an architecture-independent part, whose length is determined by the specified algorithm, and an architecture-dependent part. One, none, or both of these parts may be empty. The first eight bytes of the architecture-dependent of the additional data segment hold a copy of the `acemarchid` CSR (Section 2.4.2) of the originating machine, and are counted within the length recorded by the *AdditionalData* field. The importing implementation uses this value to determine whether the architecture-dependent additional data can be used.

AdditionalData records the length of the *entire* Additional Data. It is the responsibility of the algorithm to determine the length of the architecture-independent part and process it. Then, if *AdditionalData* is larger than then architecture-independent part, the *implementation* will check the

value of `acemarchid` to decide whether the architecture-dependent part can be used.

> Examples of additional data are: “decryption AES keys”, i.e., the last one or two AES round keys, from which one can work backwards when performing AES decryptions; precomputed point multiples for elliptic curve scalar multiplication (their computation can be far more expensive than their encryption and decryption); and auxiliary data for side-channel-protected implementations, such as key shares or other data to accelerate share reconstruction.

2.3.3.7. *ExpirationDate* Field

If non-zero, *ExpirationDate* specifies the CC expiration time as a number of hours elapsed since the base date of January 1, 2027. The 18-bit field provides for a span of approximately 30 years (2^{18} hours). Future extended formats (with *Format* = 1) may support expanded time spans.

2.4. CSRs

ACE provides a number of unprivileged CSRs.

Table 5. ACE Unprivileged CSRs.

Address	Privilege	Field	Description
0xXXX	URW	`aceerror`	Error code of the last ACE instruction.
0xXXX	RO	`acemarchid`	ID of the microarchitecture of the ACE unit.
`vstart`	URW	`acestart`	ACE start byte index.
0xXXX	URW	`aceiobuflen`	ACE input/output buffer length.
0xXXX	URW	`aceiobuftop`	ACE input/output buffer limit for current transfers.
0xXXX	URW	`acesequence`	Sequence number of the last ACE instruction.
0xXXX	URW	`aceactivecr`	Active CR for `ace.input`/`ace.output`.
0xXXX	URW	`acehtrapmask`	▯ Discuss: Mask to determine which ACE exceptions horizontally trap.
0xXXX	URW	`acehtrapvector`	▯ Discuss: Address of the horizontal ACE trap handler.

2.4.1. `aceerror`

`aceerror` is an XLEN-bit URW CSR. Only the least significant 4 bits of `aceerror` are defined, and the top XLEN - 4 bits are WARZ. It holds the error code from the most recent ACE instruction. A read of this CSR may stall until the preceding ACE operation completes. The CSR is writable to support context switching.

The phrase “*raises ACE error X*” means that `aceerror` is set to value *X*. The same event may also trigger a vertical or horizontal trap; the handler may inspect `aceerror` to determine the error type.

Table Table 6 lists the error codes defined by ACE. The non-zero error codes are also listed in

decreasing priority order, i.e., error 1 has the highest priority, and error 10 has the lowest priority.

Table 6. Error codes and their meanings

Value	Mnemonic	Examples
0	ace_err_no_error	No error.
1	ace_err_invalid_CR	Attempted to use an *invalid*, but not expired, CR.
2	ace_err_unconf_CR	Use of an unconfigured, or partially configured CR as a source in any instruction except `ace.size`.
3	ace_err_expired_CR	Attempted to use an expired CR.
4	ace_err_import_auth	Authentication failure with `ace.import`.
5	ace_err_privilege	Violation of Usage Control policies.
6	ace_err_invalid_value	Invalid value (argument or metadata), except ace_err_unimpl or invalid state.
7	ace_err_memory	Insufficient CRF memory for configuration instruction.
8	ace_err_unimpl	Not implemented algorithm or variant. Invalid state number in `ace.state`.
9	ace_err_unconf_buf	Use of ACEIOBUF as an input or an output when it was unconfigured.
10	ace_err_auth	Authentication failure in an algorithm.

2.4.2. `acemarchid`

`acemarchid` identifies the microarchitectural version of the ACE unit, analogously to `marchid` for the hart.

2.4.3. `acestart`

`acestart` is an alias for `vstart`. ACE defines it also when the V extension is not enabled.

`acestart` tracks the progress of interruptible ACE instructions, namely `ace.prov`, `ace.import`, `ace.export`, `ace.input`, and `ace.output`. It also applies to `ace.exec` when iteratively processing a multi-block operand, as `ace.exec` may be interrupted between blocks. Only these instructions honor a nonzero `acestart`; all other instructions ignore it and are restarted, with `acestart` also being cleared. Provisioning, import, and export are resumed only under certain conditions, as explained in Section 2.6.

Since ACE does not have an *intrinsic* concept of element length, `acestart` counts the number of bytes processed so far, not the number of elements. Hardware typically writes this register on interruption. Software may write zero to force a full restart and may save and restore the register across context switches. Writing an invalid or non-supported nonzero value may raise ACE exception ace_err_invalid_value.

When a vector instruction is interrupted, the CSR follows `vstart` semantics, i.e., it counts the number of processed elements. When an ACE instruction is interrupted, the CSR follows `acestart`

semantics, namely, it counts the number of bytes processed so far.

2.4.4. `aceiobuflen`

`aceiobuflen` is an XLEN-bit URW CSR that programs the ACEIOBUF length for use by `ace.input` and `ace.output`. Writing this register-even when re-writing the old value-also zeroes the buffer and sets `aceiobuftop` to the same length and `acestart` to zero.

When `aceiobuflen` is zero, the ACEIOBUF is *unconfigured* (analogous to an unconfigured CR in state Initial). Attempting to execute with an unconfigured ACEIOBUF `ace.input` and `ace.output` raise ACE error ace_err_unconf_buf.

Writing a value larger than the implementation maximum raises ACE error ace_err_memory.

To discover the maximum buffer size supported by an implementation, or by the current residual capacity of the internal memory of the ACE unit, write all-ones to `aceiobuflen`: ACE error ace_err_memory is raised, but a subsequent read of `aceiobuflen` returns the actual maximum length; the buffer is simultaneously configured to that length, `aceiobuftop` is set accordingly, and `acestart` is set to zero.

2.4.5. `aceiobuftop`

`aceiobuftop` is an XLEN-bit URW CSR that marks the upper bound of the buffer range used by `ace.input` and `ace.output`. Together with `acestart`, it defines the active transfer window.

The constraints `acestart` ≤ `aceiobuftop` ≤ `aceiobuflen` must hold; violating either bound raises ace_err_invalid_value. Modifying `aceiobuftop` does not alter or invalidate buffer contents.

Using ACEIOBUF when `aceiobuftop` − `acestart` is zero (with `aceiobuflen` nonzero) or an invalid transfer size raises ACE error ace_err_invalid_value.

> `aceiobuftop` enables per-operation length adjustments without reconfiguring the ACEIOBUF, which can be expensive due to the fact that writing to `aceiobuflen` clears the buffer and may lead to internal garbage collection. This is useful to process an incomplete “last block” or to supply a short IV or nonce.

> The dirtiness of ACEIOBUF is not tracked. We recommend software to set `aceiobuftop` to zero once the contents of the ACEIOBUF are no longer needed. Context switching code can use this information to avoid unnecessary save and restore of ACEIOBUF contents.

2.4.6. `acesequence`

`acesequence` is an XLEN-bit URW CSR; only the low 2 bits are defined. It is zero when vector registers are used for the current ACE instruction, and takes nonzero values when `ace.input`/`ace.output` are used in place of vector registers, as described in Section 2.9.2.1.

2.4.7. `aceactivecr`

`aceactivecr` is an XLEN-bit URW CSR; only the low 5 bits are defined. It binds `ace.input`, `ace.output`, `ace.exec`, and `ace.state` to a specific CR. Valid values are 0–31 (selecting a CR) or 32 (disabled).

`aceactivecr` may only be modified when `acesequence` is zero or an illegal instruction exception is raised.

When `aceactivecr` is not 32, the CR operand in all single-CR instructions and the source CR in `ace.clone` must equal the selected value; any deviation raises an illegal instruction exception, ensuring correct instruction sequencing.

2.4.8. `acehtrapmask`

Mask that determines which ACE exceptions horizontally trap. If bit *i* of this CSR is set, then ACE error *i*+1 triggers a horizontal trap, as long as `acehtrapvector` is not NULL. If bit *i* of this CSR is clear, then ACE error *i*+1 shall not trigger a horizontal trap, but it may still trigger a vertical trap.

2.4.9. `acehtrapvector`

Address of the horizontal ACE trap handler. If this pointer is NULL, horizontal traps on ACE exceptions are disabled, regardless of the value of `acehtrapmask`.

2.5. Instructions

> The encodings presented here are preliminary. The usage of `custom-0` for load-type, `custom-1` for store-type, and `custom-2` for operational instructions aligns with the base ISA and FP instruction-set conventions.

ACE provides a set of unprivileged instructions, some of which are implemented as CSRRW operations. This section specifies their semantics and encodings.

The notation `K(Xd)` denotes the CR whose index is held in GPR `Xd`. `Kn|K(Xn)` indicates two alternative encodings: a 5-bit immediate CR index (`Kn`) or a CR number supplied by GPR `Xn`. Use of the zero register `X0` as a CR index register raises an illegal instruction exception unless explicitly indicated otherwise.

Each instruction is characterized by the following properties:

- *Modifies state*: whether the instruction may modify ACE architectural state;
- *Usage-controlled*: whether the instruction is subject to usage control policies (*UsagePolicy*);
- *Trappable*: whether the instruction may be implemented by privileged code via trap-and-emulate; and
- *Interruptible*: whether execution may be interrupted (e.g., by a context switch or page fault) and later resumed or restarted.

> Trappability refers solely to the CPU-side implementation—for example, to enable

> decomposition of certain ACE instructions into smaller custom operations. This is distinct from custom algorithm implementations within the ACE unit itself (e.g., via a secure embedded CPU with updatable firmware), which are outside the scope of this specification.

2.5.1. ace.prov

Synopsis

Provision Metadata and secrets into a CR.

Mnemonic

`ace.prov Kd|K(Xd), %offset(Xs)`

Encoding

`ace.prov` is an I-type instruction.

```
{reg:[
    { bits:  7, name: 0x0b, attr: ['custom-0'] },
    { bits:  5, name: 'context', attr: ['Kd','K(Xd)'] },
    { bits:  3, name: 'ace.prov', attr: [0x4,0x5] },
    { bits:  5, name: 'Xs' },
    { bits: 12, name: 'immed12', attr: ['offset'] },
]}
```

Description

`ace.prov` provisions a CR with Metadata and Content to create a CC. `Kd` or `K(Xd)` denotes the CR to be provisioned. `Xs` points to the region of memory containing the Metadata and the key material. The PI must be stored at an 8-byte aligned address, and its length must be a multiple of 8 bytes.

If the instruction fails, the target CR is unconfigured, with the same semantics as if `ace.clear` were invoked on the same CR (Section 2.5.2).

ACE error ace_err_unimpl is raised if the combination of *Algorithm*, *AlgorithmPolicy*, and *SCProtection* is not supported.

ACE error ace_err_invalid_value is raised if the Metadata is invalid (other than ace_err_unimpl), such as, for instance: a nonzero *StateNumber* or reserved field; or an unsupported *Locality* value.

Similarly to vector store instructions, `ace.prov` appears to other harts as a sequence of individual reads.

The operation is described in detail in Section 2.6.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **NO**.
Trappable: **YES**.

Interruptible: **YES**.

> `ace.prov`, like `ace.import`, `ace.export`, `ace.input`, and `ace.output`, which are described later, can be viewed as special kinds of load and store instructions, and therefore they do not violate the load/store architecture principle.

2.5.2. ace.clear/ace.reset

Synopsis

Clears and unconfigures a CR, or the ACE state.

Mnemonic

`ace.clear Kd|K(Xd)`
`ace.reset`

Encoding

`ace.clear`/`ace.reset` is an I-type instruction.

```
{reg:[
    { bits:  7, name: 0x5b, attr: ['custom-2'] },
    { bits:  5, name: 'context', attr: ['Ks','K(Xd)'] },
    { bits:  3, name: 'ace.clear', attr: [0x4,0x5] },
    { bits:  5, name: 0x0 },
    { bits:  5, name: 0x0 },
    { bits:  7, name: 0x0 },
]}
```

Description

`ace.clear` unconfigures the state of the CR `Kd` or `K(Xd)` and releases its resources in the CRF.

`ace.reset` unconfigures all the CRs and the ACEIOBUF. It uses the encoding for `ace.clear` with indirect CR addressing and `Xd` = `X0`.

Properties

Possibly modifies state: `ace.clear` only makes any status flag (global of per-CR) Dirty if the CR was configured before. Usage-controlled: **NO**.
Trappable:
- `ace.clear`: **NO**;
- `ace.reset`: **YES** (it can be implemented as multiple `ace.clear` and a write to `aceiobuflen`).

Interruptible: **NO**.

2.5.3. ace.export

Synopsis

Export a CR as a SCC in memory.

Mnemonic

`ace.export %offset(Xd), Ks|K(Xs)`

Encoding

`ace.export` is an S-type instruction.

```
{reg:[
    { bits:  7, name: 0x27, attr: ['custom-1'] },
    { bits:  5, name: 'immed[4:0]' },
    { bits:  3, name: 'ace.export', attr: [0x0,0x1] },
    { bits:  5, name: 'Xd' },
    { bits:  5, name: 'context', attr: ['Ks','K(Xs)'] },
    { bits:  7, name: 'immed[11:5]' },
]}
```

Description

Exports the CR `Ks` or `K(Xs)` to memory at address `Xd`, using the procedure described in Section 2.6. The SCC must be stored at an 8-byte aligned address, and its length must be a multiple of 8 bytes.

`ace.export` appears to other harts as a sequence of individual stores.

The operation is described in detail in Section 2.6.

Properties

Possibly modifies state: **NO**.
Usage-controlled: **NO**.
Trappable: **YES**.
Interruptible: **YES**.

2.5.4. ace.import/ace.import.S

Synopsis

Import a SCC from memory into a CR.

Mnemonics

```
ace.import   Kd|K(Xd), %offset(Xn)
ace.import.S Kd|K(Xd), %offset(Xn)
```

Encoding

`ace.import` is an I- (or R-) type instruction.

```
{reg:[
    { bits:  7, name: 0x0b, attr: ['custom-0'] },
    { bits:  5, name: 'context', attr: ['Kd','K(Xd)'] },
    { bits:  3, name: 'ace.import', attr: ['0    0    S','0    1    S'] },
    { bits:  5, name: 'Xn' },
    { bits: 12, name: 'immed12' },
```

```
]}
```

Description

`Kd` or `K(Xd)` denotes the CR into which the SCC is to be imported. `Xn` is the GPR containing the base address of the buffer with the source SCC. The SCC base address must be 8-byte aligned.

If the instruction fails, the target CR is unconfigured, with the same semantics as if `ace.clear` were invoked on the same CR (Section 2.5.2).

ACE error ace_err_unimpl is raised if the combination of *AlgorithmPolicy*, *AlgorithmPolicy*, and *SCProtection* is not supported.

ACE error ace_err_invalid_value is raised if the Metadata is invalid (other than ace_err_unimpl), such as, for instance: nonzero bits in reserved fields; unsupported *Locality* value; or an undefined value of *StateNumber* for the *Algorithm* and *AlgorithmPolicy*.

The bit S takes the values 0 (false) or 1 (true). It is set to true if the key is in a system-defined format (`ace.import.S`) and not the format defined by the present architecture (`ace.import`).

`ace.import` appears to other harts as a sequence of individual loads.

The operation of `ace.import` is described in detail in Section 2.6.

The behavior of `ace.import.S` is implementation-defined. If there are no system-defined formats, `ace.import.S` raises an illegal instruction exception.

> The instruction encoding includes a single bit to indicate a system-defined format. If multiple system-defined formats are supported, the system is responsible for distinguishing them. These formats do not necessarily include the key value itself, they may contain a “recipe” for an implementation-defined mechanism to generate the key. Such system generated keys are system-specific and the “recipe” is in general not portable. After internal key generation, the actual key value—not the recipe—is stored in the CR and SCC. For all keys provisioned using `ace.import.S`, the *KeyType* field in the Metadata is 0, denoting a key represented by value.

> The `ace.export` instruction does not have an `ace.export.S` analog to `ace.import.S`. This means that system-specific dynamically generated keys can be imported but not exported in the same form: they must be exported in the format architected here for ACE CCs.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **NO**
Trappable: **YES**.
Interruptible: **YES**.

2.5.5. ace.exec

Synopsis

Perform a cryptographic operation in the ACE unit.

Mnemonic

The `ace.exec` instruction admits four Forms, namely

A. `ace.exec Vd, Kn|K(Xn), Vs` takes an input vector and writes to an output vector.

B. `ace.exec Kn|K(Xn), Vs` takes an input vector but does not write to an output vector.

C. `ace.exec Vd, Kn|K(Xn)` does not take an input but writes to an output vector.

D. `ace.exec Kn|K(Xn)` does not take an input vector and does not write to an output vector.

Encoding

`ace.exec` is an R-type instruction.

```
{reg:[
    { bits:  7, name: 0x5b, attr: ['custom-2'] },
    { bits:  5, name: 'rd', attr: ['Vd','-','Vd','-'] },
    { bits:  3, name: 0x0, attr: ['ace.exec'] },
    { bits:  5, name: 'rs1', attr: ['Kn|K(Xn)'] },
    { bits:  5, name: 'rs2', attr: ['Vs','Vs','-','-'] },
    { bits:  1, name: 0x0 },
    { bits:  1, name: 'r' },
    { bits:  1, name: 0x0 },
    { bits:  2, name: 'Form', attr: ['0    0','0    1','1    0','1    1'] },
    { bits:  2, name: 0x0 },
]}
```

Bit `r` (i.e., bit 26) takes the value 0 if the context is encoded as an immediate, and it takes the value 1 if a GPR (scalar integer value) is used to reference it.

For the four Forms A–D above, the encodings are:

A. `Form` = `0b00`.

B. `Form` = `0b01` and `rd` = `0b00000`.

C. `Form` = `0b10` and `rs2` = `0b00000`.

D. `Form` = `0b11` and `rs2` = `rd` = `0b00000`.

Description

Performs a cryptographic operation such as an encryption or decryption, absorbing a value in a hash, an elliptic curve point addition or scalar multiplication, etc., and possibly returning a value.

If the V extension register file is not implemented, only Form D is supported and `ace.input`,

`ace.output` must be used to provide inputs and extract outputs from `ace.exec`, as described in Section 2.9.2.1.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **YES**.
Trappable: **NO**.
Interruptible: See interruptibility rules in Section 2.9.2.

2.5.6. ace.state

Synopsis

Change the state of the algorithm encoded in the CC in a CR.

Mnemonic

The `ace.state` instruction admits three Forms, namely

A. `ace.state Kd|K(Xd), #immed7` with no auxiliary input.

B. `ace.state Kd|K(Xd), #immed7, Xs` with an auxiliary input in a GPR.

C. `ace.state Kd|K(Xd), #immed7, Vs` with an auxiliary input in a vector register.

Encoding

`ace.state` is an R-type instruction.

```
{reg:[
    { bits:  7, name: 0x5b, attr: ['custom-2'] },
    { bits:  5, name: 'Kd|K(Xd)' },
    { bits:  3, name: 'ace.state', attr:
['0    0    1','0    1    0','0    1    1'] },
    { bits:  5, name: 0x0 },
    { bits:  5, name: 'rs1', attr: ['–','Xs','Vs'] },
    { bits:  7, name: 'immed7' },
]}
```

The encodings for the three Forms are:

A. `ace.state` = `0b001`.

B. `ace.state` = `0b010`.

C. `ace.state` = `0b011`.

Description

The instruction is used to transition the state machine of the cryptographic algorithm in CR `Kd` or `K(Xd)` from one stage to another-for example, advancing from initialization to data absorption in a hash function, then to finalization and output of the digest. The `#immed` operand is a constant from a set of admissible values defined for each algorithm. Optionally, `Xs` or `Vs` contain an auxiliary input.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **YES**.
Trappable: **NO**.
Interruptible: **NO**.

It is allowed to repeat `ace.state` instructions to the current state. When the algorithm needs to prevent this, it will automatically transition to a different state after completing `ace.state`.

2.5.7. ace.error

Synopsis

Return an error code from the ACE unit.

Mnemonic

`ace.error Xd`

Encoding

The operations is a CSR read operation.

Description

The ACE Unit may be in an error state due to the previous operation. `ace.error` is implemented as a read from CSR `aceerror`, which may cause the hart to wait if the error code is not yet available (Section 2.4.1). The error code is written into GPR `Xd`. The architected error codes are given in Table 6.

Properties

Possibly modifies state: **NO**.
Usage-controlled: **NO**.
Trappable: **NO**.
Interruptible: **NO**.

2.5.8. ace.size

Synopsis

Return the size of a CC.

Mnemonics

There are four Forms of the `ace.size` instruction:

A. `ace.size Xd, Ks|K(Xs)` - Size from a context.

B. `ace.size Xd, Xs` - Size from Metadata Header.

C. `ace.size.internal Xd, Xs` - Internal capacity estimate for a CR.

D. `ace.size.iobuf Xd, Xs` - Internal capacity estimate for the ACEIOBUF.

Encoding

The encoding shares `func3` with `ace.exec`, however with bit 25 (the lsb of `func7`) equal to 1 and

the two msbs of `func7` equal to 0.

```
{reg:[
    { bits:  7, name: 0x5b, attr: ['custom-2'] },
    { bits:  5, name: 'Xd' },
    { bits:  3, name: 0x0, attr: ['ace.exec'] },
    { bits:  5, name: 'Ks/K(Xs)/Xs' },
    { bits:  5, name: 0x0 },
    { bits:  1, name: 0x1 },
    { bits:  1, name: 'r' },
    { bits:  1, name: 0x0 },
    { bits:  2, name: 'F' },
    { bits:  2, name: 0x0, attr: ['func2'] },
]}
```

The `F` field and the `r` bit may assume the following values:

- `F` = `0b00`, `r` = `0` ⇒ Form A: Size from a context given as `Ks`.
- `F` = `0b00`, `r` = `1` ⇒ Form A: Size from a context given indirectly as `K(Xs)`.
- `F` = `0b01`, `r` = `0` ⇒ Form B: Size from Metadata Header.
- `F` = `0b10`, `r` = `0` ⇒ Form C: Internal capacity of a CR.
- `F` = `0b11`, `r` = `0` ⇒ Form D: Internal capacity of the ACEIOBUF.

Description

- Form A `ace.size` returns the total size (in bytes) of the memory buffer that would be necessary to store the exported SCC. Knowing this size is necessary for any feature that relies on `ace.export`, including context switching. The instruction returns 0 if the CR is unconfigured.
- Form B `ace.size` returns the total size (in bytes) of an SCC having the Metadata section given in `Xn` into GPR `Xd`, and 0 if the algorithm is not supported.

  On RV32, `Xs` contains bits [31:0] of the Metadata. This suffices since they contain all fields required to determine the size of the SCC.

- Form C `ace.size` returns the amount of internal CRF capacity used by a CR with Metadata section matching `Xn` on the *Algorithm*, *SCProtection*, and *KeyType* Metadata fields. In this case, all other bits must be zero. If the combination of *Algorithm*, *SCProtection*, and *KeyType* fields is not supported, `ace.size` returns 0. If, instead, only the *StateNumber* is non-zero and its value is 15, resp., 14, `ace.size` returns the total, resp., the remaining internal capacity of the CRF.
  All the values returned by the instruction are expressed in terms of implementation-dependent units, and in principle cannot be compared across different implementations.

  On RV32, `Xs` contains bits [31:0] of the Metadata (as for the Metadata type instruction).

- Form D: The source register contains the desired byte length of the buffer to be allocated, and the destination register contains the size of the buffer in the ACE unit.

In general, no correlation may be assumed between the size of a PI, the corresponding CR's usage of internal resources, and the size of the SCC.

Properties

Possibly modifies state: **NO**.
Usage-controlled: **NO**.
Trappable: **NO**.
Interruptible: **NO**.

2.5.9. ace.avail

Synopsis

Verify whether an algorithm is supported.

Mnemonic

`ace.avail Xd, Xs`

Encoding

`ace.avail` is an alias to Form B of `ace.size`, as the latter returns 0 if the algorithm is not supported.

Properties

Same as `ace.size`.

2.5.10. ace.restrict

Synopsis

Restrict the policies encoded in a CC in a CR.

Mnemonic

`ace.restrict  Kd|K(Xd), Xs1`

Encoding

The encoding shares `func3` with `ace.exec`, however with bit 25 equal to 1 and `func2` equal to 1. It is thus an R-type instruction. The source register field encodes a GPR and not a vector register.

```
{reg:[
    { bits:  7, name: 0x5b, attr: ['custom-2'] },
    { bits:  5, name: 'Kd|K(Xd)' },
    { bits:  3, name: 0x0, attr: ['ace.exec'] },
    { bits:  5, name: 0x0 },
    { bits:  5, name: 'Xs1' },
    { bits:  1, name: 0x1 },
    { bits:  1, name: 'r' },
    { bits:  3, name: 0x0 },
    { bits:  2, name: 0x1, attr: ['func2'] },
]}
```

Bit r (i.e., bit 26) takes the value 0 if the context is encoded as an immediate in Bits [11:7], and it takes the value 1 if a GPR (also encoded in [11:7]) is used to reference it.

Description

This instruction can be used to:

- Activate the side-channel protection of the CC or increase it;
- Restrict the use of encryption or decryption in a CC that allows both;
- Restrict the usage of a CC (as stored in a CR) to specific modes; and
- Add a Locality or replace a Locality with a stricter one.

Let `mh` be the Metadata Header of the CC in the destination CR.

On RV64, the semantics of the instruction are:

- `Xs` follows the format of the Metadata Header. Only the fields *AlgorithmPolicy*, *SCProtection*, *Locality*, *UsagePolicy*, and *ExpirationDate* of `Xs` may be defined. All other fields must be 0, else ACE error ace_err_invalid_value is raised.
- `Xs`.*AlgorithmPolicy* is used to set the new `mh`.*AlgorithmPolicy* of the CR. No operation may be enabled, any attempt to do so raises ACE error ace_err_invalid_value. In some cases, a zero value is not permitted, such as disabling both encryption and decryption.
- `Xs`.*SCProtection* is used to turn on side-channel protection or raise its level. Attempting to remove or downgrade side channel protection, or to set a non-implemented level raises ACE error ace_err_invalid_value.
- If `Xs`.*Locality* is non-zero, then it can be used to restrict the Locality. Allowed replacements of Locality are:
  - SiPSecret → ChipFamilySecret or ChipSecret;
  - ChipFamilySecret → ChipSecret;
  - OEMSecret → ProductSecret or DeviceSecret;
  - ProductSecret → DeviceSecret;
  - MLocality → HLocality or SLocality; and
  - HLocality → SLocality.

Then, when the CR is exported as a CR, it will use the new Locality values to tweak the export and import algorithms. Generalizing the Locality is not permitted. Attempting to do so raises ACE error ace_err_invalid_value

- A zero in any of the three bits `Xs`.*UsagePolicy* means that the corresponding policy is *not* changed.
- A one in any of the three bits `Xs`.*UsagePolicy* means that usage by the corresponding mode is not allowed, and the corresponding bit in the Metadata of the destination CC is set to 1.
- If `Xs`.*ExpirationDate* is zero, the corresponding policy in `mh`.*ExpirationDate* is *not* changed.
- A non-zero value in `Xs`.*ExpirationDate* will be copied to `mh`.*ExpirationDate*, but only if that

policy was not already programmed, otherwise error ACE error ace_err_invalid_value is raised.

On RV32:

- `s` must be even, the register pair `X[s]` and `X[s+1]` are read, and `(X[s+1] @ X[s])` replaces `Xs` in the RV64 description.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **YES**.
Trappable: **NO**.
Interruptible: **NO**.

> This instruction enables several practical use cases:
>
> - Enabling side-channel protection when supported by the microarchitecture, even if the original CC was not configured with it.
> - Allowing privileged code to give a CC to several processes in a fine-grained manner. This can be achieved by cloning the CR and customizing the Filter for each clone to an authorized process, or by exporting it with a per-process *SSecret* Locality.
> - Similarly, enabling privileged code to split a CC capable of both encryption and decryption into two CCs, one only capable of encryption and the other only capable of decryption, thereby creating a one-way channel between processes.
>
> The ability to perform such customizations saves both time and computational resources by avoiding additional round-trips to trusted environments.

2.5.11. ace.clone

Synopsis

Copy a CR into another CR.

Mnemonic

`ace.clone Kd|K(Xd), Ks|K(Xs)`

Encoding

The encoding shares `func3` with `ace.exec`, however with bit 25 equal to 1 and `func2` equal to 2. The instruction is encoded using the Form D' `ace.exec` encoding, and is thus an R-type instruction.

```
{reg:[
    { bits:  7, name: 0x5b, attr: ['custom-2'] },
    { bits:  5, name: 'Kd|K(Xd)' },
    { bits:  3, name: 0x0, attr: ['ace.exec'] },
    { bits:  5, name: 'Ks|K(Xs)' },
    { bits:  5, name: 0x0 },
```

```
    { bits:  1, name: 0x1 },
    { bits:  2, name: 'R' },
    { bits:  2, name: 0x0 },
    { bits:  2, name: 0x2, attr: ['func2'] },
]}
```

Bits [27:26], i.e., field `R`, take the following values depending on how the CRs are given:

A. `00` if the source and the output CRs are both encoded as immediate values.

B. `01` if the source CR is encoded as immediate and the output CR is selected using a GPR.

C. `10` if the source CR is selected using a GPR and the output CR is encoded as an immediate value.

D. `11` if the source and the output CRs are both selected using the values of GPRs.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **NO**.
Trappable: **YES**, it can be implemented as an export followed by an import in a handler.
Interruptible: **NO**.

Description

Clones source CR `Ks` (or the CR indexed by `Xs`) into destination CR `Kd` (or the CR indexed by `Xd`).

2.5.12. ace.input

Synopsis

Load data into the ACEIOBUF.

Mnemonic

`ace.input %offset(Xs), Xl`

Encoding

`ace.input` is an S-type instruction.

```
{reg:[
    { bits:  7, name: 0x0b, attr: ['custom-0'] },
    { bits:  5, name: 'immed[4:0]' },
    { bits:  3, name: 0x4, attr: ['ace.input'] },
    { bits:  5, name: 'Xl' },
    { bits:  5, name: 'Xs' },
    { bits:  7, name: 'immed[11:5]' },
]}
```

Description

In all cases where the vector registers are not sufficiently long, or not implemented, inputs can be read directly from memory and written directly to memory. For this reason, the architecture

defines `ace.input` and `ace.output` (cf. next section). `ace.input` reads `Xl` bytes from memory starting at address `%offset(Xs)` and copies them to the input buffer, starting at offset `acestart`.

If `Xl` == 0, the operation is a NOP.

`Xl` may not be greater than `aceiobuftop` - `acestart`.

If `Xl` is greater than `aceiobuftop` - `acestart`, ACE error ace_err_invalid_value is raised.

The intended usage of `ace.input` and `ace.output` is explained in detail in Section 2.9.2.1.

`ace.output` (see below) and `ace.input` can be used to save and restore the contents of the ACEIOBUF freely as long as `aceactivecr`'s value is 32, for instance by context switching code. Also, these operations start at the offset specified in `acestart`, so the state of the buffer can easily be saved and restored with the same approach used for vectors.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **YES**
Trappable: **YES**.
Interruptible: **YES**.

2.5.13. ace.output

Synopsis

Save the contents of the ACEIOBUF to memory.

Mnemonic

`ace.output %offset(Xd), Xl`

Encoding

`ace.output` is an I-type instruction.

```
{reg:[
    { bits:  7, name: 0x27, attr: ['custom-0'] },
    { bits:  5, name: 'Xd' },
    { bits:  3, name: 0x4, attr: ['ace.output'] },
    { bits:  5, name: 'Xl' },
    { bits: 12, name: 'immed' },
]}
```

Description

This is the output operation corresponding to `ace.input`. `ace.output` writes the contents of the input buffer, starting at offset `acestart`, to memory starting at address `%offset(Xs)`.

If `Xl` == 0, the operation is a NOP.

`Xl` may not be greater than `aceiobuftop` - `acestart`.

If `Xl` is greater than `aceiobuftop` - `acestart`, ACE error ace_err_invalid_value is raised.

Properties

Possibly modifies state: **YES**.
Usage-controlled: **YES**.
Trappable: **YES**.
Interruptible: **YES**.

2.6. Provisioning, Export and Import of Cryptographic Contexts

The length of a PI must be a multiple of 8 bytes. The first 8 bytes contain the **Metadata** Header, as defined in Section 2.3.3. The subsequent data contain the **Content**, key material and any algorithm-specific data used to initialize the CR. The Content's format is entirely determined by the *Algorithm* and *KeyType* fields within the Metadata, and is specified in Section 2.9 for each architected algorithm.

Format of Provisioning Input

`bytes [ 0 .. 7 ]` Metadata Header.
`bytes [ 8 .. u-1 ]` PI Content: Keys and other secrets. Here, `u` = `len_in_bytes(PI)`.

The SCC format is used when exporting a CC from a CR and also to re-import the CC into a CR. Its size must be a multiple of 8 bytes. The first 8 bytes contain the Metadata Header, as defined in Section 2.3.3. The format and length of the exported Content are algorithm-specific (Section 2.9) and, excluding the Additional Data of length `mh`.*AdditionalData*, are determined solely by the *Algorithm*, *KeyType*, and *AlgorithmPolicy* fields in the Metadata.

Format of Sealed Cryptographic Context

`bytes [ 0 .. 7 ]` Metadata Header, not encrypted.
`bytes [ 8 .. 23 ]` SIV produced by the encryption method (128 bits).
`bytes [ 24 .. u-1 ]` Encrypted Serialized Content of the CR. Here, `u` = `len_in_bytes(SCC)`.

The entire Metadata and Content are authenticated, but only the Content is encrypted. The AEAD method used for this purpose is described in Section 2.6.5.

The Serialized Content section under Data Structures in an algorithm specification (Section 2.9) defines the structure of the `content[]` array. Architecturally, `content[]` is the serialization of the content of a CR for export as an SCC.

> While `content[]` provides a view of (certain fields of) the content of a CR that matches the serialized SCC format, it need not be converted to and from the internal representation and explicitly stored in the ACE unit's memory before export and after import.
>
> `content[]` is a virtual representation: its byte ordering does not imply physical contiguity within the CR, nor is there a direct byte-for-byte correspondence. Changes to `content[]` are reflected in the underlying CR representation (except for excluded fields), and vice versa.

> In a threshold implementation, a single key byte in `content[]` may correspond to multiple bytes across the key shares within the CR. Reading that byte may return the XOR of the corresponding bytes in the shares; writing to it may distribute the value across the corresponding bytes in the shares such that their XOR equals the input. These operations may be deferred until the end of an provisioning or import instruction: writing to `context[]` may initially write only to the first share of a key, and at the end of the provisioning or import instruction, the other shares would be generated.

2.6.1. System Keys

System Keys are given by an index into an internal, system-specific System Keys Table (SKT), and therefore, in a PI or in a SCC, in place of the key value itself, a 64-bit field is used to store the index.

For threshold implementations of system keys, ACE may copy the key material from the SKT into the CR and expand it into multiple shares. However, the key must be still be exported as a single index.

It is permissible to include some of the shares of a system key in a threshold implementation in the optional Additional Data of the content in order to accelerate share reconstruction, provided that the key cannot be reconstructed from these shares.

2.6.2. Size Functions

Let `mh` denote a Metadata Header. In order to determine the total lengths of the PI and of the SCC, the implementation provides the following functions or an equivalent mechanism:

1. `MH_size(mh)` returns the byte length of the Metadata Header. In the current specification, it is always 8.
2. `PI_content_size(mh)` returns the byte length of the content section of a PI. It depends only on `mh`.*Algorithm*, `mh`.*AlgorithmPolicy*, and `mh`.*KeyType*.
3. `SCC_content_size(mh)` returns the byte length of the fixed part of the content section of an SCC. It depends only on `mh`.*Algorithm*, `mh`.*AlgorithmPolicy*, and `mh`.*KeyType*. It does not include the length of the variable part of the content of length *AdditionalData* * 8 bytes.

The total size in bytes of a PI is therefore

`len_in_bytes(PI)` = `MH_size(mh)` + `PI_content_size(mh)`,

and the total size in bytes of an SCC is

`len_in_bytes(SCC)` = `MH_size(mh)` + 16 + `SCC_content_size(mh)` + *AdditionalData* * 8.

2.6.3. Interruption Points

The provisioning, export and import operations may only be interrupted at memory read or write operations. The instructions may be interrupted because of a memory exception, and any other type of interrupt must wait for the next memory read or write. The provisioning and import instructions may resume from the point of interruption only if the control flow is returned to them

immediately after the interrupt handler has completed: The privileged interrupt handler must ensure that the instruction is restarted only in this situation, and in any other circumstance, including context switching, the instruction must be restarted, in which case if the instruction is `ace.prov` or `ace.import` it must also clear the CR. Export may resume only of the CR was not used before returning to the interrupted control flow. The implementation will also restart any operation if the internal state of the CR is not in a valid interrupted state.

> The initial cryptographic phase of `ace.export` is currently architected to be uninterruptible. Even if this phase is executed concurrently with other operations, an interrupt handler may need to wait for it to complete before proceeding with any access to the CR. A very long such phase could become a performance bottleneck for very long SCCs, or cause timing issues. To address this, we could allow the SIV computation to be interrupted and use odd, or negative, values in `acestart` to denote this.

2.6.4. Provisioning

We start with the algorithm performed by `ace.prov`, which reads a PI and uses it to provision a CR.

Listing 1. Provisioning a CR with Metadata and content

```
M : array of bytes // in memory

// PREEMPTIBLE WITH `acestart` IN [0 .. 7]
Metadata : bits(64) = M[0 .. 7]
z ← PI_content_size(Metadata[31:0]);

if (z == 0) { error ace_err_unimpl; }
if (there is not enough memory for the algorithm encoded in the Metadata) then
   { error ace_err_memory; }
if (Metadata is inconsistent) { error ace_err_invalid_value; }

- Complete Metadata parsing.
// NEXT OPERATION PREEMPTIBLE WITH `acestart` IN [8 .. Z+7]
- Sequentially read and parse the z bytes at M+8.
```

2.6.5. Export and Import

To export and import CRs we use AES-GCM-SIV cite:[DBLP-journals-iacr-GueronLL17,RFC8452].

> AES-GCM-SIV has been chosen because it only required single passes of data in main memory, reducing the number of external memory accesses. GCM uses also a single pass but requires separate nonce and tag values, increasing the size of the exported data by one block.

We define the Key Derivation (Listing 2) function first. The generation of an SCC from a CR (Listing 3) and the import of an SCC into a CR (Listing 4) are defined next.

Listing 2. RFC8452 Key Derivation

```
function RFC8452_KeyDeriv(key : bits(256)) = {
  struct {
    AESE256(key, (zeros(125) @ 0b101))[63:0] @ AESE256(key, (zeros(125) @
0b100))[63:0] @
    AESE256(key, (zeros(125) @ 0b011))[63:0] @ AESE256(key, (zeros(125) @
0b010))[63:0],
    AESE256(key, (zeros(125) @ 0b001))[63:0] @ AESE256(key, (zeros(125) @
0b000))[63:0]
  }
}
```

Due to the computational cost of the derivation functions, implementations may cache recently used derived keys indexed by Locality.

For a Metadata Header `mh`, `get_localities(mh)` returns a variable length array with zero or more Locality Secrets, corresponding to the active localities, ordered by increasing index value, as defined in Section 2.3.3.4. During SCC export and import, these Locality Secrets are absorbed, in ascending index order, to the input of the AEAD tag computation, before the metadata. If the *Locality* field is zero, no Locality Secrets are absorbed.

The export operation maintains a per-CR flag indicating that an export is in progress. The reason is that operations that modify the internal state may be performed on a CR with an interrupted export operation pending. In this case, resuming the export operation may use an incorrect SIV for the remaining SCC data, causing a subsequent import to fail authentication. Therefore, any operations that modifies the internal state of the CR will also unset this flag. If an attempt is made to resume an export operation while this flag is cleared, it is restarted instead.

Listing 3. Generation of an SCC from a CR

```
& mh : bits(64) = CR.Metadata_header;         // from the CR
CR.exporting : bool;                          // "hidden" per-CR ACE unit state
local SIV, tmp : bits(128);
local enc_key : bits(256);
local auth_key : bits(128);

// CR.exporting is always false except during export.
CR.exporting = true;

// Note: CR.content[] includes the optional extra content at the end

z : int ← SCC_content_size(mh);
z ← z + mh._AdditionalData_ * 8;
CR.content : array[z] of bytes;

// In external memory: Metadata Header, then SIV, then Content
M : array[MH_size(mh) + 16 + z] of bytes;
let last_block_fractional : Boolean ← z % 16 != 0;
```

```
let num_blocks : int ← ceil(z / 16);

{enc_key, auth_key} ← RFC8452_KeyDeriv(CSK);
L : list ← {} if (mh.Locality == 0) else get_localities(mh.Locality);

// Include Locality "prefix" (zero or more 128-bit blocks)
tmp ← POLYVAL(auth_key, L @ zeros(64) @ mh @ CR.content[]);
SIV ← AESE(enc_key, 0 @ tmp[126:0]);

// FOLLOWING MEMORY WRITE PREEMPTIBLE, `acestart` FROM 0 TO 7
M[0 .. 7] ← mh;

// FOLLOWING MEMORY WRITE PREEMPTIBLE, `acestart` FROM 8 TO 23
M[8 .. 23] ← SIV;

foreach(i from 0 to num_blocks - 1) {
  if ((i == num_blocks - 1) and last_block_fractional)
    // FOLLOWING MEMORY WRITE PREEMPTIBLE, `acestart` FROM 24+i*16 TO 31+i*16
    M[24+i*16 .. 31+i*16]
      ← CR.content[i*16 .. 7+i*16]
      xor AESE(enc_key, 1 @ SIV[126:32] @ (SIV[31:0]+bin(i,32) % 2**32)[63:0];
  else
    // FOLLOWING MEMORY WRITE PREEMPTIBLE, `acestart` FROM 24+i*16 TO 39+i*16
    M[24+i*16 .. 39+i*16]
      ← CR.content[i*16 .. 15+i*16]
      xor AESE(enc_key, 1 @ SIV[126:32] @ (SIV[31:0]+bin(i,32) % 2**32);
}

CR.exporting ← false;
```

The import algorithm needs some special care:

- The first block read from memory by the operation is the Metadata Header `mh`:
  - If strictly fewer than four bytes are read before the instruction is interrupted, ACE leaves `acestart` at 0, and the instruction will be restarted upon resumption.
  - If four bytes or more have been read, the value of `mh.StateNumber` in the SCC is known. In this case, the instruction may be restarted with `acestart` ≥ 4 recording the amount of data read.
- Af the beginning of the import operation, the hardware saves the `StateNumber` value in the metadata and sets `mh.StateNumber ← ace_state_invalid`. At the end of the import operation, `mh.StateNumber` is set to the correct value.
  - Since context switching code cannot export a partially imported CR when saving the state, it will clear the CR instead, forcing the hardware to restart the import operation.
  - Exceptions that immediately return to the interrupted operation, such as memory exceptions, may cause the hardware to resume the operation instead, if the CR was touched.

Listing 4. Importing an SCC into a CR

```
M : array of bytes;         // from memory
```

```
CR.saved_SN : bits(4);      // this is a "hidden" per-CR ACE unit state

local enc_key : bits(256);
local auth_key : bits(128);
local SIV, tmp : bits(128);

// FOLLOWING MEMORY READ PREEMPTIBLE, `acestart` FROM 0 TO 7
tmp[63:0] ← M[0 .. 7]; // read Metadata Header

CR.saved_SN : bits(4) ← tmp[63:0].StateNumber;
CR.Metadata_header ← tmp[63:0];
CR.Metadata_header.StateNumber ← ace_state_invalid;

& mh = CR.Metadata_header;

z ← SCC_content_size(mh);
z ← z + mh._AdditionalData_ * 8;

if (z == 0) { error ace_err_unimpl; }
if (there is not enough memory for the algorithm encoded in the Metadata) then
    error ace_err_memory;

num_blocks ← ceil(z / 16);
last_block_fractional : Boolean ← (z % 16 != 0);

{enc_key, auth_key} ← RFC8452_KeyDeriv(CSK);

// Absorb Locality "prefix" (zero or more 128-bit blocks)
tmp ← zeros(128);
L : list ← {} if (mh.Locality == 0) else get_localities(mh.Locality);
foreach(i from 0 to length(L)-1) {
  tmp ← tmp xor L[i];
  tmp ← Montmul(tmp, auth_key);
}

// FOLLOWING MEMORY READ PREEMPTIBLE, `acestart` FROM 8 TO 23
SIV ← M[8 .. 23];
tmp ← tmp xor (zeros(64) @ mh);
tmp ← Montmul(tmp, auth_key);

foreach(i from 0 to num_blocks - 1) {
  if ((i == num_blocks - 1) and last_block_fractional) {
    // FOLLOWING MEMORY READ PREEMPTIBLE, `acestart` FROM 24+i*16 TO 31+i*16
    content[i*16 .. 7+i*16] ← M[24+i*16 .. 31+i*16]
      xor AESE(enc_key, 1 @ SIV[126:32] @ (SIV[31:0]+bin(i,32) % 2**32)[63:0];
    tmp[63:0] ← tmp[63:0] xor content[i*16 .. 7+i*16];
  } else {
    // FOLLOWING MEMORY READ PREEMPTIBLE, `acestart` FROM 24+i*16 TO 39+i*16
    content[i*16 .. 15+i*16] ← M[24+i*16 .. 39+i*16]
      xor AESE(enc_key, 1 @ SIV[126:32] @ (SIV[31:0]+bin(i,32) % 2**32);
    tmp ← tmp xor content[i*16 .. 15+i*16];
```

```
  }
  tmp ← Montmul(tmp, auth_key);
}
tmp ← AESE(enc_key, 0 @ tmp[126:0]);

if (tmp != SIV) { error ace_err_import_auth; }

- Now the content can be deserialized.
- This includes computing any derived values not saved as part of the SCC,
- as well as creating shares for threshold implementations.

// FINALLY, restore the StateNumber from the SCC
mh.StateNumber ← CR.saved_SN;
```

2.7. CR Invalidation

In certain situations, in addition to raising an ACE error, an ACE instruction may cause a CR to be **invalidated**. This means that the following happens:

- The CR is put into State *invalid*;
- For instructions that refer to that CR:
  - `ace.prov`, `ace.clear` (thus also `ace.reset`), `ace.import`, `ace.export`, `ace.size` and `ace.avail` work normally.
  - `ace.exec`, `ace.clone`, and `ace.restrict` raise an ACE error ace_err_invalid_CR;
  - `ace.input` and `ace.output` raise ACE error ace_err_invalid_CR when `aceactivecr` points to an invalid CR;
  - `ace.state` raises an ACE error ace_err_invalid_CR, unless it is used to return to State *initial*;
  - `ace.error` always returns error ace_err_invalid_CR after any operation involving the CR except for `ace.import` and `ace.export`, `ace.clear`, `ace.prov`, and `ace.size`, which all work normally, and using `ace.state` to return to State *initial*.
- The current instruction is terminated.

In State *completed*, only `ace.prov`, `ace.export`, `ace.import`, `ace.clear`, and `ace.error` are permitted.

2.8. Exception Handling via Horizontal Traps

This section is currently informative and non-normative.

ACE has been architected so that in properly written code, the most common executed instructions, namely `ace.exec` and `ace.state`, rarely have to deal with errors. Errors should occur mostly with configuration operations, caused by insufficient CRF capacity or while authenticating SCCs.

However, revocation of localities or reached expiration dates may cause a CR to suddenly become invalid just before executing an `ace.exec` or `ace.state` instruction. Requiring software to check for errors after each operation would impose significant burden on the software and possibly degrade

performance.

Such horizontal traps can be used to implement a try/catch mechanism: At the start of a subroutine that performs a cryptographic protocol, the address of the "catch" code is written in CSR `acehtrapvector` (Section 2.4.9) and the mask with the error types to catch is loaded in CSRs `acehtrapmask` (Section 2.4.8). When exiting the routine, the two CSRs are cleared.

From the point of view of the architecture, the trap would be precise, i.e., all instructions before the one that causes the trap are completed, and the address of the instruction should be set in the User EPC (`uepc`) CSR (if it is not defined, we defined it). A CSR like `ucause` would not be used in this case, with `aceerror` providing the instruction.

> The main application of such code is when revocation or expiration cause a CR to become invalid. In such contexts, it is not the duty of the cryptographic subroutine to resolve the issue, but of the code that makes use of the latter, as the keys may have to be renegotiated, and this may even need user intervention.

> If our definition of horizontal traps is not allowed, we may have to follow the development of the HFI extension or implement a solution that traps to S-mode first and then immediately returns to the catch code. We will discuss this with the ARC.

2.9. Algorithms

This section is normative.

This section specifies the behavior of the ACE instructions for the supported cryptographic algorithms. Some specifications define algorithm families that can be instantiated with any primitive meeting certain properties, typically constraints on the block size `b` and the key size `k`—such as GCM or GCM-SIV, which are well defined not only with AES but also with other block ciphers, such as SM4. By contrast, the specifications for other algorithm suites are specific only to some primitives, for instance, Ascon.

2.9.1. Definition of an Algorithm in ACE

Each algorithm is fully defined by the following information:

Parameters

These include:

- The *block size* `b`, which is the size of the basic unit of information processed by the algorithm, usually the block size of the main underlying primitive;
- The *key size* `k`, for keyed algorithms; and
- Any other parameter (for instance, counter size) that is necessary to instantiate the algorithm with compatible primitives.

Data Structures

These include:

1. **Provisioning Input** (Content): The data following the Metadata Header in the PI, such as keys and other required secrets. The fields are listed in the order they appear in the PI, each with its length.
2. **Internal State** (Content): Information that must be maintained in the internal state of a CC, besides the information provided by the metadata. The ordering of the fields is not given, and their lengths may differ from those listed, for example for threshold or fault-tolerant implementations.
3. **Serialized Content**: The data that follows the Metadata Header in the plaintext portion of an SCC. The fields are listed in the order they appear in an SCC, each with its length.

Algorithm-Specific Functions

Any other function that is specific to the algorithm.

State Machine

The state machine is defined by the following information:

1. A list of **States** in which the internal state machine of the logic is executed, together with their representation according to the rules set in Section 2.3.3.3.
2. A complete list of the **allowed state transitions**, including the precise conditions under which each transition is allowed, the Form of `ace.state` used for each of these transitions and the meaning of any additional parameter;
3. The enumeration of the **allowed instructions** at each state and their **behavior**.

The instructions permitted in each algorithm state are typically `ace.exec` and `ace.state`, though this is not a general rule. For `ace.exec`, only one of its forms (see Section 2.5.5) is allowed per state, along with the corresponding replacement sequence defined in Table 7. If a Form of `ace.exec`, or `ace.state` not explicitly allowed in the current algorithm state is used, then an illegal instruction exception is raised.

If a non-allowed state transition is requested in `ace.state`, including requesting encryption or decryption from a CC not allowing it, an ACE error ace_err_unimpl is raised. This also invalidates the CR.

Pseudocode is provided to illustrate the software implementation of ACE-supported algorithms using the ACE ISA. For clarity, the pseudocode generally uses fixed parameters and omits the `ace.exec` Forms A-C replacement sequences. The examples are non-normative.

> In many algorithms, an `ace.exec` or `ace.state` operation could logically imply a state change. For example, after setting an IV or nonce in an AEAD mode, the instruction might naturally transition to a subsequent state. Deviations from this behavior may be necessary to avoid security issues. An example is given by the automatic transitions from hash finalization to verification, or from hash verification to the *completed* state (Section 2.9).

2.9.2. Generic Rules

2.9.2.1. Usage of `ace.input` and `ace.output` to Replace Vector Inputs and Outputs

When ACEIOBUF is implemented, in all algorithms the following applies: any occurrence of the "Original Instructions" in Table 7 may be substituted by the corresponding "Replacement Sequence" where `ace.input` and `ace.output` instructions are used in place of the vector register inputs and outputs, even if we do not explicitly write it each time.

Table 7. Replacement sequences for `ace.exec` and `ace.state` using `ace.input` and `ace.output`

Instruction and Form	Original Instruction	Replacement Sequence

`ace.exec`, Form A	`ace.exec Vd, Kn|K(Xn), Vs`	`1: ace.input Xl, Vs`
		`2: ace.exec Kn|K(Xn)`
		`3: ace.output Vd, Xl`

`ace.exec`, Form B	`ace.exec Kn|K(Xn), Vs`	`1: ace.input Xl, Vs` `2: ace.exec Kn|K(Xn)`
`ace.exec`, Form C	`ace.exec Vd, Kn|K(Xn)`	`1: ace.exec Kn|K(Xn)` `2: ace.output Vd, Xl`
`ace.state`, Form C	`ace.state Kd|K(Xd), #immed, Vs`	`1: ace.input Xl, Vs` `2: ace.state Kd|K(Xd), #immed`

The only operations performed by `ace.input` and `ace.output` are to provide the inputs to the cryptographic operations and read out their output. No computation is performed on the data by `ace.input` and `ace.output`. The entirety of the computations and internal state changes of the `ace.exec` and `ace.state` operations remains in the `ace.exec` and `ace.state` operations

`ace.input` and `ace.output` may be used if enabled and `aceiobuflen` is non-zero, otherwise an ACE error ace_err_unconf_buf is raised. If `aceiobuflen` is non-zero, and the range size `aceiobuftop` - `acestart` is an invalid size for the algorithm, the `ace.exec` operation is not performed and ACE error ace_err_invalid_value is raised.

In all cases, the `ace.input` and `ace.output` instructions must be temporarily tied to a specific context in order to properly determine the allowed sequences. This is done by setting the `aceactivecr` CSR (Section 2.4.7) first. The user must not forget to set it back to 32 at the end of their code.

The architecture uses the `acesequence` CSR to track the progress of the replacement sequences. The initial value of `acesequence` is 0. It remains zero while we are using the forms of `ace.exec` and `ace.state` specified in the algorithm. As soon as one of the following events happens:

1. An `ace.input` is completed in place of a Form A or B `ace.exec`, or Form C `ace.state`; or
2. A Form D `ace.exec` is completed in place of a Form C `ace.exec`,

then `acesequence` is set to 1, to denote that the first instruction in the replacement sequence has completed. Then, `acesequence` keeps track of how many instructions of the replacement sequence have been completed. At the end of the replacement sequence, `acesequence` is reset to 0.

The ordering of the instructions in a replacement sequence is enforced by the architecture

according to the value of `acesequence` and the state number of the CR configured in `aceactivecr`. Not following the instruction ordering raises an illegal instruction exception and invalidates the CR.

`acesequence` can be freely read and written in all modes. In particular, `acesequence` may be saved and restored by privileged code at context switch together with `acestart`. Abusing this register may also allow to arbitrarily repeat `ace.exec` or `ace.state` instructions. However, if the algorithm and its state machine are correctly designed, this will not allow the user to compromise the internal state of the algorithm.

2.9.2.2. Interruptibility

Except for exceptions caused by memory accesses or the interruption of vector accesses, most ACE operations are uninterruptible.

The operation of `ace.exec` on a basic processing unit, in any state of any algorithm, is uninterruptible, unless explicitly allowed by the algorithm. When `ace.exec` operates iteratively on an inputs who length is an integer multiple of the basic processing unit, each block is processed uninterruptibly, but the instruction may be interrupted between blocks; the hardware sets `acestart` to the interruption point.

An implementation must wait before accessing a CR that is still asynchronously being written to.

2.9.2.3. Notation in the Algorithm Descriptions

- `INPUT`: The input data to be processed by `ace.exec` or `ace.state`. This can be the input vector register, or the ACEIOBUF, as described in Section 2.9.2.1.
- `OUTPUT`: The output data produced by `ace.exec`. This can be the output vector register, or the ACEIOBUF, as described in Section 2.9.2.1.
- `IOLEN`: Either `VL*SEW` or the length `aceiobuftop` of the ACEIOBUF in bytes, depending on whether vector registers are used or the ACEIOBUF is used.

2.9.2.4. Single-Share Key Export Rule for Threshold implementations

Any CC with its algorithm implemented as a threshold implementation will always provision, import and export *all* its keys and secrets as *single, reconstructed shares*.

The PI and SCC Formats will be the same as for the non-threshold implementation variants of the same algorithm, with the only difference being the value of the *SCProtection* in the Metadata.

For the purpose of architecture specification, all values are also always represented as a single share in this document. Threshold implementations are simply implementation variants of the same algorithm.

> This rule assumes that reconstructing or re-randomizing a key is a fast operation, such as an `xor` with random values. This requires the ACE execution unit to have access to a random number generator. Besides ensuring the compactness of an SCC, this rule also guarantees key portability across architectures that implement side-channel protection in different ways. If reconstructing the values is efficient but generating the shares is a slow process, the implementation may take

advantage of the optional Additional Data in the Content to store information that can accelerate share generation, such as a copy of some (or all) shares.

2.9.2.5. Representation of System Keys in the Data Structures

System Keys will change all the algorithms described here in the following way:

1. In a PI and in an SCC, the key field will contain the index of the key in the SKT in place of the value.
2. This field is reduced from the original key size to a 64-bit value. This saves space in an SCC and allows for faster export and import operations.
3. For algorithms that require more than one key, such as XEX/XTS, a single index is still used to refer to an entry in the SKT that contains multiple keys—and thus a single 64-bit value in the PI and SCC.

2.9.3. States for the Symmetric Algorithms Defined in this Specification

The following table, together with Table 3, defines the states used by the symmetric algorithms in this specification.

Table 8. Constants defining CC states for symmetric algorithms

Value	Mnemonic for the Constant	State Name	Description
1	ace_state_operate	*operate*	Generic operational state.
	ace_state_hash_absorb	*hash_absorb*	Start and initialize a hash function of the tag/MAC part of an algorithm and start absorbing blocks of data (in the case of an AEAD algorithm, this usually is the associated data).
2	ace_state_hash_last_block	*hash_last_block*	In this state additional data may be absorbed, such as partial blocks and lengths, if the hash/tag algorithm processes the last block of its input or of the associated data part of its input in a different way from the other blocks.
3	ace_state_hash_finalize	*hash_finalize*	Finalize the internal state of a hash/tag function.
4	ace_state_hash_verify	*hash_verify*	Compare the internal hash/tag value to the provided one.
5	ace_state_hash_output	*hash_output*	Enter the hash output phase of a hash/tag function.
6	ace_state_encrypt	*encrypt*	In this state encryption is performed, as well as an update to the tag computation if this operation is part of the algorithm.
7	ace_state_decrypt	*decrypt*	In this state decryption is performed, as well as an update to the tag computation if this operation is part of the algorithm.

Value	Mnemonic for the Constant	State Name	Description
8	ace_state_enc_last_block	*enc_last_block*	For the encryption path of authenticated encryption modes of operation to process the last block separately.
9	ace_state_dec_last_block	*dec_last_block*	For the decryption path of authenticated encryption modes of operation to process the last block separately.
10	ace_state_enc_tag_finalize	*enc_tag_finalize*	Finalize the tag computation for AE mode encryption.
11	ace_state_dec_tag_finalize	*dec_tag_finalize*	Finalize internal tag computation for AE mode decryption, verify the value.
12	ace_state_set_aux_value	*set_aux_value*	Set a public/auxiliary value in an algorithm.

The `ace.state` instruction used to transition to states *hash_absorb*, *encrypt*, or *decrypt* often also sets a public value. For algorithms requiring multiple public values or additional parameters (e.g., tag length), these can be set by transitioning to State *set_aux_value* using a Form of `ace.state` with an auxiliary argument, i.e., Form B or C.

2.9.3.1. Electronic Codebook Modes (Direct Encryption/Decryption)

ECB mode cite:[nist-SP-800-38A-1] is the simplest way to use a block cipher.

Parameters

- `b` = block size (8|`b`). `IOLEN` must be a multiple of `b` (including `b`).
- `k` = key size (8|`k`).

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	Key or System Key index

- **Internal State** (Content):
  - `key`: `k` bits.
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	Key or System Key index

Algorithm-Specific Functions

- `enc_blk(K,p) → c` encrypts plaintext block `p` using the key `K`, and returns the ciphertext result `c`.

- `dec_blk(K,c) → p` decrypts ciphertext block `c` using the key `K`, and returns the plaintext result `p`.

State Machine

- **States:**
  - *initial*: The CC is ready.
  - *encrypt*: Encrypting blocks of input.
  - *decrypt*: Decrypting blocks of input.
- **Allowed State Transitions:**
  - From any valid state to *encrypt*, if encryption is allowed.
  - From any valid state to *decrypt*, if decryption is allowed.
- **Behavior:**
  - In State *initial*, no `ace.exec` instruction is allowed.
  - In States *encrypt* or *decrypt*, we can only issue Form A `ace.exec` instructions, i.e., "`ace.exec OUTPUT, Kn|K(Xn), INPUT`."
    - If the state is *encrypt*, then `OUTPUT ← enc_blk(key, INPUT)`.
    - If the state is *decrypt*, then `OUTPUT ← dec_blk(key, INPUT)`.
    - If `IOLEN > b`, then "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" encrypts/decrypts block-by-block. Here, and in the rest of this document, this means from the blocks in the least significant positions to the most significant positions:
      1. `foreach(i from 0 to IOLEN-b by b)` {
         a. `OUTPUT[i+b-1:i] ← enc_blk(key, INPUT[i+b-1:i])` }

In Listing 5 we illustrates how to use an ECB CC to encrypt a block of data.

2.9.3.2. Note on Short Vectors

This paragraph is not normative.

It is conceivable to support a vector length `IOLEN` smaller than a cipher's block size `b` in encryption modes. We add a `b`-bit field called `buffer` to the state of the CR. Let $r \leftarrow \lceil IOLEN/b \rceil$. We define first how to replace a Form A `ace.exec` instruction:

1. The idea is to replace a Form A `ace.exec` instruction by r Form B `ace.exec` instruction followed by r Form C `ace.exec` ones. This replacement sequence is tracked in `acesequence` using values 1 to $2r$-1 (after the $2r$-th instruction, `acesequence` is set to zero and not to $2r$).
2. The i-th Form B `ace.exec` instructions copies its input into bits $[\min(i \cdot IOLEN, b)-1 : (i-1) \cdot IOLEN]$ of the `b`-bit `buffer`, for $i = 1,2,..,r$. The use of "min" ensures that the $b-r \cdot IOLEN$ significant bits of the last input are discarded if `IOLEN` does not divide `b` exactly.
3. The r-th Form B `ace.exec` instruction also applies the primitive to this `buffer`.
4. The i-th Form C `ace.exec` instructions can be issued to write out bits $[\min(i \cdot IOLEN, b)-1 : (i-1) \cdot IOLEN]$ of the `b`-bit `buffer` to its output vector register, for $i = 1,2,..,r$.

5. After the *r*-th Form C `ace.exec` instruction is completed, `acesequence` is reset to zero.

Form B and C `ace.exec` instructions can be replaced in a similar way.

Specifying this behavior would complicate both the architecture and its implementation. It would to require a `b`-bit field tp each CR implementing such a cipher where `b` is larger than `IOLEN`, whereas a single ACEIOBUF would require fewer resources. It would also burden the software using the feature by requiring to use more instructions, potentially in variable length loops for portability. Using existing architectural features, such as reconfiguring the vector lengths, or using vector register groups, would be a simpler approach–beside the aforementioned use of ACEIOBUF.

We do define piece-wise loading of the state for functions with significantly longer inputs, such as hash functions, because these may use the state of the hash function to absorb partial inputs instead of requiring a separate buffer to be used only when full, and on some architectures the long inputs to hash functions could require unsupported vector lengths.

2.9.3.3. Chaining/Feedback Modes

There is no need to explicitly architect the behavior of chaining modes, such as Cipher Block Chaining (CBC), Cipher Feedback (CFB) and Output Feedback (OFB)–all defined in cite:[nist-SP-800-38A-1]–as they are easily implemented in software on top of the ECB, with no intermediate value requiring confidentiality.

2.9.3.4. CTR/LFSR/XCTR/XLFSR Keystream Generation

In the “non-X” modes, i.e., CTR mode and its analog with an LFSR replacing the arithmetic counter, the sum of the lengths of IV and counter/LFSR equals the block size of the underlying keyed PRP or PRF. IV and counter/LFSR are concatenated to form the input to the latter.

In the “X” modes, both IV and counter/LFSR have the same size as the block size of the underlying keyed PRP or PRF. IV and counter/LFSR are `xor`ed together to form the input to the keyed PRP or PRF.

Parameters

- `b` = block size (8|`b`). For this algorithm `IOLEN` must be a multiple of `b`.
- `k` = key size (8|`k`).
- `j` = counter size (8|`j`).
- `n` = IV size (8|`n`).
- If the mode is CTR or LFSR, it is `b = n + j`, else it is `b = n = j`.

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index

- **Internal State** (Content):
  - `key`: `k` bits.

- `IV`: `n` bits.
- `ctr`: `j` bits.

- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index
ii	`n`/8	`IV`
iii	`j`/8	`ctr`

Algorithm-Specific Functions

- `keystream_block(p) → c` — usually an encryption function.
- `tick_ctr()` — updates the `ctr` (either by a modular increment or a LFSR).

State Machine

- **States:**
  - *initial*: The CC is ready.
  - *operate*: In this state the CC will produce a keystream.
- **Allowed State Transitions:** any.
- **Behavior:**
  - In State *initial*, the `ctr` and `IV` fields are set to 0 (`ctr` is set to 1 if the algorithm is LFSR-based).
  - To transition to State *operate*, a Form C `ace.state` instruction must be issued.
    - This sets the value of `IV` to `INPUT`.
    - If `IOLEN` > `b`, only the `b` least significant bits of `INPUT` are considered.
  - In State *operate*, only Form C `ace.exec` instructions are allowed.
    "`ace.exec OUTPUT, Kn|K(Xn)`" performs:

    1. If in a CTR or LFSR mode then `tmp ← keystream_block(IV @ ctr)`,
       else (i.e., if in a XCTR or XLFSR mode) `tmp ← keystream_block(IV xor ctr)`,
    2. `tick_ctr()`, and
    3. `OUTPUT ← tmp`.

    If `IOLEN` is a multiple of `b`, the above three commands are applied to each of the `IOLEN/b` `b`-bit blocks of `INPUT`, producing the corresponding `b`-bit block of `OUTPUT` each time.

Note that if `ace.state` is issued to return to State *initial*, the CC can be reused with a new IV.

Example pseudocode is given in Listing 6 (for the case `b` = `IOLEN` = 128).

2.9.3.5. XEX/XTS modes (one or two keys)

The XTS mode was designed for storage confidentiality cite:[DBLP-journals-ieeesp-Martin10] and made into a Recommendation by NIST in 2010 cite:[nist-SP-800-38E]. XTS is based on the XEX

construction, formalized by Phillip Rogaway cite:[DBLP-conf-asiacrypt-Rogaway04], to design *Tweakable Block Ciphers* cite:[DBLP-conf-crypto-LiskovRW02]. A Tweakable Block Cipher (TBC) is a block cipher which, besides the key and the plaintext or ciphertext, also takes a tweak as input. The tweak is used together with the key to select the permutation computed by the block cipher.

The XEX construction builds a TBC □ from a block cipher `E`, with key `K`, tweak `T`, and plaintext `P`, as follows:

`□(K, T, P) = E(K, P xor M) xor M` where `M = E(K, T)` is called a *mask*.

In XEX mode, once a mask is computed, it is then updated by using a Galois Multiplication for any successive block instead of updating the tweak and encrypting it again.

The NIST standard uses two different keys, one for the data encryption path and one for the encryption of the tweak. The use of a single key or two independently sampled keys is determined by the algorithm, and it is not a configuration parameter for the algorithm. We first define XEX mode functionality.

Parameters

- `b` = block size (8 | `b`). For this algorithm we require `IOLEN` to be an integer multiple of `b`.
- `k` = key size (8 | `k`).

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key1` or System Key index
ii	`k`/8 or 0	`key2` or System Key index for `key1` given

- **Internal State** (Content):
  - `key1`: `k` bits.
  - `key2`: `k` bits — omitted if only one key is used or if both keys come from an entry in the SKT.
  - `mask`: `b` bits.
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key1` or System Key index
ii	`k`/8 or 0	`key2` or System Key index for `key1` given
iii	`b`/8	`mask`

Algorithm-Specific Functions

- `enc_blk(K, p) → c` encrypts block `p` using the key `K`, and returns the result `c`.
- `dec_blk(K, c) → p` decrypts block `c` using the key `K`, and returns the result `p`.

- `update_mask(mask) → mask` updates the mask by a Galois Field multiplication in GF(2^b).

State Machine

- **States:** *initial*, *encrypt*, and *decrypt* (defined as before, cf. Section 2.9.3.1).
- **Allowed State Transitions:**
  - *initial* → *encrypt*, if encryption is allowed.
  - *initial* → *decrypt*, if decryption is allowed.
  - *encrypt*, *decrypt* → *initial*.
- **Behavior:**
  - In State *initial*:
    - The `mask` field is set to zero.
    - No `ace.exec` operation is allowed.
  - The `ace.state` instruction to transition to either *encrypt* or *decrypt* must be of Form C. It sets the tweak, and the `mask` field is used to store it: `mask ← INPUT`.
    - If `IOLEN` > `b`, only the `b` least significant bits of `INPUT` are considered.
    - Then, the following operations are performed:
      1. if two independent keys are used then
         a. `mask ← enc_blk(key2, mask)`
      2. else
         a. `mask ← enc_blk(key1, mask)`
         b. `mask ← update_mask(mask)`.
  - In States *encrypt* and *decrypt*, only `ace.exec` instructions of Form A are allowed. "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" performs the following operations:

    1. In State *encrypt*, it computes `OUTPUT = mask xor enc_blk(key1, INPUT xor mask)`.
       In State *decrypt*, it computes `OUTPUT = mask xor dec_blk(key1, INPUT xor mask)`.
    2. `mask ← update_mask(mask)`.

    If `IOLEN` is a multiple of `b`, the above commands are executed `IOLEN/b` times, processing each `b`-bit block of `INPUT` into a `b`-bit block of `OUTPUT`.

Note that if `ace.state` is issued to return to State *initial*, the CC can be reused with a new tweak.

Encryption pseudocode is provided in Listing 7, decryption is similar

2.9.3.6. Generic Tweakable Block Ciphers

The specification of a generic tweakable block cipher is similar to that of a non-tweakable block cipher in ECB mode. Since `ace.exec` does not admit variants with two inputs, and we have a single input buffer, we set the tweak with a separate instruction. If block and tweak sizes are different, a reconfiguration of the vector length may be required between operations, unless the tweak is shorter than the block size, in which case the instruction setting the tweak will only consider the

necessary bits.

Some commonly used TBCs are `Deoxys` cite:[Deoxys-141], `Skinny` cite:[DBLP-conf-crypto-BeierleJKL0PSSS16], `QARMA` cite:[DBLP-journals-tosc-Avanzi17], and `QARMAv2` cite:[DBLP-journals-tosc-AvanziBDEGNR23].

Parameters

- `b` = block size (8 | `b`). For this algorithm we require `IOLEN` to be an integer multiple of `b`.
- `k` = key size (8 | `k`).
- `t` = tweak size (8 | `k`).

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index

- **Internal State** (Content):
  - `key`: `k` bits.
  - `tweak`: `t` bits.
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index
ii	`t`/8	`tweak`

State Machine

- **States:**
  - *initial*, *encrypt*, and *decrypt* defined as before.
  - *set_aux_value*, for setting the tweak value in the next encryption or decryption operation.
- **Allowed State Transitions:**.
  - *initial* → *set_aux_value*
  - *set_aux_value* → *encrypt*, if encryption is allowed.
  - *set_aux_value* → *decrypt*, if decryption is allowed.
- **Behavior:**
  - In State *initial*:
    - Upon entering the state, the `tweak` field is cleared.
  - In each state, we can issue Form C `ace.state` instructions, which perform:
    1. `tweak` ← `INPUT`, and
    2. *Instead of transitioning to state set_aux_value, the state remains unchanged (can be initial, encrypt, or decrypt).*

- In State *encrypt*, respectively, *decrypt*, besides Form C `ace.state` instructions, only `ace.exec` instructions of Form A are allowed, where "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" encrypts, respectively, decrypts `INPUT` into `OUTPUT`.

This algorithm does not automatically update the tweak value.

2.9.3.7. GCM Mode

The GCM mode was designed by David McGrew and John Viega cite:[DBLP-conf-indocrypt-McGrewV04] and was made into a NIST Recommendation in 2007 cite:[nist-SP-800-38D].

Parameters

- `k` = key size (8|`k`).
- `b` = block size (8|`b`). For this algorithm we require `IOLEN` to be an integer multiple of `b`.
- `c` = counter size (8|`b`). It must be `c < b`.

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index

- **Internal State** (Content):
  - `key`: `k` bits.
  - `ctr`: `c` bits.
  - `IV`: `b-c` bits.
  - `auth_key`, `tag`: `b` bits.
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index
ii	`c`/8	`ctr`
iii	(`b-c`)/8	`IV`
iv	`b`/8	`tag`

There are no fields for the lengths of the AD and of the plaintext, as they are maintained by the calling software. Also, `hash_key` is not serialized, and thus it is recomputed upon `ace.import`.

Algorithm-Specific Functions

- `enc_blk(key, p) → c` encrypts block `p` using the key, and returns the result `c`.
- `absorb(data)` absorbs data into the hash function by performing:
  1. `tag ← tag xor INPUT`.

2. `tag ← Galoismul(tag, hash_key)`.

State Machine

- **States:**
  - *initial*: The CC is ready.
  - *hash_absorb*: Process Associated Data (AD) into the tag.
  - *encrypt*: Encrypt the plaintext and absorb the ciphertext into the tag.
  - *enc_tag_finalize*: Finalize the encryption tag, prepare to return its value.
  - *decrypt*: Absorb the ciphertext into the tag and decrypt it.
  - *dec_tag_finalize*: Finalize the encryption tag and keep it internally.
  - *hash_verify*: Compare the tag against a user supplied value.
  - *completed*: The algorithm has completed its operation correctly.
- **Allowed State Transitions:**
  - *initial* → *hash_absorb*.
  - *hash_absorb* → *encrypt*, if encryption is allowed.
  - *hash_absorb* → *decrypt*, if decryption is allowed.
  - *encrypt* → *enc_tag_finalize* → *completed*.
  - *decrypt* → *dec_tag_finalize* → *hash_verify* → *completed*.
- **Behavior:**
  - In State *initial* :
    - No `ace.exec` operation is allowed.
    - The following initialization operations are performed:
      1. `hash_key ← enc_blk(key, zeros(b))`,
      2. `tag ← zeros(b)`,
      3. `ctr ← zeros(c-1) @ 1`, and
      4. `IV ← zeros(b)`.
  - To transition to State *hash_absorb*: the `ace.state` instruction must be a Form C to set the IV, i.e., the fixed part of the counter. Then:
    1. `IV ← INPUT[b-c-1:0]`.
       - If the `ace.state` instruction is repeated, it will just update the counter again.
  - If the AD is empty, the transition to State *hash_absorb* is necessary to initialize the counter.
  - In State *hash_absorb*, only Form B of `ace.exec` is allowed. "`ace.exec Kn|K(Xn), INPUT`" absorbs a data block in `tag`, i.e.:

    1. `absorb(INPUT)`.

    If `IOLEN` > `b`, the above operation is executed `IOLEN/b` times.

- In State *encrypt*, only `ace.exec` instructions of Form A are allowed. "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" encrypts `INPUT` into `OUTPUT`, and absorbs `OUTPUT` into the `tag`, namely:

  1. If `ctr == ones(c-1)`, then
     a. the CR is **invalidated**.
  2. else
     a. `tmp ← INPUT xor enc_blk(key, IV @ ctr)`.
     b. `absorb(tmp)`.
     c. `ctr ← ctr + 1 % 2`^{c}.
     d. `OUTPUT ← tmp`.

  If `IOLEN` is a multiple of `b`, the above operations are executed `IOLEN/b` times, encrypting each `b`-bit block of `INPUT` into the corresponding `b`-bit block of `OUTPUT`.

- To transition to State *enc_tag_finalize*, a Form C `ace.state` instruction must be used to provide the following value, which must be set by the software:

  `INPUT ← bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2)` .

  The authentication tag is computed as follows:

  1. `absorb(INPUT)`.
  2. `tag ← tag xor enc_blk(key, IV @ zeros(c))`.

  Finally, `tag` is returned in `OUTPUT`, and the state is changed to *completed*.
  If `IOLEN` > `b`, only the `b` least significant bits of `INPUT` are considered.

- In State *enc_tag_finalize*, only `ace.exec` instructions of Form C are allowed.
  "`ace.exec OUTPUT, Kn|K(Xn)`" writes the `tag` to `OUTPUT`, and the state transitions to *completed*.

- In State *decrypt*, only `ace.exec` instructions of Form A are allowed.
  `ace.exec OUTPUT, Kn|K(Xn), INPUT` absorbs the ciphertext `INPUT` into the `tag`, then decrypts `INPUT` into `OUTPUT`, namely:

  1. If `ctr == ones(c-1)`, then
     a. the CR is **invalidated**.
  2. else
     a. `absorb(INPUT)`.
     b. `OUTPUT ← INPUT xor enc_blk(key, IV @ ctr)`.
     c. `ctr ← ctr + 1 % 2`^{c}.

  If `IOLEN` is a multiple of `b`, the same behavior applies as in State *encrypt*, namely the above operations are executed `IOLEN/b` times, decrypting each `b`-bit block of `INPUT` into the corresponding `b`-bit block of `OUTPUT`.

- To transition to State *dec_tag_finalize*, a Form C `ace.state` instruction must be used to provide the following value, which must be set by the software:

  ```
  INPUT ← bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2)
  ```

  The authentication tag is computed as follows:

  1. `absorb(INPUT)`.
  2. `tag ← tag xor enc_blk(key, ctr[b-1,c] @ zeros(c))`.

  If `IOLEN` > `b`, only the `b` least significant bits of `INPUT` are considered.

- To transition to State *hash_verify*, a Form C `ace.state` instruction must be used to provide the value against which the internally computed tag must be compared: If the values match, then `aceerror` is set to ace_err_no_error, else an ACE error ace_err_auth is raised. The state transitions to *completed*.
- Upon completion of `ace.import`, the ACE unit recomputes `hash_key`, i.e.:
  1. `hash_key ← enc_blk(key, zeros(b))`.

Listing 8 and Listing 9 provide pseudocode for GCM encryption and decryption.

> GCM security is not compromised if `H` is revealed. Consequently, an implementation of GCM based on an ECB mode CC would be acceptable, for instance as in Listing 10. However, this significantly increases code size and requires support for carry-less multiplication, which may not be available in all hardware.
>
> Furthermore, a GCM implementation is necessary to support the use case where the IV is also set in advance and not modifiable by the user. Thus, we need to be able to configure a GCM CC with both secret key and the IV. This is described in the next section.

2.9.3.8. GCM with Set IV

The mode of operation "GCM with a Set IV" is a variant of the GCM mode where the initialization vector (IV) is predefined and not generated randomly. This variant *cannot* be emulated with an ECB CC because it would reveal the internal data.

A key difference between GCM and GCM with Set IV is in the format of the Provisioning Input:

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index
ii	(`b`-`c`)/8	`IV`
iii	(Padding)	Zero padding

- **Internal State** (Content):

Same as for GCM.

- **Serialized Content**:
  Same as for GCM.
- **States and Allowed Transitions:**
  Same as for GCM.
- **State Machine Behavior:**
  The differences with respect to GCM are:
  - To transition to State *hash_absorb*: The `ace.state` instruction shall be of Form A, i.e., *without any additional inputs*, since the IV is already included in the PI.
  - When transitioning back to State *hash_initial*: The CR's `IV` field is *not* reset to zero.

> For all the following IV- or nonce-based modes, versions with a set IV or nonce (such as for GCM-SIV) can also be similarly defined. For instance, for GCM-SIV (Section 2.9.3.9), `nonce` would be included in the PI exactly as `IV` is included in the PI for GCM, and the transition to *hash_absorb* would also use a Form A `ace.state`. All other semantics would be unchanged.

2.9.3.9. GCM-SIV Mode

The GCM-SIV mode is described in cite:[DBLP-journals-iacr-GueronLL17] and cite:[RFC8452]. While GCM-SIV's decryption state machine is similar to the GCM's, the encryption state machine is different:

- During encryption, the *hash_absorb* state absorbs the entire associated data and plaintext. The *enc_tag_finalize* state then absorbs the lengths and produces the SIV/tag. Finally, the *encrypt* state performs a second pass to encrypt the plaintext using the SIV/tag as the initialization vector.
- During decryption, the *hash_absorb* state absorbs only the associated data. The *decrypt* state decrypts the ciphertext and simultaneously absorbs the plaintext into the hash. Finally, the *dec_tag_finalize* state absorbs the lengths, produces the SIV/tag, and internally compares it to the provided SIV/tag.

Parameters

- `k` = key size (64|`k`).
- `b` = block size, (64|`b`). For this algorithm we require `IOLEN` to be an integer multiple of `b`.
- It must be `k = b` or `k = 2b`.
- It must be established by the algorithm whether the authentication tag is `b` or `3b/4` bits long.

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index

- **Internal State** (Content):

- `key`, `enc_key`: `k` bits.
- `auth_key`: `b` bits.
- `nonce`: `3b/4` bits.
- `ctr`: `b/4` bits.
- `tag`: `b` bits.
- `SIV`: `b` or `3b/4` bits.

  A `b`-bit `SIV` is computed by clearing the most significant bit of `tag` and encrypting it.
  For a `3b/4`-bit `SIV`, a full `b`-bit `tag` value is kept while absorbing data. The public `SIV` value is computed by clearing the most significant bit of `tag`, encrypting it, and then removing the `b/4` least significant bits.

- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index
ii	3`b`/32	`nonce`
iii	`b`/32	`ctr`
iv	`b`/8	`tag`
v	`b`/8	`SIV`

> The internal state fields `enc_key` and `auth_key` are not serialized, and must be recomputed upon each `ace.import`.

Algorithm-Specific Functions

- `enc_blk(key, p) → c` encrypts block `p` using the key, and returns the result `c`.
- `absorb(data)` absorbs data into the hash function by performing a step of (a variant of) GHASH or POLYVAL as specified by the algorithm (for instance, AES-GCM-SIV uses POLYVAL):
  1. `tag ← tag xor INPUT`.
  2. `tag ← Montmul(tag, hash_key)` or `Galoismul(tag, hash_key)`, depending on the algorithm.

State Machine

- **States:**
  - *initial*: The CC is ready.
  - *set_aux_value:* Set the value of `nonce`.
  - *hash_absorb*: Set the SIV and process AD and, if encrypting, also the plaintext into the tag.
  - *encrypt*: Encrypt the plaintext.
  - *enc_tag_finalize*: Finalize the tag with the AD and PT lengths, prepare to return its value.
  - *decrypt*: Absorb the ciphertext into the tag and decrypt it.
  - *dec_tag_finalize*: Finalize the tag with the AD and CT lengths, then compare it against the `SIV`.

- *completed*: The algorithm has completed its operation.

- **Allowed State Transitions:**
  - *initial* → *set_aux_value* → *hash_absorb*.
  - For encryption: *hash_absorb* → *enc_tag_finalize* → *encrypt*.
  - For decryption: *hash_absorb* → *decrypt* → *dec_tag_finalize* → *completed*.
  - Transition to *encrypt* possible only if encryption is allowed.
  - Transition to *decrypt* possible only if decryption is allowed.
- **Behavior:**
  - In State *initial* :
    - Upon entering the state, the following initialization operations are performed:
      1. (`enc_key`, `auth_key`) ← `RFC8452_RFC8452_KeyDeriv(key)`,
      2. `nonce`, `ctr`, `tag`, `SIV ← 0`.
    - No `ace.exec` operation is allowed.
    - Upon issuing a Form C `ace.state Kn|K(Xn), #ace_state_set_aux_value, INPUT`, the value of `nonce` is set to `INPUT`. (If the instruction is repeated, there is no harm, since it just overwrites the `nonce` before any other operation is performed.)
  - In State _set_aux_value:
    - Upon issuing a Form C `ace.state Kn|K(Xn), #ace_state_hash_absorb, INPUT`:
      - The value of `SIV` is set to `INPUT`.

> This version of the instruction is meant to be used during decryption. It is harmless to invoke it during encryption, however, as the `SIV` field will be overwritten later with the correct synthetic value.

    - Upon issuing a Form A `ace.state Kn|K(Xn), #ace_state_hash_absorb`:
      - The `SIV` is not modified (it is assumed to be already zero).

> This version of the instruction is meant to be used during encryption. If used during decryption, by leaving the `SIV` to zero it will most likely lead to an authentication error.

  - In State *hash_absorb*, only Form B of `ace.exec` is allowed. This instruction absorbs its `INPUT` in the `tag`, i.e., it performs `absorb(INPUT)`.
    If `IOLEN` is a multiple of `b`, `absorb()` is executed on each of the `IOLEN/b` blocks in `INPUT`.
  - In State *enc_tag_finalize*, only an `ace.exec` instruction of Form A is allowed. The value

    `INPUT ← bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2)`,

    is set by the user. Then, "`ace.exec`" finalizes the SIV/tag computation by performing:

1. `absorb(INPUT)`.
2. `tmp ← enc_blk(enc_key, 0 @ (tag xor N)[b-2:0])`
3. If tag length is `b` bits, then
   a. `SIV ← tmp` and `OUTPUT ← tmp`,
4. else (tag length is `3b/4` bits)
   a. `SIV ← tmp[b-1:b/4]` and `OUTPUT ← (tmp >> b/4)`.

If `IOLEN` > `b`, only the `b` least significant bits of `INPUT` are considered.

- In State *encrypt*, only Form A of `ace.exec` is allowed, namely "`ace.exec OUTPUT, Kn|K(Xn), INPUT`," to encrypt `INPUT` into `OUTPUT` as follows:

1. If `ctr == 0`, then the CR is **invalidated**.
2. If the `SIV` is `b`-bits long, then
   a. `OUTPUT ← INPUT`
      `xor enc_blk(enc_key, 1 @ SIV[b-2:b/4] @ (SIV[b/4-1:0] + ctr % 2`$^{b/4}$`)),`
3. else (the `SIV` is `3b/4`-bits long)
   a. `OUTPUT ← INPUT xor enc_blk(enc_key, 1 @ SIV[3b/4-2:0] @ ctr)`.
4. `ctr ← ctr + 1`.

If `IOLEN` > `b`, the above operations are applied to each of the `IOLEN/b` blocks in `INPUT`.

- In State *decrypt*, only Form A of `ace.exec` is allowed. "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" decrypts `INPUT` into `OUTPUT` and then absorbs the plaintext `OUTPUT` into the `tag`:

1. If `ctr == 0`, then the CR is **invalidated**.
2. If the `SIV` is `b`-bits long, then
   a. `OUTPUT ← INPUT`
      `xor enc_blk(enc_key, 1 @ SIV[b-2:b/4] @ (SIV[b/4-1:0] + ctr % 2`$^{b/4}$`)),`
3. else (it is `3b/4`-bits long)
   a. `OUTPUT ← INPUT xor enc_blk(enc_key, 1 @ SIV[3b/4-2:0] @ ctr`.
4. `absorb(OUTPUT)`.
5. `ctr ← ctr + 1`.

If `IOLEN` > `b`, the above operations are applied to each of the `IOLEN/b` blocks in `INPUT`.

- In State *dec_tag_finalize*, only an `ace.exec` instruction of Form B, is allowed, namely "`ace.exec Kn|K(Xn), INPUT`," where it is the responsibility of the user to set

  `INPUT ← bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2)`.

  `ace.exec` then performs the following operations:

1. `absorb(INPUT)` (which modifies `tag`).

2. `tmp ← enc_blk(enc_key, 0 @ (tag xor N)[b-2:0])`.
3. If tag length is `b` bits, then
   a. `tag ← tmp`,
4. else (tag length is `3b/4` bits)
   a. `tag ← tmp[3b/4-1:0] @ zeros(b/4)`.
   b. If `tag` and `SIV` match, then
      i. `aceerror` ← ace_err_no_error,
   c. else
      i. `aceerror` ← ace_err_auth.
5. The state transitions to *completed*.

If `IOLEN` > `b`, only the `b` least significant bits of `INPUT` are considered.

- Upon completion of `ace.import`, the ACE unit recomputes `enc_key` and `auth_key`, i.e.:
  1. (`enc_key`, `auth_key`) ← `RFC8452_RFC8452_KeyDeriv(key)`.

Encryption and decryption pseudocode is found in Section 4.5.

2.9.3.10. OCB Mode

We only architect the OCB3 variant of OCB as specified in RFC 7253 cite:[RFC7253] and cite:[DBLP-journals-joc-KrovetzR21]. While this algorithm is asymptotically more efficient than GCM and GCM-SIV, it has a much heavier setup and its exported state requires significantly more memory than the other modes. On the other hand, since it uses only a single key and no derived keys, with ciphers with an expensive key expansion like the AES it may use fewer internal resources.

The state machine of this algorithm closely follows RFC 7253 (except for the numbering of the bit fields, which is big endian in the RFC). The algorithm is identical to the RFC for 128-bit-wide block ciphers, with a straightforward generalization to other widths according to the security proof of the original algorithm and a lower bound on nonce length following cite:[cryptoeprint-2023-326].

Parameters

- `b` = block size. `b` is a power of two, and ≥ 64. For OCB, we require `IOLEN` to be an integer multiple of `b`.
- `k` = key size, `b`|`k`.

Also, define `g` = `log_2(b)`.

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index

- **Internal State** (Content):

- `key`: `k` bits.
- `N`: `u+1` bits (at most `u` bits are used).
- `L`, `offset`, `hash_A`, `hash_P` : `b` bits.
- `N_len`, `tag_len`, `last_blk_len`: `g` bits.

- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index
ii	`b`/8	`N`
iii	`b`/8	`L` (only `u+1` bits used)
iv	`b`/8	`offset`
v	`b`/8	`hash_A`
vi	`b`/8	`hash_P`
vii	`2`	`tag_len`
viii	`2`	`last_blk_len`
ix	`2`	`N_len`
x	`2`	Padding

Algorithm-Specific Functions

- `enc_blk(K,p) → c` encrypts block `p` using the key `K` and returns the result `c`.
- `dec_blk(K,c) → p` decrypts block `c` using the key `K` and returns the result `p`.
- `double(S)`: Let `C` be `0x1B` for `b` = 64, `0x87` for `b` = 128, and `0x425` for `b` = 256, then:

  If S[`b`-1] == 0, then { `double(S) ← S << 1` } else { `double(S) ← (S << 1) xor C` } .

State Machine

- **States:**
  - *initial*: The CC is ready.
  - *set_aux_value:* Set the length and the value of the nonce.
  - *hash_absorb*: Set the tag length, and process the AD.
  - *hash_finalize*: Set the length of the final block of the AD and process it.
  - *encrypt*: Process the plaintext by absorbing it into the tag and encrypting it.
  - *enc_last_block*: Provide the last block of the plaintext, then process it.
  - *enc_tag_finalize*: Return the tag.
  - *decrypt*: Process the ciphertext by decrypting it and absorbing the plaintext into the tag.
  - *dec_last_block*: Provide the last block of the ciphertext, then process it.
  - *hash_verify*: Compare a user-provided value with the internally computed tag.
  - *completed*: The algorithm has completed its operation.

- **Allowed State Transitions:**
  - *initial* → *set_aux_value* → *hash_absorb* → *hash_finalize*
  - *hash_finalize* → *encrypt*, if encryption is allowed.
  - *hash_finalize* → *decrypt*, if decryption is allowed.
  - *encrypt* → *enc_last_block* → *enc_tag_finalize* → *completed*.
  - *decrypt* → *dec_last_block* → *hash_verify* → *completed*.
- **Behavior:**
  - In State *initial* :
    - No `ace.exec` operation is allowed.
    - The following initialization operations are performed:
      1. `N ← 1 @ zeros(u)`.
      2. `hash_A ← zeros(b)`
      3. `hash_P ← zeros(b)`
  - To move to *set_aux_value*, a Form B `ace.state` must be used, where the auxiliary argument `Xs` sets the nonce length `N_len`. `N_len` must be at least `g-1`. If not, ACE error ace_err_invalid_value is raised, and the CR is **invalidated**.
  - In State *set_aux_value*, only `ace.exec` instructions of Form B, "`ace.exec Kn|K(Xn), INPUT`", are allowed, whereby `N` is set to `INPUT`, namely `N ← zeros(u-N_len) @ INPUT[N_len-1:0]`
    - If "`ace.exec Kn|K(Xn), INPUT`," is repeated, it will rewrite `N`.
    - If no such `ace.exec` is issued, `N` is not changed, which means that its most significant bit is 1. This condition results in an error when transitioning to *hash_absorb*.
  - To move to *hash_absorb*, a Form B `ace.state` must be used, where the auxiliary argument `Xs` sets the tag length `tag_len`.
    - If this argument (interpreted as an unsigned value) is larger than `b` or is equal to `0` (the latter occurs also if `X0` is passed as the auxiliary argument), an ace_err_invalid_value error is raised, and the CR is **invalidated**.
    - If `N[u] == 1`, this means that `N` was not set in State *set_aux_value*. We do not allow code that skips nonce setting, as this mode is not secure under nonce reuse, therefore in this case ACE error ace_err_invalid_value is raised, and the CR is **invalidated**.
    - Following operations are performed:
      1. `tag_len ← Xs`.
      2. `offset ← zeros(b)`
      3. `L ← enc_blk(key, zeros(b))`
      4. `L ← double(double(L))`
  - In State *hash_absorb*, only `ace.exec` instructions of Form B are allowed: "`ace.exec Kn|K(Xn), INPUT`" absorbs a data block in `tag`, i.e.:
    1. `L ← double(L)`

2. `offset ← offset xor L`
3. `hash_A ← hash_A xor enc_blk(key, INPUT xor offset)`

If `IOLEN` > `b`, the above operation is executed `IOLEN/b` times.

- To move to *hash_finalize*, a Form B `ace.state` operation sets the length of the final block of the AD in `last_blk_len`.
- In State *hash_finalize*, only one `ace.exec` instruction of Form B is allowed if `last_blk_len` != `0`. "`ace.exec Kn|K(Xn), INPUT`" which absorbs the last data block in `tag`, i.e.:

  - `tmp : bits(b)`

  1. `offset ← offset xor enc_blk(key, zeros(b))`
  2. `tmp ← (zeros(b-last_blk_len-1) @ 1 @ INPUT[last_blk_len-1:0]) xor offset`
  3. `hash_A ← hash_A xor enc_blk(K, tmp)`

  If `last_blk_len == 0`, however, no `ace.exec` can be executed, and only an `ace.state` instruction is allowed to enter State *encrypt* or *decrypt*.

- Upon entering states `encrypt` or `decrypt`, the "setup" and "init" stages of the OCB algorithm are performed:

  - `N_ext : bits(b)` (`N_ext` means "N, extended")
  - `Stretch : bits(3b/2)`
  - `bot : bits(g-1)`
  - `Ktop : bits(b)`

  1. `offset ← zeros(b)`
  2. `L ← enc_blk(key, zeros(b))`
  3. `L ← double(double(L))`
  4. `N_ext ← N[N_len-1:0] @ 1 @ zeros(b-g-1-N_len) @ bin(tag_len,g)`
  5. `bottom ← int(N_ext[b-1:b-g+1])` // Rogaway numbers the bits in the opposite order.
  6. `Ktop ← enc_blk(key, zeros(g-1) @ N_ext[b-g:0])`
  7. `Stretch ← (Ktop[b/2-1:0] xor Ktop[b/2+7:8]) @ Ktop`
  8. `offset ← Stretch[b+bottom:bottom]`

- In State *encrypt*, only `ace.exec` instructions of Form A are allowed: "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" absorbs the plaintext `INPUT` into the `tag` and encrypts `INPUT` into `OUTPUT`, namely:

  1. `L ← double(L)`
  2. `offset ← offset xor L`
  3. `hash_P ← hash_P xor INPUT`

4. `OUTPUT ← offset xor enc_blk(key, INPUT xor offset)`

If `IOLEN > b`, the above operation sequence is executed `IOLEN/b` times, encrypting each `b`-bit block of `INPUT` into the corresponding `b`-bit block of `OUTPUT`.

- To move to *enc_last_block*, a Form B `ace.state` instruction must be used to set the bit length of the final block of the plaintext in `last_blk_len`.
- In State *enc_last_block*, only an `ace.exec` instruction of Form A "`ace.exec OUTPUT, Kn|K(Xn), INPUT`," is allowed if `last_blk_len` is nonzero. In this case, following operations are performed:

  - `tmp : bits(last_blk_len)`
  - `tmp2 : bits(b)`
  - `aux : bits(b)`

  1. `aux ← enc_blk(key, zeros(b))`
  2. `offset ← offset xor aux`
  3. `tmp ← enc_blk(key, offset)[last_blk_len-1:0]`
  4. `tmp ← INPUT[last_blk_len-1:0] xor tmp`
  5. `OUTPUT ← zeros(b-last_blk_len) @ tmp`
  6. `tmp2 ← (zeros(b-last_blk_len-1) @ 1 @ tmp) xor offset`
  7. `aux ← double(aux)`
  8. `hash_P ← enc_blk(key, hash_P xor tmp2 xor offset xor aux) xor hash_A`

  On the other hand, if `last_blk_len` is zero, only an `ace.exec` instruction of Form D, "`ace.exec Kn|K(Xn)`," is allowed, which performs:

  - `aux : bits(b)`

  1. `aux ← enc_blk(key, zeros(b))`
  2. `aux ← double(aux)`
  3. `hash_P ← enc_blk(key, hash_P xor offset xor aux) xor hash_A`

  In both cases the state then transitions to *enc_tag_finalize*.

- In State *enc_tag_finalize*, only an `ace.exec` instruction of Form C, "`ace.exec OUTPUT,Kn|K(Xn)`," is allowed, which performs `OUTPUT ← zeros(b-tag_len) @ hash_P[tag_len-1:0]`.
- In State *decrypt*, only `ace.exec` instructions of Form A are allowed. Such an `ace.exec` decrypts `INPUT` into `OUTPUT`, and absorbs the plaintext `OUTPUT` into the `tag`, namely:

  - `tmp : bits(b)`

  1. `L ← double(L)`
  2. `offset ← offset xor L`

3. `tmp ← offset xor enc_blk(key, INPUT xor offset)`
4. `hash_P ← hash_P xor tmp`
5. `OUTPUT ← tmp`

If `IOLEN` > `b`, the same behavior applies as in State *encrypt*.

- To move to State *dec_last_block*, a Form B `ace.state` operation must be used to also contextually set the bit length of the final block of the ciphertext in `last_blk_len`.
- In State *dec_last_block*, only an `ace.exec` instruction of Form A "`ace.exec OUTPUT, Kn|K(Xn), INPUT`," is allowed if `last_blk_len` is nonzero. In this case, the same operations as in State *enc_last_block* are performed except for Line 6, which is instead

  6. `tmp2 ← (zeros(b-last_blk_len-1) @ 1 @ INPUT[last_blk_len-1:0]) xor offset`

  and the state is changed to *hash_verify*.

  If `last_blk_len` is zero, only an `ace.exec` instruction of Form D, "`ace.exec Kn|K(Xn)`," is allowed, which performs the same operations as in State *enc_last_block*, except for the final state transition, which also in this case is to *hash_verify*.

- To complete the algorithm in the decryption branch, a Form C `ace.state` operation must be used to (confirm the) transition to State *hash_verify* and compare the `tag_len` least significant bits of `INPUT` with the `tag_len` least significant bits of `hash_P`:

  1. If the values match, then `aceerror` is set to ace_err_no_error,
  2. else `aceerror` is set to ace_err_auth.
  3. The state transitions to *completed*.

Pseudocode for OCB encryption is given in Section 4.6.

2.9.3.11. CMAC Functions

CMAC is an algorithm for message authentication codes constructed from a block cipher. It was designed by Tetsu Iwata and Kaoru Kurosawa and became a NIST Recommendation in May 2005 cite:[nist-SP-800-38B].

Parameters

- `b` = block size (8|`b`). For this algorithm we require `IOLEN` to be an integer multiple of `b`.
- `k` = key size (8|`k`).

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index

- **Internal State** (Content):

- `key`: `k` bits.
- `hash`: `b` bits.
- `last_block_len`: 64 bits.

- **Serialized Content**:

Pos.	Size in Bytes	Field
i	`k`/8 or 8	`key` or System Key index
ii	`b`/8	`hash`
iii	`8`	`last_block_len`

Algorithm-Specific Functions

- `enc_blk(key, p) → c` encrypts block `p` using the key `key`, and returns the result `c`.
- `gen_subkeys(K) → (K1,K2)` generates subkeys `K1` and `K2` from the master key `K` as follows: Let `C` be `0x1B` for `b` = 64, `0x87` for `b` = 128, and `0x425` for `b` = 256, then
  1. `L ← enc_blk(K, zeros(b))`
  2. if (`msb(L) == 0`) then { `K1 ← L << 1` } else { `K1 ← (L << 1) xor C` }.
  3. if (`msb(K1) == 0`) then { `K2 ← K1 << 1` } else { `K2 ← (K1 << 1) xor C` }.
  4. return `(K1, K2)`.

State Machine

- **States:**
  - *initial*: The CC is ready.
  - *hash_absorb*: Absorb data into the CMAC function.
  - *hash_last_block*: Set the length of the last block to absorb, then absorb it.
  - *hash_output*: Output the CMAC.
  - *completed*: The CC has completed the CMAC operation.
- **Allowed State Transitions:**
  - *initial* → *hash_absorb* → *hash_last_block* → *hash_output* → *completed*.
  - From any valid state to State *initial*.
- **Behavior:**
  - In State *initial*:
    - `last_block_len ← zeros(64)`
    - `hash ← zeros(b)`
    - No `ace.exec` operation is allowed.
  - In State *hash_absorb*, we can only issue `ace.exec` instructions of Form B. "`ace.exec Kn|K(Xn), INPUT`," absorbs the data block `INPUT` into the hash as follows:
    1. `hash ← enc_blk(key, hash xor INPUT)`.

If `IOLEN` > `b`, then the above operations are applied to each `b`-bit block of `INPUT`.

- Upon issuing a Form B "`ace.state Kn|K(Xn), #ace_state_hash_last_block, Xs`" where `Xs` contains the bit length of the last block, the following happens:
  1. if `last_block_len` != 0, then the CR is **invalidated**. (The previous block is not complete.)
  2. if `Xs > b` then the CR is **invalidated**.
  3. `last_block_len ← Xs`.
- In State *hash_last_block*, we can only issue a single `ace.exec` instruction of Form B, i.e., "`ace.exec Kn|K(Xn), INPUT`", to absorb the data block `INPUT`. If `IOLEN` > `b`, only the least significant `b` bits of `INPUT` are considered.
  1. `(K1, K2) ← gen_subkeys(key)`.
  2. if `last_block_len` == `b`, then {
     a. `tmp ← hash xor INPUT xor K1`
  3. } else {
     a. `tmp ← INPUT`
     b. Remove the `b - last_block_len` most significant bits of `tmp` and pad the resulting value with a `1` and sufficiently many `0` bits to make it `b` bits long.
     c. `tmp ← hash xor tmp xor K2` }
  4. `hash ← enc_blk(key, tmp)`
  5. The state is changed to *hash_output*.
- In State *hash_output*, we can issue one `ace.exec` instruction of Form C, i.e., "`ace.exec OUTPUT, Kn|K(Xn)`", to output the hash into `OUTPUT`. If `IOLEN` > `b`, only the bits of `OUTPUT` beyond the `b`-th are cleared. Upon issuing that instruction, the CC transitions to State *completed* with no error.

Pseudocode is given in Section 4.7.

2.9.3.12. Hash and MAC Functions, and XOFs

This section defines an interface for hash and MAC functions. A completely general interface cannot be defined because of the variety of constructions. However, the one given here is sufficiently general to allow describing most common functions, such as, for instance, the SHA2 cite:[nist-fips-180-4], SHA3 and SHAKE cite:[nist-fips-202], as well as HMAC cite:[nist-fips-198-1].

Parameters

- `b` = block size (8|`b`). This is the size of an absorbed block and may be different from the size of the internal state that is kept by the function. For a sponge function, this value is the *rate*.
- `t` = tag len (8|`t`). This is the size of the digest, or of the block that is output each time in a XOF. The present description assumes `t` ≤ `b`.
- `n` = size of the internal state that is kept by the function (8|`n`). For a sponge function, `n-b` is the *capacity* of the function. Either the block size is an integer multiple of the vector length or the

vector length is an integer multiple of the block size.

- `k` = key size (8|`k`). Present only if keyed, i.e., a MAC.

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	If keyed: `k`/8, else 8	`key` or System Key index

- **Internal State** (Content):
  - `key`: `k` bits. Present only if keyed, i.e., a MAC.
  - `state`: `n` bits.
  - `last_block_len`: int. Present only if this Hash/MAC provides special treatment for a last block that goes beyond padding and appending the length. The latter must be handled by user software instead.
  - `input_base`: int. Offset within input/output to read from/write to.
  - `block_base`: int. Length of absorbed data so far for the current block. Only if needed by the algorithm.
  - `cumul_len`: int. Cumulative length of data absorbed so far (if the algorithm must process and incorporate it in the hash on its own).
  - `block`: `b` bits. Included only if a partial block must be maintained—for instance when the inputs are not directly absorbed into `state`.
- **Serialized Content**:

Pos.	Size in Bytes	Field	Notes
i	If keyed: `k`/8 or 8	`key` or SKT index	—
	If unkeyed: 0	none	—
ii	`n`/8	`state`	—
iii	2	`last_block_len`	Present only if this Hash/MAC has special internal treatment for a fractional last block that goes beyond padding and appending the length, which are handled by user software.
iv	2	`block_base`	Amount of data stored so far into `block`. Only if used by the algorithm, else zero padding.
v	4	`cumul_len`	Cumulative length of all data absorbed so far. Only if the algorithm must process and incorporate it in the hash on its own, else zero padding.
vi	8·⌈`block_base`/64⌉	`block`	Only if `block_base` is defined and != 0, and the block must be stored separately from the `state`.

Pos.	Size in Bytes	Field	Notes
vii	8	`acemarchid`	Only if 8·*AdditionalData* > `block_base`, which implies that some implementation-dependent Additional Data follows.

> Regarding the `last_block_len`/`block_base`/`cumul_len` group: if any of these fields is not used, it is replaced by a corresponding padding block. If all three are not used by the hashing algorithm, then the 64-bit block is omitted, and `block` is clearly also

Algorithm-Specific Functions

- `absorb()`: Process `block` into the hash or MAC.
- `absorb_last_block()`: Process `block` into the hash or MAC, using a function for the last block.
- `finalize()`: Process the `state` into the digest value. We assume this either re-uses `block` (the entire digest, or a unit of XOF output should not be bigger than that) or the digest will be part of `state`.
- `update()`: Updates the `state` during the digest output stage.

State Machine

- **States:** *initial*, *hash_absorb*, *hash_last_block*, *hash_output* and *completed*.
  - *initial*: The CC is ready.
  - *hash_absorb*: Absorb data into the hash function.
  - *hash_last_block*: Set the length of the last block to absorb, then absorb it.
  - *hash_output*: Output the hash.
  - *completed*: The CC has completed the operation (XOFs do not have this state).
- **Allowed State Transitions:**
  - *initial* → *hash_absorb* → *hash_last_block* → *hash_output* → *completed*.
  - From any valid state to State *initial*.
- **Behavior:**
  - In State *initial*:
    - `last_block_len`, `block_base` and `block` are set to zero.
    - `state`: the initial value is set as specified by the algorithm.
    - Any `ace.exec` will result in the CR being **invalidated**.
  - In State *hash_absorb*, we can only issue `ace.exec` instructions of Form B, i.e., "`ace.exec Kn|K(Xn), INPUT`", to absorb the data block `INPUT` into the hash as follows.

    1. `input_base ← 0` // or `acestart` upon resumption
    2. while `(input_base < IOLEN)` do { // as long as we have input to process
       a. `amount ← min(IOLEN-input_base, b-block_base)`
       b. `block[block_base + amount - 1 : block_base] ←`
          `← INPUT[input_base + amount - 1 : input_base]`

(If there is no `block` field, could also be something like
`state[block_base + amount - 1 : block_base] ←`
`← state[block_base + amount - 1 : block_base] +`
`+ INPUT[input_base + amount - 1 : input_base]`,
it could directly replace part of `state`, or some other function.)

c. `input_base ← input_base + amount`

d. `block_base ← block_base + amount`

e. if (`block_base` == `b`) then { // it's time to absorb the block

i. `absorb()`

ii. `block_base ← 0`

iii. **Here** the instruction may be interrupted, with `acestart ← input_base`.

} }

- Upon issuing a Form B "`ace.state Kn|K(Xn), #ace_state_hash_last_block, Xs`" where `Xs` contains the length of the last block, the following happens:

  1. if `block_base` != 0, then the CR is **invalidated**. (The previous block is not complete.)
  2. if `Xs > b`, then the CR is **invalidated**.
  3. if `Xs == 0`, then the CR is **invalidated**.
  4. `last_block_len ← Xs`.

  Then, state transitions to *hash_last_block* as expected.

- State *hash_last_block* is very similar to *hash_absorb*, the differences being:
  - `last_block_len` is used in place of `b` throughout.
  - `absorb_last_block()` is called instead of `absorb()` in Step 3.d.i.
  - The instruction is terminated after `absorb_last_block()`—only one block is absorbed—and the state transitions to *hash_output*.

- In State *hash_output* we have two possible situations: we are in a hash function or a XOF.
  - Upon entering the state, the following operations are first performed:
    1. `block[t - 1 : 0] ← finalize()`
    2. `block_base ← 0`
  - For each Form C `ace.exec` instruction "`ace.exec OUTPUT, Kn|K(Xn)`", the following operations are performed:
    - `amount : int`, amount of data to copy from `block` to `OUTPUT`
    - `#define output_base input_base`

    1. `output_base ← 0` // or `acestart` upon resumption
    2. while (`output_base` < `IOLEN`) do { // as long as there is room in the OUTPUT

       a. `amount ← min(IOLEN - output_base, t - block_base)`

b. `OUTPUT[output_base + amount - 1 : output_base] ←`
`← block[block_base + amount - 1 : block_base + amount]`

c. `output_base ← output_base + amount`

d. `block_base ← block_base + amount`

e. if (`block_base` == `t`) then {

i. if we are in a Hash function, transition to *completed*, and return.

ii. `update()`

iii. `block_base ← 0`

iv. **Here** the instruction can be interrupted, with `acestart ← output_base`.

} }

2.9.3.13. HMAC

Hash function acceleration is sufficient to compute HMACs, and no dedicated algorithms are needed for either performance or security reasons.

2.9.3.14. SHA-3 Family Specific Rules

We describe here the algorithms for the primitives in the SHA-3 family, which currently includes SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128 and SHAKE256. These primitives are designed around a special parameter choice of KECCAK cite:[DBLP-conf-eurocrypt-BertoniDPA13] standardized by NIST cite:[nist-fips-202]. These are instantiations of the algorithm specified in Section 2.9.3.12, and we need to describe only the differences with respect to the behavior described there.

Parameters

- `b` = rate (8 | `b`). It is `b` = 1600 - `c`, where `c` = 448, 512, 768, or 1024 is the capacity.
- `n` = 1600
- `t` = `c/2`

Data Structures

- **Provisioning Input** (Content): Empty.
- **Internal State** (Content):
  - `state`: `n` bits.
  - `last_block_len`: int.
  - `input_base`: int.
  - `block_base`: int.
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	200	`state`

Pos.	Size in Bytes	Field
ii	2	`last_block_len`
iii	2	`block_base`
iv	4	Padding

Algorithm-Specific Functions

- `P()`: computes `KECCAK-p[1600, 24]` on `state`.
- `absorb()`: in this case there is no explicit operation to copy data into `state`, as the inputs will be directly XORed into it, however `absorb` applies `P()` to the state.
- `absorb_last_block()`: this is a NOP.
- `finalize()`: this is a NOP.
- `update()`: updates the `state` during the digest output stage. It applies `P()` to the state.

State Machine

- **States:** *initial*, *hash_absorb*, *hash_last_block*, *hash_finalize*, *hash_output* and *completed*.
- **Allowed State Transitions:**
  - *initial* → *hash_absorb* → *hash_last_block* → *hash_finalize* → *hash_output* → *completed*.
  - From any valid state to State *initial*.
- **Behavior:**
  - In State *initial*, `state` is zeroed.
  - In State *hash_absorb*, the input data is directly XORed to `state`.
  - In State *hash_last_block*, the input data is directly XORed to `state`.
  - Upon transitioning to State *hash_output*:
    1. The suffix and padding string `S` is generated.
    2. If the bit length of `S` is `b-block_base`:
       a. `S` is directly XORed to rate part of the state, and
       b. `P()` is applied.
    3. If the bit length of `S` is `2b-block_base`:
       a. The first `b-block_base` bits are XORed to the rate part of the state,
       b. `P()` is applied,
       c. the remaining `b` bits are XORed to the rate part of the state, and
       d. `P()` is applied again.

The rest of the algorithm follows Section 2.9.3.12.

2.9.3.15. Ascon-AEAD128

Ascon is a public permutation cite:[DBLP-journals-joc-DobraunigEMS21] standardized by the NIST cite:[nist-SP-800-232] together with various cryptographic algorithms designed around it.

In all ASCON algorithms, the ordering of the bits must be verified!

Ascon-AEAD128 is an AEAD algorithm. In Ascon-AEAD128, it is the responsibility of the user to apply proper padding to the AD and plaintext. The inputs are therefore assumed to be properly padded. Similarly, it is the responsibility of the user to remove the padding from the decryption of the ciphertext to recover the original plaintext.

Parameters

- `b` = 128. For this algorithm we require `IOLEN` to be an integer multiple of `b`.
- `k` = 128.

Data Structures

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	16 or 8	`key` or System Key index

- **Internal State** (Content):
  - `key`: 128 bits, or 64 bits if it is a System Key index.
  - `state[0 .. 4]`: five 64-bit words, i.e., 320 bits.
  - `tag_len`: 8 bits as int.
  - `last_block_len`: 9 bits as int.
  - `AD_empty`: Boolean;
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	16 or 8	`key` or System Key index
ii	8	`state`[0]
iii	8	`state`[1]
iv	8	`state`[2]
v	8	`state`[3]
vi	8	`state`[4]
vii	2	`last_block_len`
viii	2	`tag_len`
ix	4	Padding

Algorithm-Specific Functions

- `ASCON(p)` applies `p` rounds of ASCON to the `state`.

State Machine

- **States:**

- *initial*: The CC is ready.
- *hash_absorb*: Set the nonce, and process the AD.
- *encrypt*: Encrypt the plaintext.
- *enc_last_block*: Encrypt the last block of the plaintext.
- *hash_output*: Return the tag.
- *decrypt*: Decrypt the ciphertext.
- *dec_last_block*: Decrypt the last block of the ciphertext.
- *hash_verify*: Compare a user-provided value with the internally computed tag.
- *completed*: The algorithm has completed its operation.

- **Allowed State Transitions:**
  - *initial* → *hash_absorb*.
  - *hash_absorb* → *encrypt*, if encryption is allowed.
  - *hash_absorb* → *decrypt*, if decryption is allowed.
  - *encrypt* → *enc_last_block* → *hash_output* → *completed*.
  - *decrypt* → *dec_last_block* → *hash_verify* → *completed*.
- **Behavior:**
  - In State *initial* :
    - No `ace.exec` operation is allowed.
    - The following initialization operations are performed:
      1. `state[0] ← 0x00001000808c0001`
      2. `state[1] ← key[63:0]`
      3. `state[2] ← key[127:64]`
      4. `state[3] ← zeros(64)`
      5. `state[4] ← zeros(64)`
      6. `tag_len ← 128`
      7. `AD_empty ← true`
  - To transition to State *hash_absorb*, a Form C `ace.state` must be used, whose `INPUT` sets the nonce of the algorithm as follows:

    1. `state[3] ← INPUT[63:0]`
    2. `state[4] ← INPUT[127:64]`
    3. `ASCON(12)`
    4. `state[3] ← state[3] xor key[63:0]`
    5. `state[4] ← state[4] xor key[127:64]`

    If `IOLEN` > 128, only the 128 least significant bits of `INPUT` are considered.

- In State *hash_absorb*, only `ace.exec` instructions of Form B are allowed. "`ace.exec Kn|K(Xn), INPUT`" absorbs a data block into the state, i.e.:

  1. `AD_empty ← false`
  2. `ASCON(8)`
  3. `state[0] ← state[0] xor INPUT[63:0]`
  4. `state[1] ← state[1] xor INPUT[127:64]`

  If `IOLEN` > 128, this sequence of four operations is executed `IOLEN/128` times, once for each 128-bit block of `INPUT`.

- Upon entering State *encrypt* or *decrypt*, the following domain separation operation is performed:

  1. `lsb(state[4]) ← lsb(state[4]) xor 1`

- In State *encrypt*, only `ace.exec` instructions of Form A are allowed. "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" encrypts `INPUT` into `OUTPUT`, namely:

  1. `state[0] ← state[0] xor INPUT[63:0]`
  2. `state[1] ← state[1] xor INPUT[127:64]`
  3. `OUTPUT ← state[1] @ state[0]`
  4. `ASCON(8)`

  If `IOLEN` is a multiple of 128, these operations are executed `IOLEN/128` times, encrypting each 128-bit block of `INPUT` into the corresponding 128-bit block of `OUTPUT`.

- In State *enc_last_block*, only one `ace.exec` instruction of Form A, to encrypt the last block is allowed. "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" performs

  1. `state[0] ← state[0] xor INPUT[63:0]`
  2. `state[1] ← state[1] xor INPUT[127:64]`
  3. `OUTPUT ← state[1] @ state[0]` // only the 128 lsbs of `OUTPUT` are written to.
  4. The state machine then transitions to State *hash_output*.

  In this state only one 128-bit block is processed.
  The differences between the operation of `ace.exec` in this state and in State *encrypt* are the absence of the final `ASCON(8)` and the automatic transition to State *hash_output*.

- In State *hash_output*, only one `ace.exec` instruction of Form C is allowed, to compute the tag. "`ace.exec OUTPUT, Kn|K(Xn)`" performs

  1. `state[2] ← state[2] xor key[63:0]`
  2. `state[3] ← state[3] xor key[127:64]`
  3. `ASCON(12)`
  4. `state[3] ← state[3] xor key[63:0]`

5. `state[4] ← state[4] xor key[127:64]`
6. `OUTPUT ← state[4] @ state[3]` // only the 128 lsbs of `OUTPUT` are written to.
7. The state machine then transitions to State *completed*.

In this state only one 128-bit block is processed.

- In State *decrypt*, only `ace.exec` instructions of Form A are allowed, i.e., `ace.exec OUTPUT,Kn|K(Xn),INPUT`. This operation performs:

  - `tmp : bits(128)`

  1. `tmp ← (state[0] xor INPUT[63:0]) @ (state[1] xor INPUT[127:64])`
  2. `state[0] ← INPUT[63:0]`
  3. `state[1] ← INPUT[127:64]`
  4. `ASCON(8)`
  5. `OUTPUT ← tmp`

  If `IOLEN` is a multiple of 128, these operations are executed `IOLEN/128` times, decrypting each 128-bit block of `INPUT` into the corresponding 128-bit block of `OUTPUT`.

- In State *dec_last_block*, only a Form A `ace.exec` instruction is allowed. "`ace.exec OUTPUT, Kn|K(Xn), INPUT`" performs:

  - `tmp : bits(128)`

  1. `tmp ← (state[1] xor INPUT[127:64]) @ (state[0] xor INPUT[63:0])`
  2. `state[0] ← INPUT[63:0]`
  3. `state[1] ← INPUT[127:64]`
  4. The state machine transitions to State *hash_verify*
  5. `OUTPUT ← tmp`

- In State *dec_tag_finalize*, a Form B `ace.state Kn|K(Xn), #ace_state_dec_tag_finalize, Xs` instruction may be executed, setting the tag len `tag_len` to the value of `Xs`, provided it is not smaller than 32 and not larger than 128. If the value is out of bounds, the instruction raises ACE error ace_err_invalid_value and the CR is **invalidated**.
- In State *hash_verify*, only one `ace.exec` instruction of Form D is allowed, to verify the tag. "`ace.exec Kn|K(Xn), INPUT`" performs

  1. `state[2] ← state[2] xor key[63:0]`
  2. `state[3] ← state[3] xor key[127:64]`
  3. `ASCON(12)`
  4. `state[3] ← state[3] xor key[63:0]`
  5. `state[4] ← state[4] xor key[127:64]`
  6. `INPUT[tag_len-1:0]` is compared to `(state[4] @ state[3])[tag_len-1:0]`.

7. If the values match, then `aceerror` is set to ace_err_no_error, else ACE error ace_err_auth is raised. The state is changed to *completed*.

2.9.3.16. Ascon-AEAD128 with set Nonce

Similarly to GCM with Set IV, Ascon-AEAD128 can also be made into a version with a set nonce. In this case the State *set_aux_value* is skipped, and the PI has the following form.

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	16 or 8	`key` or System Key index
ii	16	`nonce`

In State *initial*, words 3 and 4 of `state` are initialized thus:

1. `state[3] ← nonce[63:0]`
2. `state[4] ← nonce[127:64]`

Once in State *completed*, the CC cannot be brought back to State *initial*.

2.9.3.17. Ascon-AEAD128 with Nonce Masking

The NIST specification also describes a version of Ascon-AEAD128 with *Nonce Masking*. This variant is described as follows:

- **Provisioning Input** (Content):

Pos.	Size in Bytes	Field
i	16 or 8	`K1` or System Key index
ii	16 or 0	`K2` or System Key index in `K1`

- **Internal State** (Content):
  1. `K1`: 128 bits, or 64 bits if it is a System Key index.
  2. `K2`: 128 bits, omitted if a System Key is used.
     - `state[0 .. 4]`: five 64-bit words, i.e., 320 bits.
     - `tag_len`: 8 bits.
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	16 or 8	`K1` or System Key index
ii	16 or 0	`K2` or System Key index in `K1`
iii	8	`state`[0]
iv	8	`state`[1]

Pos.	Size in Bytes	Field
v	8	`state`[2]
vi	8	`state`[3]
vii	8	`state`[4]
viii	2	`tag_len`
ix	6	Padding

The algorithm has the same states as Ascon-AEAD128 as described in Section 2.9.3.15, with the key `key` equal to `K1` and the nonce `N` replaced throughout by `N xor K2`.

For encryption and decryption with both nonce masking and a set nonce, the algorithm described in Section 2.9.3.16 is used, giving `K1` as the key `key` and providing `N xor K2` as the nonce value.

2.9.3.18. Ascon-Hash256

In this mode it is the responsibility of the user to apply proper padding to the inputs.

Parameters

- `b` = 64. For this algorithm we require `IOLEN` to be an integer multiple of `b`.

Data Structures

- **Provisioning Input** (Content): Empty.
- **Internal State** (Content):
  - `state[0 .. 4]`: five 64-bit words, i.e., 320 bits.
  - `countdown` : integer going from 3 down to 0.
- **Serialized Content**:

Pos.	Size in Bytes	Field
i	8	`state`[0]
ii	8	`state`[1]
iii	8	`state`[2]
iv	8	`state`[3]
v	8	`state`[4]
vi	1	`countdown` (from 3 down to 0)
vii	7	Padding

Algorithm-Specific Functions

- `ASCON(p)` applies `p` rounds of ASCON to the `state`.

State Machine

- **States:** *initial*, *hash_absorb*, *hash_finalize*, and *completed*.
- **Allowed State Transitions:**

- *initial* → *hash_absorb* → *hash_finalize* → *completed*.

- **Behavior:**
  - In State *initial* :
    - No `ace.exec` operation is allowed.
    - The following initialization operations are performed:
      1. `state[0] ← 0x0000080100cc0002`
      2. `state[1] ← zeros(64)`
      3. `state[2] ← zeros(64)`
      4. `state[3] ← zeros(64)`
      5. `state[4] ← zeros(64)`
      6. `ASCON(12)`
  - In State *hash_absorb*, only `ace.exec` instructions of Form B are allowed. "`ace.exec Kn|K(Xn), INPUT`" absorbs a data block into the state, i.e.:

    1. `state[0] ← state[0] xor INPUT`
    2. `ASCON(12)`

    If `IOLEN` > `64`, these operations are executed `IOLEN/64` times, once for each `64`-bit block of `INPUT`.

  - Upon entering *hash_finalize*, `countdown` is set to 3.
  - In State *hash_finalize*, at most four `ace.exec` instructions of Form C are allowed. To compute the tag, "`ace.exec OUTPUT, Kn|K(Xn)`" performs

    1. `ASCON(12)`
    2. `OUTPUT ← state[0]`
    3. if `countdown == 0` then transition to State *completed*.
    4. `countdown ← countdown - 1`

    If `IOLEN` ≥ `128`, these operations are executed `IOLEN/64` times, writing to `OUTPUT`[63:0] the first time, then `OUTPUT`[127:64], `OUTPUT`[191:128] and `OUTPUT`[255:192]. If more than one `ace.exec` instructions are required, writing restarts from `OUTPUT`[63:0]. If `countdown` reaches zero before the entire output registers is written to, the state of the unwritten bits is undefined: for instance this may occur if `IOLEN` = `192` or `IOLEN` > `256`. This is usually not a problem, since the output vectors will be usually configured for 64-bit, 128-bit, or 256-bit blocks, in which case four, two, or one `ace.exec` instructions are required, respectively.

> There is no need to provide "keyed" versions of this algorithm. This can be done by a trusted entity by providing a CC after absorbing some secret content in it first.

2.9.3.19. Ascon-XOF128

The algorithm is the same as Ascon-Hash256 (Section 2.9.3.18) with the following differences:

1. The IV written to `state[0]` in State *initial* is `0x0000080100cc0002`.
2. There is no `countdown` field (and therefore also no padding: the last 8 bytes of the serialized content of Ascon-Hash256 are not present).
3. As much output as desired can be produced in State *hash_finalize*, and the algorithm never transitions to State *completed*.

2.9.3.20. Ascon-CXOF128

Ascon-CXOF128 differs from Ascon-XOF128 in that the message is prepended with the customization string. Except for the initialization of the state, Ascon-CXOF128 is identical to Ascon-XOF128 with customization string and message concatenated. The IV for Ascon-CXOF128 is `0x0000080000cc0004` (in place of `0x0000080000cc0003`), and it is set by the algorithm. The states and their semantics are otherwise identical to Ascon-XOF128, and the management of the customization string is left to the user.

2.9.4. Elliptic Curve Asymmetric Algorithms

> This section needs finalization. Currently, we only define scalar multiplication with known and unknown base point and ECDSA/EdDSA sign/verify for the NIST curves. The Edwards curves ed25519 and ed448 are missing (same for Brainpool and SM2). ECC based KEMs are also missing, or at least instructions on how to implement them with the basic functionality offered. Further curves and algorithms can be defined later.

Parameters

- For secp256r1 (NIST P-256) the size `b` of the main fields (coordinates, signature R and S fields, scalars) is 256 bits. The length `h` of the hash used in the signature scheme is 256 bits. The random number used as nonce in signature generation is an `h`-bit string as well.
- For secp384r1 (NIST P-384), `b` is 384 bits, and `h` is 384 bits.
- For secp521r1 (NIST P-521), `b` is 576 bits, of which only 521 are used, and `h` is 512 bits.

Data Structures

- **Provisioning Input** (Content): Empty.

> In the Metadata, Algorithm only defines a curve. AlgorithmPolicy enables signature generation (bit 12) and verification (bit 13), where all four combinations are admissible.

- **Internal State** (Content):
  - `HasGenerator`: Bool.
  - `HasSecondPoint`: Bool.
  - `HasScalar`: Bool.

- `HasPublicKey`: Bool.
- `HasSignature`: Bool.
- `HasHash`: Bool.
- `HasRandomNumber`: Bool.
- `OutputType`: Bool.
- `Progress`: integer.
- `block_base`: integer.
- `Generator` (X, {Y}): one or two `b`-bit strings representing the X {and Y} coordinate{s}.
- `SecondPoint` (X, {Y}): one or two `b`-bit strings representing the X {and Y} coordinate{s}.
- `Scalar`: `b`-bit string.
- If signature generation or verification is allowed:
  - `PublicKey` (X, {Y}): one or two `b`-bit strings representing the X {and Y} coordinate{s}}.
  - `Signature` (R,S): two `b`-bit strings representing the `R` and `S` components.
  - `Hash`: a `h`-bit string representing the hash.
- If signature generation is allowed:
  - `RandomNumber`: a `h`-bit string representing the random number.
- Additional data, for instance precomputed multiples of the Generator.

- **Serialized Content**:

Pos.	Size in Bytes	Field	
i	2	Bit 0: `HasGenerator`	Bit 1: `HasSecondPoint`
		Bit 2: `HasScalar`	Bit 3: `HasPublicKey`
		Bit 4: `HasSignature`	Bit 5: `HasHash`
		Bit 6: `HasRandomNumber`	Bit 7: `OutputType`
ii	2	`Progress`	
iii	4	Padding	
iv	2`b`/8	`Generator`, X coordinate	
v	2`b`/8	`Generator`, Y coordinate	
vi	`b`/8	`Scalar`: `b`-bit string. Can be the private key.	

`SCC_content_size()` returns only the length of the content from Position i to Position vi.

The size of the following fields is included in the *AdditionalData* field of the Metadata Header.

Pos.	Size in Bytes	Field
vii	If `mh`.*AdditionalData* != 0: 8, else: 0	`acemarchid`

Pos.	Size in Bytes	Field
viii	If `HasSecondPoint`: `b`/8, else: 0	`SecondPoint`, X coordinate.
ix	If `HasSecondPoint`: `b`/8, else: 0	`SecondPoint`, Y coordinate. Does not exist if Edwards Curve.
x	If `HasPublicKey`: `b`/8, else: 0	`PublicKey`, X coordinate.
xi	If `HasPublicKey`: `b`/8, else: 0	`PublicKey`, Y coordinate. Does not exist if Edwards Curve.
xii	If `HasSignature`: `b`/8, else: 0	`Signature`, R value.
xiii	If `HasSignature`: `b`/8, else: 0	`Signature`, S value.
xiv	If `HasHash`: `h`/8, else: 0	`Hash`: a `h`-bit string representing the hash.
xv	If `HasRandomNumber`: `h`/8, else: 0	`RandomNumber`.
xvi	(variable)	Additional data from the **Internal State** (Content)..

The `HasRandomNumber` flag is set and the `RandomNumber` field is used only during signature generation. `RandomNumber` is exported only if the signature operation is interrupted.

The size in bytes of Fields vii to xv is:

$$\texttt{varlen} \leftarrow 8 + (\texttt{HasPublicKey} \mathrel{!=} 0) \cdot 2\texttt{b}/8 + (\texttt{HasSignature} \mathrel{!=} 0) \cdot 2\texttt{b}/8 + \\ + (\texttt{HasHash} \mathrel{!=} 0) \cdot \texttt{h}/8 + (\texttt{HasRandomNumber} \mathrel{!=} 0) \cdot \texttt{h}/8.$$

If *AdditionalData* > `varlen`, Position xvi contains the microarchitectural dependent part, which starts with `acemarchid`.

State Machine

- **States:** Beside *initial* (0), *invalid* (14), and *completed* (15), ECC algorithms define:
  - *set_generator* (1)
  - *set_scalar* (2)
  - *gen_rnd_scalar* (3)
  - *point_mul* (4)
  - *sign_generate* (5)
  - *sign_verify* (6)
  - *set_hash* (7)
  - *set_publickey* (8)
  - *set_signature* (9)
  - *output* (10)
- **Allowed State Transitions:**
  - *initial* → *set_generator*, *set_scalar*, *set_hash*, *set_publickey*, *sign_generate*, *set_signature*.
  - One can freely transition between *set_generator*, *set_scalar*, *set_hash*, and *set_publickey*.

- From any of *initial*, *set_generator*, *set_scalar*, *set_hash*, *set_publickey* →
  - *point_mul* only if `HasGenerator` and `HasScalar`;
  - *sign_generate* only if `HasGenerator`, `HasPublicKey`, and `HasHash`;
  - *sign_verify* only if `HasGenerator`, `HasScalar`, `HasPublicKey`, `HasHash`, and `HasSignature`.
- *point_mul* → *output* → *completed*.
- *sign_generate* → *output* → *completed*.
- *sign_verify* → *completed*.

In State *set_generator* we can set a group generator, or a default one.

In State *set_scalar* we can either set a scalar for scalar multiplication or private key.

For each *Var* □ {`Generator`, `Scalar`, `Hash`, `PublicKey`, `Signature`}, the Boolean variable `Has`*Var* is set when all operations required in State *set_Var* have been successfully executed.

- **Behavior:**
  - All the "set" and "output" operations work like the input and output of blocks in the Hash functions, in other words, they use `Progress` (just as the Hash/MAC functions use `block_base`) to track how much information has been read in or written out.
  - `Progress` can be used by instructions such as Point Multiplication, Signature Generation or Verification, to track the progress of the operations in case they are interrupted. In this case, if the CR is exported, in the Metadata, *AdditionalData* must be non-zero, to allow `acemarchid` to be compared, and then the Additional Data must also contain sufficient information to allow resumption. If there is a `acemarchid` mismatch, both `RandomNumber` (Position xv, if present) and the additional data at Position xvi are discarded, and the operation is restarted.
  - To transition to State *set_generator*, a Form A or B `ace.state` is used.
    - If no parameter is given, or the value of the GPR used in the Form B `ace.state` is zero:
      1. The default generator of the curve is set.
      2. A state change operation can be issued, including to states that require a set `Generator`.
    - Otherwise:
      1. If `Generator` contains a value, it is erased and `HasGenerator` is set to False.
      2. `Progress` ← 0.
      3. A value for `Generator` is loaded, using one or more Form B `ace.exec` operations, each one increasing `Progress` by the amount of data provided by `ace.exec`.
      4. Once `Progress` has reached the length of the generator point (excess data in the last `ace.exec` is ignored):
         - `HasGenerator` is set to True.
         - No further `ace.exec` instructions may be issued.
         - A state change operation can be issued, including to states that require a set `Generator`. This state change also clears `Progress`.

- To transition to any of the States *set_scalar*, *set_hash*, *set_publickey*, or *set_signature* a Form A `ace.state` is used.
  1. If there is a current value in the corresponding field X = `Scalar`, `Hash`, `PublicKey`, or `Signature`, it is erased, and the Boolean `Has`X is set to False.
  2. `Progress` ← 0, and a value for Field X is loaded, using one o more Form B `ace.exec` operations (similarly to `Generator`).
  3. Once the values `Progress` has reached the length of field X (excess data in the last `ace.exec` is ignored):
     - The corresponding Boolean `Has`X is set to True.
     - No further `ace.exec` instructions may be issued.
     - A state change operation can be issued, including to states that require a value for X to be set. This state change also clears `Progress`.
- It is allowed to transition to *set_scalar* also using a Form B `ace.state` with a non-zero value in the GPR. In this case, a random value is set, and `HasScalar` is set to true.
- When the operations in states *point_mul* and *sign_generate* are completed:
  1. If the output is a point, then `OutputType` ← False, and if the output is a signature, then `OutputType` ← True. This allows the ACE implementation to determine not only which output must be written out, but also its length.
     - If `OutputType` == False, the output is in `SecondPoint`.
     - If `OutputType` == True, the output is in `Signature`.
  2. `Progress` ← 0.
  3. One or more Form C `ace.exec` instructions may be issued to export segments of the result to memory, each time increasing `Progress` by the length of the output of `ace.exec`.
  4. Data beyond the length of the output is replaced with zeros in the last exported part.
  5. Once `Progress` has reached the length of the output, the state transitions automatically to *completed*.
- Upon returning to State *initial*:
  - If a Form B `ace.state` is used, the parameter `Xs` will determine whether the fields `Generator`, `Scalar`, and `PublicKey` are reset/erased or kept for further operations. This is determined by Bits 0, 1, 2, and 3 of `Xs` as follows:
    - If Bit 0, 1, 3, resp., 3 is set, then `Generator`, `SecondPoint`, `Scalar`, resp., `PublicKey` is erased, and the corresponding Boolean flags `HasGenerator`, `HasScalar`, resp., `HasPublicKey` is unset as well.
    - If Bit 4 is set, then `Generator` is copied onto `SecondPoint`. If Bit 0 is set, then `Generator` is erased afterwards and `HasGenerator` is set to False.
    - If Bit 5 is set, then `SecondPoint` is copied onto `Generator`. If Bit 1 is set, then `SecondPoint` is erased afterwards and `HasSecondPoint` is set to False.
  - A Form A `ace.state` does not modify these states.
  - `Signature` and the corresponding Boolean `HasSignature` are always reset.

EdDSA formats are similar, but, for instance, the public key is represented as a single `b`-bit string. Depending on the algorithm, in states *set_generator* or *set_publickey* we may input non-compressed or compressed points.

Chapter 3. Privileged Architecture

The ACE Privileged Architecture consists of CSRs and of optional custom instructions to implement parts of the unprivileged ISA.

3.1. Extensions Overview

This list is an early draft.

- If `Zlv` or `Zlio` are implemented: `misa` must implement the `L` bit, `*status` must implement `ACES`.
- `SmaceCSK` ⇒ Defines the Context Sealing Key CSRs (`macecsk{0..7}`), which are all MRO.
- `SmaceCSKw` ⇒ ACE Context Sealing Key CSRs are MRW. Depends on `SmaceCSK`.
- `Smacepbootscrt` ⇒ Defines the ACE OS Secret CSRs (`macePhysBootSecret{0..3}`).
- `Smacevbootscrt` ⇒ Defines the ACE Boot Secret CSRs (`haceVirtBootSecret0{0..3}`).
- `Smacestatus` ⇒ Defines the ACES per CC CSRs (`scrstatus`, `vscrstatus`, `mcrstatus`).

3.2. Dependencies on Other Extensions

The ACE Privileged Architecture depends on the Indirect CSR Access mechanism defined in Smcsrind/Sscsrind.

3.3. CSRs

ACE extends `*status` and `misa`, and provides up to 4 direct and up to 28 indirect CSRs.

Table 9. ACE Privileged CSRs.

0xXXX	SRW	scrstatus	Per-CR dirtiness and laziness tracking
0xXXX	HRW	vscrstatus	Per-CR dirtiness and laziness tracking
0xXXX	MRW	mcrstatus	Per-CR dirtiness and laziness tracking
0xXXX	SRW	sacetrapmask	Discuss: Mask to determine which ACE exceptions vertically trap
(Indirect)	MRW (RZ)	macecsk{0..7}	ACE Context Sealing Key
(Indirect)	MRW (RZ)	macePhysBootSecret{0..3}	Locality: Physical (host) Boot Session Secret
(Indirect)	HRW (RZ)	haceVirtBootSecret{0..3}	Locality: Virtual (guest) Boot Session Secret
(Indirect)	MRW (RZ)	maceLocality{0..3}	Locality: Mutually isolate processes below M-mode
(Indirect)	SRW (RZ)	haceLocality{0..3}	Locality: Mutually isolate processes below H-mode
(Indirect)	HRW (RZ)	saceLocality{0..3}	Locality: Mutually isolate U-mode processes

3.3.1. ACE field in `misa`

Bit `L` (11) of `misa` indicates that the ACE extension is present. If ACE is implemented, this bit shall not be read-only zero.

3.3.2. ACE Context Status in `mstatus` and `sstatus`

The ACES field is added to `mstatus`[26:25] and shadowed in `sstatus`[26:25] (see Section 3.3.6). ACES is defined analogously to the floating-point context status field FS and the vector context status field VS. It tracks the dirtiness of the entire ACE state, including CRs, ACEIOBUF, and all ACE-specific CSRs. If ACE is implemented, ACES shall not be read-only zero. Encodings are given in Table 10.

Table 10. Encoding of the ACES field

Value	Name	Meaning
0	Off	ACE disabled. The ACE state is inaccessible. Executing any ACE instruction raises an illegal instruction exception.
1	Initial	ACE enabled.
2	Clean	ACE enabled. At least one CR or ACEIOBUF is configured; state matches the last context swap.
3	Dirty	ACE enabled. At least one CR, ACEIOBUF, or ACE-specific CSR may have changed since the last context swap. Unconfigured CRs that were previously configured also cause Dirty, since context-switching code must erase any stale saved state.

ACES state transitions follow the same rules as FS, VS, and XS (see cite:[RISCV-ISA-Priv]).

> Precise management of `mstatus.ACES` is an optimization that software may use to reduce context-switch overhead.

When `mstatus.ACES` is Initial or Clean, any instruction that modifies ACE state sets `mstatus.ACES` to Dirty. Implementations may also set `mstatus.ACES` to Dirty from Initial or Clean at any time, even in the absence of a state change.

When `mstatus.ACES` (resp. `sstatus.ACES`) is Dirty, `mstatus.SD` (resp. `sstatus.SD`) is 1; otherwise, `mstatus.SD` is set per existing specifications.

Writes to ACES do not modify any other ACE state. Setting ACES to Off, Initial, or Clean does not clear CRs or ACEIOBUF; any state reset must be explicit and must precede such a write.

In implementations with a writable `misa.L`, the `mstatus.ACES` field may be present even when `misa.L` is clear, analogously to `mstatus.FS` and `mstatus.VS`.

3.3.3. ACE Context Status in `vsstatus`

When the H extension is present, an ACES field is added to `vsstatus`[26:25], defined analogously to `mstatus.FS` and `mstatus.VS`, and tracking the dirtiness of CRs, ACEIOBUF, and all ACE-specific CSRs.

When V=1:

- Both `vsstatus.ACES` and `mstatus.ACES` are in effect.
- Any ACE instruction raises an illegal instruction exception if either field is Off.
- If neither field is Off, any instruction that modifies ACE state sets both `mstatus.ACES` and `vsstatus.ACES` to Dirty.
- Implementations may set `ACES` to Dirty from Initial or Clean at any time, even without a state change.
- When `vsstatus.ACES` is Dirty, `vsstatus.SD` is 1; else, `vsstatus.SD` is set per existing specifications.

In implementations with a writable `misa.L`, `vsstatus.ACES` may be present even when `misa.L` is clear.

3.3.4. `scrstatus` for Dirtiness Tracking

The 64-bit register `scrstatus` tracks per-CR dirtiness.

`scrstatus[i]` encodes Clean (0) or Dirty (1) for CR #*i*.

Bits [63:32] are WPRI.

When the privilege mode is S, VS, or U:

- Any instruction that modifies CR #*i* sets `scrstatus[i]` to Dirty.
- Implementations may set `scrstatus[i]` to Dirty from Clean at any time, even without a state change.

> Context-switching code observes only Clean or Dirty. To determine whether a CR is configured, software may execute `ace.size` (Section 2.5.8) on it; a return value of zero indicates an unconfigured CR, and any stale saved state for it shall be erased and released.

> `scrstatus` is independent of the S extension. In an M-U system, M-mode may use it to optimize U-mode context switches.

> `sstatus.ACES` is also in effect.

3.3.5. `vscrstatus` for Dirtiness Tracking

The 64-bit register `vscrstatus` tracks per-CR dirtiness for virtualized contexts.

`vscrstatus[i]` encodes Clean (0) or Dirty (1) for CR #*i*.

Bits [63:32] are WPRI.

`vscrstatus` exists only when the H extension is implemented.

When the privilege mode is VS or VU:

- `vscrstatus` shadows `scrstatus`; accesses that would target `scrstatus` instead access `vscrstatus`.

- Any instruction that modifies CR #*i* sets `vscrstatus[i]` to Dirty.
- Implementations may set `vscrstatus[i]` to Dirty from Clean at any time, even without a state change.

`sstatus.ACES`, `vsstatus.ACES`, and `scrstatus` are also in effect when V=1.

3.3.6. `mcrstatus` for Dirtiness Tracking

A `mcrstatus` register may be defined, *mutatis mutandis*, in the same manner as `scrstatus` and `vscrstatus`, to allow M-mode to optimize context switches across Supervisor domains. In such a case, the `ACES` field in `sstatus` shall be unshadowed and be independent instead.

3.3.7. Extension of `*crstatus` for Lazy Loading

To support lazy loading of CR values, `scrstatus` and `vscrstatus` are extended to 64 bits. On RV32, 32-bit CSRs `scrstatush` and `vscrstatush` shadow, respetively, bits [63:32] of `scrstatus` and `vscrstatus`.

X`crstatus[i+32]` encodes Standard (0) or Lazy (1) for CR #*i*.

When X`crstatus[i+32]` is Lazy, the semantic of but X`crstatus[i]` changes as follows: X`crstatus[i]` is 0 to signal that the authoritative value of the CR has been saved in VS, or in S if V=0. X`crstatus[i]` is 1 to signal that the authoritative value of the CR has been saved in S and V=1.

These extensions to X`crstatus` address the use case where context switching code does not restore a CR upon state restore, but sets the X`crstatus[i+32]` bit instead to cause a trap on access `K(`*i*`)` and this trap serves to import the value. The optimization can be extremely efficient if all the trap handler has to do is to check whether the associated SCC can be imported and, if not, to free another CR first. The latter can use a random selection or an implementation-defined HW assisted mechanism that marks the longest untouched configured CR.

Therefore, it is important to know *which* environment has been the last one to export the authoritative value of `K(`*i*`)`.

This feature would be necessary also to support VM migration, where it may happen that a VM is migrated to a target device where the CRF capacity is insufficient to import all the SCCs saved by the hypervisor.

We recommend to define two CSRs: one to set the ad-hoc handler for lazy loading, and one to store the number of the CR to lazy-load.

3.3.8. Propagation of `*crstatus` Status Values

Per-CR status propagates from less-privileged to more-privileged modes, analogously to the FS, VS, and ACES fields in `*status`, but with only two states: Clean and Dirty. Any instruction that sets bit #*i* of `vscrstatus` (resp. `scrstatus`) to Dirty also sets the corresponding bit of `scrstatus` and `mcrstatus` (resp. `mcrstatus`). The ACES fields of `*status` are also set accordingly.

3.3.9. U-mode Status for HFI

This section is not normative.

A `ucrstatus` in the Unprivileged specification may become useful if the HFI extension is ratified, together with corresponding fields in a `ustatus` CSR. This may require the addition of another 32 bits worth of status to signal that the authoritative value of a CR is instead keep by U-mode itself, if such a mechanism is required.

3.3.10. `sacetrapmask`

Mask to determine which ACE exceptions vertically trap. Bit i set makes ACE error i+1 cause a trap.

3.3.11. `macecsk` Group

The optional `macecsk` group consists of XLEN-bit M-mode WARZ CSRs: eight in RV32, four in RV64.

A write to the i-th CSR sets bits $[(i+1)\cdot\text{XLEN}-1 : i\cdot\text{XLEN}]$ of the CSK.

Access outside M-mode raises an illegal instruction exception.

> M-mode firmware must be aware of whether the CSK is configurable, ephemeral, or fixed. Persistence mechanisms for configurable CSKs are implementation-specific. Transferring a CSK to another endpoint—such as the M-mode firmware of another device, a local TPM (for instance to preserve a CSK upon suspension or host hibernation), or a cloud-based HSM—requires an authenticated public-key key-wrapping mechanism.
>
> Alternative reconfiguration mechanisms (e.g., a Caliptra-style cite:[caliptra] secure IP block) are permitted.

3.3.12. `macePhysBootSecret` Group

The `macePhysBootSecret` group consists of M-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i-th CSR sets bits $[(i+1)\text{XLEN}-1 : i\cdot\text{XLEN}]$ of the Locality Secret PhysBootSecret (Table 4).

Access outside M-mode or Debug mode raises an illegal instruction exception.

3.3.13. `haceVirtBootSecret` Group

The `haceVirtBootSecret` group consists of H- and VS-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i-th CSR sets bits $[(i+1)\text{XLEN}-1 : i\cdot\text{XLEN}]$ of the Locality Secret VirtBootSecret (Table 4).

Access below VS-mode raises an illegal instruction exception.

3.3.14. `maceLocality` Group

The `maceLocality` group consists of M-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i-th CSR sets bits $[(i+1)\text{XLEN}-1 : i\cdot\text{XLEN}]$ of the Locality Secret MLocality (Table 4).

Access outside M-mode and Debug mode raises an illegal instruction exception.

3.3.15. `haceLocality` Group

The `haceLocality` group consists of H-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i-th CSR sets bits $[(i+1)\text{XLEN}-1 : i\cdot\text{XLEN}]$ of the Locality Secret HLocality (Table 4).

This group shall be implemented whenever the H extension is implemented. Access below H-mode raises an illegal instruction exception.

3.3.16. `saceLocality` Group

The `saceLocality` group consists of (V)S-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i-th CSR sets bits $[(i+1)\text{XLEN}-1 : i\cdot\text{XLEN}]$ of the Locality Secret SLocality (Table 4).

The group is accessible in S/HS-mode, VS-mode, M-mode, and Debug mode; access from any other mode raises an illegal instruction exception.

3.4. Trappable (Emulated) Operations

This section is currently non-normative.

Instructions `ace.prov`, `ace.export`, `ace.import`, `ace.input`, and `ace.output` support long inputs and outputs. To reduce hardware complexity, these operations can be trapped and emulated in software. Upon a trap, the handler identifies the trapped instruction and uses `ace.size` on the Metadata to determine the size of the data to be transferred. Then, the emulation uses custom instructions to move data between the ACE unit and the CPU. Three types of supporting operations are required:

- An "`ace.mv`" instruction that loads a specified number of bytes from a GPR or vector register into the state of a CR or in the input buffer within the ACE unit. Initially, `ace.size` returns the total payload length to transfer. After each move, this value is updated to reflect the remaining bytes. The instruction also updates the `acestart` CSR to follow the internal progress.
- A corresponding "`ace.mv`" instruction that exports a specified number of bytes from a CR or from the input buffer to a GPR or vector register. The behavior of `ace.size` will be similar to that of the `ace.mv` instruction, but it will count the number of bytes to be exported.
- An "`ace.close`" instruction to suspend or finalize the procedure. If executed before all bytes are transferred, the CR state matches that of an interrupted monolithic instruction, including any

special value of the *StateNumber* Metadata field and the value of `acestart`. If executed after all bytes are transferred, it also performs any final cryptographic operations, such as authenticating the CR.

All three instructions can be implemented as CSR RW instructions, for instance:

- `aceactivecr`, XLEN bits wide, determines what architectural state we are transferring to or from: 0..31 for the CRs and, say, 33 for the ACEIOBUF.
- `acestart`, XLEN bits wide, holds the offset within the CR's virtually serialized `content[]` or the ACEIOBUF.
- `acecontent` (new), XLEN bits wide, to which we write the content of the CR or the ACEIOBUF or from which we read the current content. A write to `acecontent` increases `acestart` by XLEN/8.

Interrupting the firmware implementation of `ace.prov`, `ace.export`, `ace.import`, `ace.input`, or `ace.output` must leave the architectural state indistinguishable from the state expected from interrupting a monolithic implementation of the instructions. The supporting instructions themselves must be uninterruptible.

Chapter 4. Pseudocode Examples

4.1. Electronic Codebook Modes (Direct Encryption/Decryption)

The following pseudocode illustrates how to use an ECB CC to encrypt a data using a block cipher. In this example, as with most other examples, we assume that the block cipher has a width of 128 bits (`IOLEN` is also 128).

Listing 5. Simple Example: Encryption with no Authentication

```
- Call OS API to get the necessary SCC. Assume X5 points to the SCC.
- Allocate memory for ciphertext buffer.

vsetvli zero, 4, e32, m1, ta, ma         // Use 128-bit vectors
ace.import K0, (X5)                      // Import the SCC into CR K0
ace.error X1
if (X1 != 0) then { handle error }

ace.state K0, #ace_state_encrypt         // Configure CR for encryption
ace.error X1
if (X1 != 0) then { handle error }

// assume plaintext in whole blocks
foreach(i from 0 to len_in_bytes(plaintext)/16-1) {
   V0 ← plaintext[16*i .. 16*i+15]
   ace.exec V0, K0, V0                   // Encrypt one block of plaintext
   ciphertext[16*i .. 16*i+15] ← V0
}
```

4.2. CTR/LFSR/XCTR/XLFSR Keystream Generation

Listing 6. Encrypt with Keystream Generation

```
// Acquire payload, initialize
- Info: plaintext = *PT
- Process gets a Keystream Generation CC from an API. X5 = pointer to the CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma         // Use 128-bit vectors
ace.import K0, (X5)

// The next operations set the nonce
V1 ← nonce
ace.state K0, #ace_state_operate, V1

// Compute keystream, use it to encrypt plaintext
```

```
foreach(i from 0 to len_in_bytes(PT)/16-1) {
   V0 ← PT[16*i .. 16*i+15]
   ace.exec V1, K0
   V1 ← V1 xor V0
   CT[16*i .. 16*i+15] ← V1
}
```

4.3. XEX/XTS modes (one or two keys)

We provide only encryption pseudocode, decryption is similar (also this example assumes `b` = `IOLEN` = 128).

Listing 7. Pseudocode for encryption in any IV-based non-authenticated mode without associated data

```
// Acquire payload and SCC, initialize
- Info: plaintext = *PT
- Call an API to get the corresponding SCC.  X5 = pointer to it.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma         // Use 128-bit vectors
ace.import K0, (X5)

V0 ← IV
ace.state K0, #ace_state_encrypt, V1     // Configure CR for encryption
foreach(i from 0 to ceil(len_in_bytes(PT)/16)-1) {
   V0 ← PT[16*i .. 16*i+15]
   ace.exec V0, K0, V0                   // Encrypt one block of plaintext
   CT[16*i .. 16*i+15] ← V0
}
```

To implement the XTS mode, the ciphertext stealing part is easy to realize using the XEX algorithm together with vector masking and shifting operations.

4.4. GCM Mode

In Listing 8 we show pseudocode for GCM encryption, again with `b` = `IOLEN` = 128, and `c` = 32). The code assumes that both AD and plaintext (or the ciphertext for the decryption) are zero-padded to a multiple of `b` bits, in this case 128 bits.

Listing 8. GCM Encryption

```
// Acquire payload, initialize
- Info: authenticated data = *AD, plaintext = *PT
- Process gets a GCM encryption CC from an API.  X5 = pointer to the CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma         // Use 128-bit vectors
ace.import K0, (X5)                      // Import, and compute the derived Auth Key
```

```
// The next operations set the nonce
ace.state K0, #ace_state_set_aux_value
V0 ← nonce
ace.exec K0, V0

// Compute AD contribution to Auth Data
ace.state K0, #ace_state_hash_absorb
foreach(i from 0 to len_in_bytes(AD)/16-1) {
   V0 ← AD[16*i .. 16*i+15]
   ace.exec K0, V0                      // Absorb it into hash
}

// Encrypt plaintext and continue tag computation
ace.state K0, #ace_state_encrypt
foreach(i from 0 to len_in_bytes(PT)/16-1) {
   V0 ← PT[16*i .. 16*i+15]
   ace.exec V1, K0, V0
   CT[16*i .. 16*i+15] ← V1
}

// Finally, we absorb the lengths into tag, finalize it, and output it
V1 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(PT), 64)
ace.state K0, #ace_state_enc_tag_finalize, V1
ace.exec V2, K0                         // V2 contains the finalized hash
```

GCM decryption differs from encryption as follows: the roles of `PT` and `CT` are reversed, states *encrypt* and *enc_tag_finalize* are replaced by states *decrypt* and *dec_tag_finalize*, and the last lines are as follows:

Listing 9. GCM Decryption

```
// Finally, we absorb the lengths into tag, finalize it, and verify it
ace.state K0, #ace_state_dec_tag_finalize
V1 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(CT), 64)
ace.exec V2, K0, V1                     // V2 contains the finalized hash

V1 ← tag_from_payload
ace.state K0, #ace_state_hash_verify, V1
ace.error X1
if (X1 != 0) then { handle error }      // Do not use the plaintext if error occurred
```

Listing 10. Implementing GCM encryption using an ECB CC

```
// Acquire payload, initialize
- Info: authenticated data = *AD, plaintext = *PT
- Process gets an ECB encryption CC from an API. Assume X5 is the pointer to the CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma
```

```
ace.import K0, (X5)
V5[127:32] ← IV
V5[31:0] ← zeros(31) @ 1

V3 ← zeros(b)                           // Auth Data 1
ace.exec V4, K0, V3                     // compute H
foreach(i from 0 to ceil(len_in_bytes(AD)/16)-1) {
   V1 ← AD[16*i .. 16*i+15]
   V3 ← V3 xor V1
   V3 ← Galoismul(V4, V3)
}

counter ← 1
foreach(i from 0 to ceil(len_in_bytes(PT)/16)-1) {
   V5[32:0] ← counter
   V1 ← PT[16*i .. 16*i+15]
   ace.exec V2, K0, V5                  // create mask by encrypting the counter
   V1 ← V1 xor V2                       // CTR encrypt
   V3 ← V3 xor V1
   V3 ← Galoismul(V4, V3)
   CT[16*i .. 16*i+15] ← V1
   counter ← counter + 1
}

V3 ← V3 xor (len_in_bits(AD) @ len_in_bits(PT))
V3 ← Galoismul(V4, V3)
V5[31:0] ← zeros(31) @ 1                // Prepare IV @ 1 for tag encryption
ace.exec V2, K0, V5
tag ← V3 xor V2
```

4.5. GCM-SIV Mode

We show in Listing 11 how to perform AEAD encryption with a GCM-SIV CC (with `b` = `IOLEN` = 128). Listing 12 is the corresponding pseudocode for decryption.

Listing 11. GCM-SIV Encryption

```
// Acquire payload, initialize
- Info: authenticated data = *AD
- Info: plaintext = *PT
- Call an API to get the corresponding key from a key management applet or HW block.
- The API returns the SCC. Assume X5 is the pointer to it.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma         // Use 128-bit vectors
ace.import K0, (X5)

// Set the nonce (and the derived Authentication Key).
ace.state K0, #ace_state_set_aux_value
V0 ← nonce
```

```
ace.exec K0, V0

foreach(i from 0 to len_in_bytes(AD)/16-1) {
   V0 ← AD[16*i .. 16*i+15]              // Load a block of associated data
   ace.exec K0, V0                       // Absorb it into hash
}

foreach(i from 0 to len_in_bytes(PT)/16-1) {
   V0 ← PT[16*i .. 16*i+15]              // Load a block of plaintext
   ace.exec K0, V0                       // Absorb it into hash
}

V0 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(PT), 64)
ace.exec K0, V0                          // Absorb lengths into hash

// Finalize the SIV computation, set and return it
ace.state K0, #ace_state_enc_tag_finalize
ace.exec V2,K0
tag ← V2

// Encrypt the plaintext
ace.state K0, #ace_state_encrypt
foreach(i from 0 to len_in_bytes(PT)/16-1) {
   V0 ← PT[16*i .. 16*i+15]              // Load a block of plaintext
   ace.exec V1, K0, V0                   // Encrypt the block
   CT[16*i .. 16*i+15] ← V1              // Store a block of ciphertext
}
```

Listing 12. GCM-SIV Decryption

```
// Acquire payload, initialize
- Info: authenticated data = *AD
- Info: ciphertext = *CT
- Call an API to get the corresponding key from a key management entity.
- The API returns the SCC. Assume X5 is the pointer to it.
- Allocate memory for plaintext buffer PT (if not same as CT).

vsetvli zero, 4, e32, m1, ta, ma         // Use 128-bit vectors
ace.import K0, (X5)

// The next operations set the nonce and SIV.
V0 ← nonce
ace.state K0, #ace_state_set_aux_value, V0

// Compute AD contribution to Auth Data
V0 ← SIV
ace.state K0, #ace_state_hash_absorb, V0
foreach(i from 0 to len_in_bytes(AD)/16-1) {
   V0 ← AD[16*i .. 16*i+15]
   ace.exec K0, V0                       // Absorb it into hash
```

```
}

// Decrypt ciphertext and continue computation of tag
foreach(i from 0 to len_in_bytes(CT)/16-1) {
   V0 ← CT[16*i .. 16*i+15]
   ace.exec V1, K0, V0
   PT[16*i .. 16*i+15] ← V1
}

// Finally, we absorb the lengths of AD and plaintext into tag AND verify
ace.state K0, #ace_state_enc_tag_finalize
V0 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(PT), 64)
ace.exec K0, V0
ace.error X1
if (X1 != 0) then { handle error }       // Do not use the plaintext if error
```

4.6. OCB Mode

In this section we show pseudocode for OCB encryption and decryption (`b` = `IOLEN` = 128).

Listing 13. OCB Encryption

```
// Acquire payload, initialize
- Info:
 - authenticated data = *AD with bit length AD_len
 - plaintext = *PT with bit length PT_len
 - nonce N with bit length N_len
 - tag length tag_len

- Process gets an OCB encryption CC from an API.  X5 = pointer to the CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero,4,e32,m1,ta,ma              // Use 128-bit vectors
ace.import K0, (X5)

// The next operations set the nonce
X1 ← N_len
ace.state K0, #ace_state_set_aux_value, X1
V0 ← N
ace.exec K0, V0

// The next state change also sets the tag length
X1 ← tag_len
ace.state K0, #ace_state_hash_absorb, X1

AD_fullblocks ← floor(AD_len/128)
PT_fullblocks ← floor(PT_len/128)

// Compute AD contribution to tag
foreach(i from 0 to AD_fullblocks-1) {
```

```
   V0 ← AD[16*i .. 16*i+15]
   ace.exec K0, V0                       // Absorb it into hash
}
X1 ← AD_len mod 128                      // Can be zero
ace.state K0, #ace_state_hash_finalize, X1
if (X1 mod 128 != 0) then {
   V0 ← zeros(128)
   V0 ← AD[AD_fullblocks*16 .. AD_fullblocks*16+ceil(X1/8)]
   ace.exec K0, V0                       // Absorb it into hash and encrypt
}

// Encrypt plaintext and continue computation of tag
ace.state K0, #ace_state_encrypt
foreach(i from 0 to PT_fullblocks-1) {
   V0 ← PT[16*i .. 16*i+15]
   ace.exec V1, K0, V0
   CT[16*i .. 16*i+15] ← V1
}
X1 ← PT_len mod 128                      // Can be zero
ace.state K0, #ace_state_last_block, X1
if (X1 mod 128 != 0) then {
   V0 ← zeros(128)
   V0 ← PT[PT_fullblocks*16 .. PT_fullblocks*16+ceil(X1/8)]
   ace.exec V1, K0, V0                   // Absorb it into hash and encrypt
   CT[PT_fullblocks*16 .. PT_fullblocks*16+ceil(X1/8)] ← V1
} else
   ace.exec K0

ace.exec V1, K0                          // output tag
```

OCB decryption differs from encryption as follows: the roles of `PT` and `CT` are reversed, states *encrypt* and *enc_tag_finalize* are replaced by states *decrypt* and *dec_tag_finalize*, and the last lines are as follows:

Listing 14. OCB Decryption

```
V1 ← tag
ace.state K0, #ace_hash_verify, V1
ace.error X1
if (X1 != 0) then { handle error }       // do not use the plaintext if error
```

4.7. CMAC Functions

Listing 15. CMAC Computation

```
- Info: CMAC input = *M with bit length M_len
- Process gets a CMAC CC from an API. Assume X5 is the pointer to the CC.

vsetvli zero, 4, e32, m1, ta, ma         // Use 128-bit vectors
```

```
ace.import K0, (X5)

ace.state K0, #ace_hash_absorb
blocks = floor(M_len/128)
remainder = M_len % 128
if remainder == 0 then
   blocks ← blocks - 1
   remainder ← 128

foreach(i from 0 to blocks-1) {
   V0 ← M[16*i .. 16*i+15]
   ace.exec K0, V0
}

X1 ← remainder
ace.state K0, #ace_hash_last_block, X1
V0 ← M[16*blocks .. 16*blocks+15]
ace.exec K0, V0

ace.state K0, #ace_hash_hash_finalize
ace.exec V0, K0  // this is the value of the CMAC
```

Chapter 5. Annexes

5.1. Programmable CSKs vs. ID-based Usage Control vs. Localities

This section is informative and non-normative.

ACE provides a layer of usage control based on the use of Localities.

- One option is to add an optional Metadata Extension block that contains values of `scontext` and `mcontext` (from Sdtrig) to match against, or alternative values based on ad hoc CSRs.

  This is a lightweight mechanism. Since filtering at one mode is inactive when the hart is in a higher mode, context switching can be performed without modifying the CSK.

  1. A programmable CSK can restrict not only the usage, but even the import of CCs. This offers stronger security by also preventing unauthorized `ace.import` operations, but incurs higher overhead as it typically requires an `ecall` to modify the CSK, and increases pressure on the ACE unit's internal resources in case keys derived from the most recent CSK values are cached.
  2. Localities programmable at M/H/S could be used instead. Since these Localities can be combined with the HW Binding localities, this would be as flexible as context-based filtering, but based on significantly longer strings.

The envisioned usage model employs context-based filtering at the User level, and also at the (Virtual) Supervisor level in high-performance systems where migration is not required (e.g., automotive applications). Cryptographic domain separation via the CSK or Localities is intended for Virtual Machines, Supervisor Domains, Worlds, and TEEs, as these demand stronger security guarantees.

Although the architecture does not prevent assigning unique CSKs to U-mode processes, doing so would impose a substantial burden on system software due to per-context-switch `ecall` overhead. Hardware support—such as automatic CSK switching based on contexts could alleviate this burden. Such a mechanism is not under consideration due to its architectural complexity, inflexibility, and hardware cost. For example, it would necessitate secure management of large CSK tables, including saving, restoring, and potentially migrating entire CSK blocks.

5.2. Architectural Model

ACE is architected as an *attached* unit. This allows implementations both within each core and as a block shared by multiple cores, with a single instance of the hardware and one separate architectural state for each hart. The present specification mandates only its behavior, not its implementation.

Consequently, ACE instructions behave as messages over a tightly coupled coprocessor interface. The ACE unit accepts operations, executes them independently of the issuing hart, and later returns results. To enforce this decoupling, the ACE unit may lock certain architectural states, for instance

Context Registers and CSRs, causing the hart to wait when attempting to read them.

This design permits a certain degree of concurrency: ACE operations may proceed even while the control flow of the hart is preempted, and long-running cryptographic computations are allowed. Certain interrupts, however, may be permitted to suspend ACE operations.

For example, let us consider a U-mode process executing an ACE instruction that modifies only internal state, say, a vector register. If the process is preempted, the ACE unit may continue its work until the context-switch handler attempts to access that register, at which point the hart would wait for the operation to finish. Alternatively, an implementation may adopt a more precise exception model: halting the ACE unit, setting `acestart` to allow later resumption. Optionally the implementation may impose a timeout before halting precisely. The architecture permits all these approaches and fully specifies the behavior of ACE instructions when interrupted at admissible points.

5.3. Threat Model

This section is informative and non-normative.

The ACE threat model is straightforward.

Assets consist of *keys* and any other information that could aid an adversary in recovering plaintext or forging ciphertext. This includes items such as masks in tweakable XEX modes and derived keys in AES-GCM-SIV. The critical properties of these assets are their *confidentiality* and *integrity*. Protecting the *Availability* of the keys is not a goal of the architecture.

Adversaries are assumed capable of compromising processes by exploiting software vulnerabilities. They are also able to gain access to memory contents, for instance, by abusing hardware interfaces or by SoC/memory interposition. The model does not include adversaries capable of sophisticated hardware attacks, such as using a Focused Ion Beam/Scanning Electron Microscope (FIB/SEM) to tamper with the hardware's functionality.

Side channel attacks are addressed optionally. ACE offers interfaces to both SCA-unprotected and SCA-protected implementations of cryptographic algorithms, if these options are provided by the designer.

Threats can be addressed in four ways:

C: *Control* the threat by implementing full or partial mitigations.

A: *Accept* the threat and its associated risks.

T: *Transfer* the threat to another party better suited to address it.

S: *Suppress* the features that give rise to the threat.

ACE is designed to avoid the "**A**" and "**S**" options (as much as possible). It *controls* threats through its own mechanisms and *transfers* residual risk—specifically, the risk of key extraction from the privileged or trusted software and hardware environments that manage keys—to those same environments. For example, trusted software environments may employ control-flow integrity,

memory encryption with integrity protection, and anti-replay mechanisms, while hardware key-management blocks can be hardened to protect entrusted keys.

Direct plaintext leakage, memory corruption attacks against the ACE-using process, and Release of Unverified Plaintext (RUP) attacks on the cryptographic algorithms are considered out of scope. These threats cannot be mitigated by ACE because the processed information resides within the user execution context. Consequently, these threats are *accepted*.

Index

A

ACE
- algorithm
  - Ascon, 77
  - CMAC, 70
  - ECB, 50
  - GCM, 57
  - general hashing, 72
  - invalid state, 44
  - keystream generation, 52
  - MAC functions, 72
  - SHA-3, 76
  - threshold variants, 48
  - XEX, 54
  - XOF, 72
  - XTS, 99
- error codes, 22
- instruction
  - ace.avail, 33
  - ace.clear, 26
  - ace.clone, 35
  - ace.error, 31
  - ace.exec, 29
  - ace.export, 27
  - ace.import, 27
  - ace.import.S, 27
  - ace.input, 36
  - ace.output, 37
  - ace.prov, 25
  - ace.reset, 26
  - ace.restrict, 33
  - ace.size, 31
  - ace.state, 30
- trappable instruction, 96

C

Context Register, 13
- invalidation, 44

Context Register File, 15

Cryptographic Context, 13, 15
- access, 13
- Content, 13
- export, 13
- Locality, 14
- management operations, 13
- Metadata, 13, 17
  - Header, 17
- object-oriented, 13
- Sealed, 38
- sealed, 13
- usage control, 13
- usage operations, 13

CSR
- ACE field in `misa`, 92
- `aceactivecr`, 24
- `aceerror`, 21
- `acehtrapmask`, 24
- `acehtrapvector`, 24
- `aceiobuflen`, 23
- `aceiobuftop`, 23
- `acemarchid`, 22
- ACES field in mstatus, 92
- ACES field in sstatus, 92
- ACES field in vsstatus, 92
- `acesequence`, 23
- `acestart`, 22
- `haceLocality`, 96
- `haceVirtBootSecret`, 95
- `macecsk`, 95
- `maceLocality`, 96
- `macePhysBootSecret`, 95
- `mcrstatus`, 94
- `saceLocality`, 96
- `sacetrapmask`, 95
- `scrstatus`, 93
- `vscrstatus`, 93

D

design
- high performance, 9
- scalable, 10

R

RVV-mini, 15

V

VM migration, 10

Bibliography

bibliography::[]